

第七章 图

7.1 图的基本概念

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性问题

7.5 有向无环图及其应用

7.6 最短路径

本章学习要点

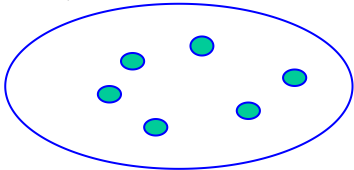
逻辑结构

回顾

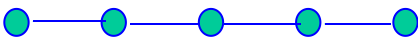
方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

划分方法一

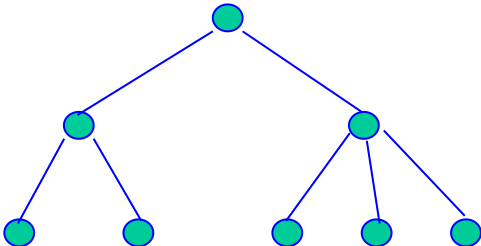
集合——数据元素间除“同属于一个集合”外，无其它关系
如，一堆沙子



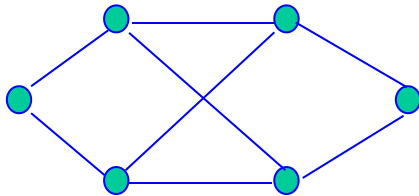
线性结构——一个对一个，如线性表、栈、队列
如，一根链条，一个单词中的所有字母



树形结构——一个对多个，如树
如，一棵树，家谱树等



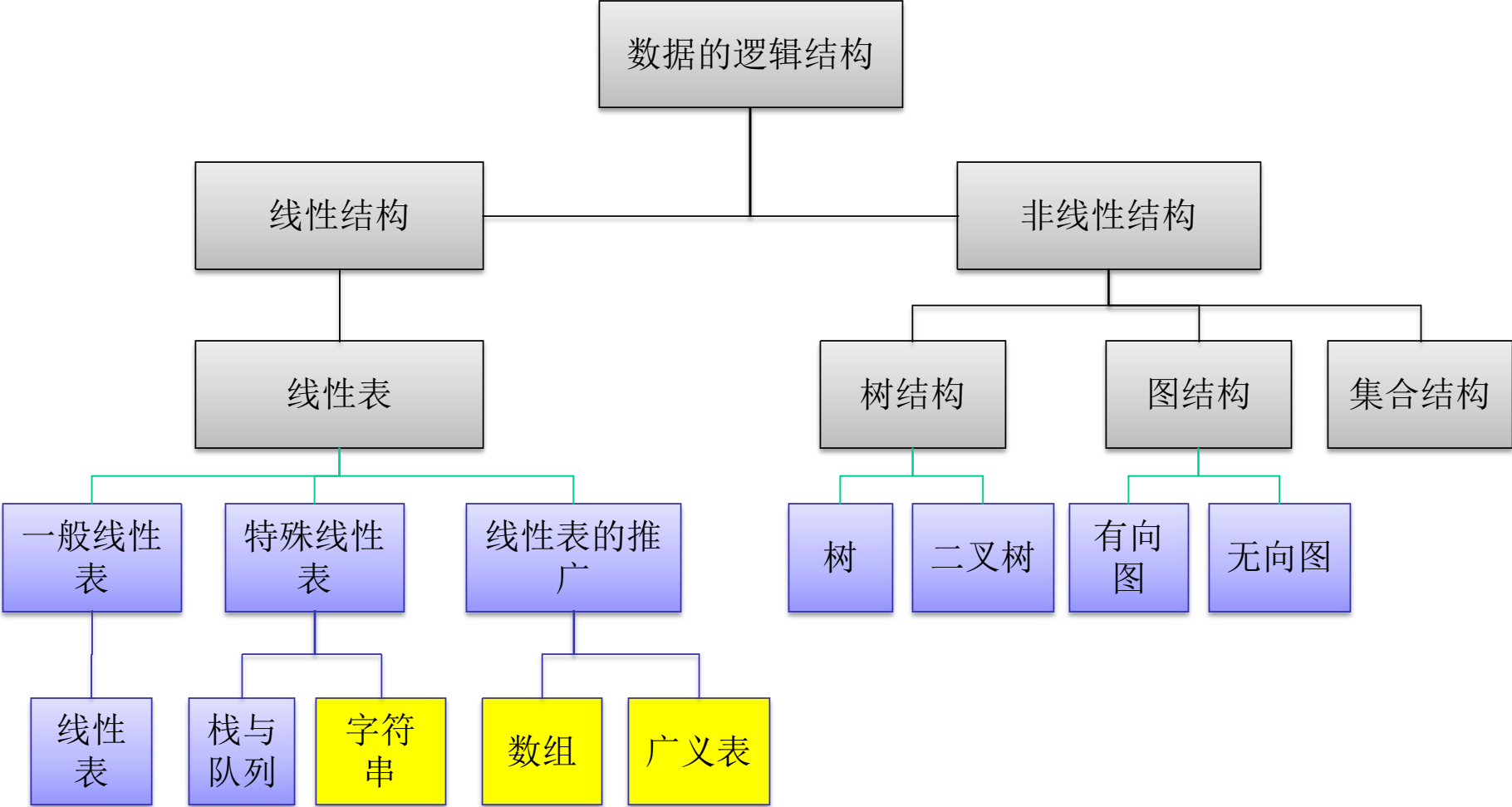
图形结构——多个对多个，如图
如，交通图，高铁图



逻辑结构总结

回顾

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	



存储结构

回顾

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

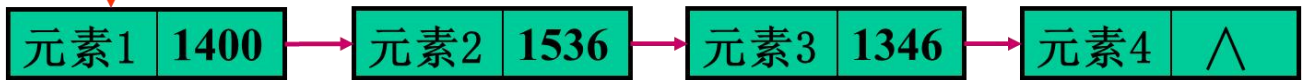
顺序存储结构——

借助元素在存储器中的**相对位置**
表示数据元素间的逻辑关系（如前后关系）
相对位置可通过计算得到

存储地址	存储内容
L_0	元素1
L_0+m	元素2
	
$L_0+(i-1)*m$	元素i
	
$L_0+(n-1)*m$	元素n

链式存储结构——

借助指示元素存储地址的**指针**
表示数据元素间的逻辑关系（如前后关系），
需通过遍历（依次访问）链表得到指定元素的存储位置



7.1 图的基本概念

非线性结构，数据元素之间呈多对多的关系。

将数据元素作为顶点，数据元素间的关系作为顶点间的连接，即由**顶点集合V**和一个描述顶点之间关系---边的**集合VR(或E)**组成。

7.1.1 图的定义

$$\mathbf{Graph}=(V,VR)$$

或

$$\mathbf{Graph}=(V,E)$$

V：顶点(数据元素)的有穷非空集合。

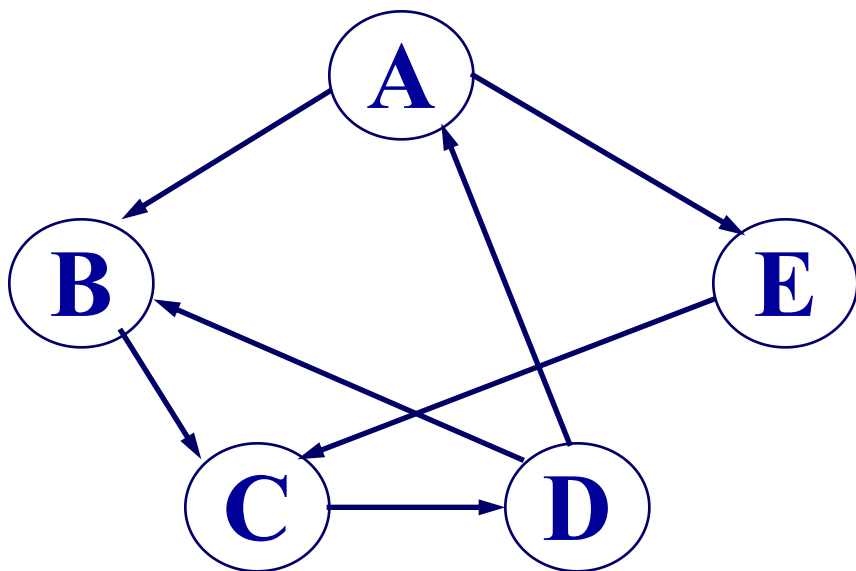
VR(或E)：弧(关系)的有穷集合。

有向图

例1: $G_1 = (V_1, VR_1)$

$V_1 = \{A, B, C, D, E\}$

$VR_1 = \{\langle A, B \rangle, \langle A, E \rangle,$
 $\langle B, C \rangle, \langle C, D \rangle, \langle D, B \rangle,$
 $\langle D, A \rangle, \langle E, C \rangle \}$

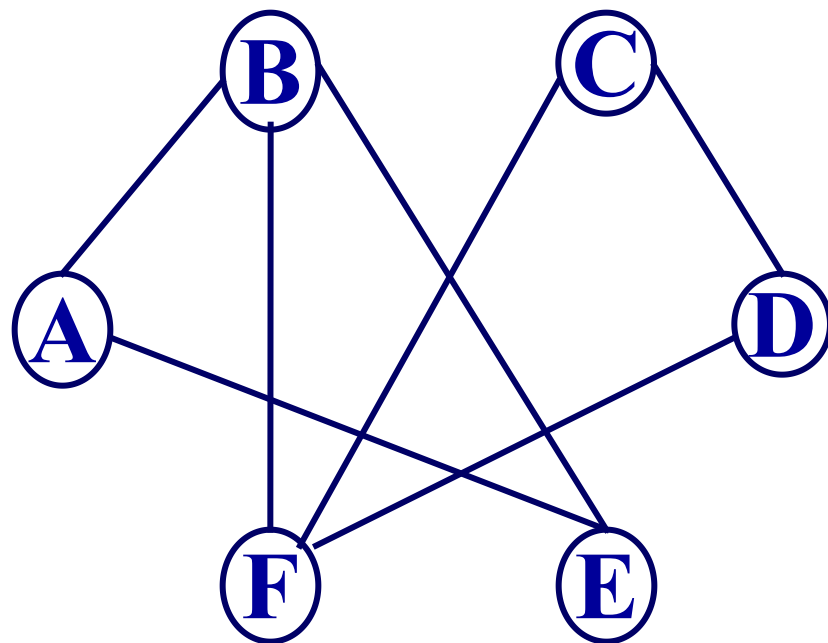


无向图

例2: $G_2 = (V_2, VR_2)$

$V_2 = \{A, B, C, D, E, F\}$

$VR_2 = \{(A, B), (A, E),$
 $(B, E), (C, D), (D, F),$
 $(B, F), (C, F) \}$



7.1.2 图的相关术语

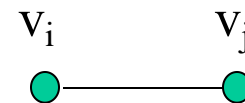
顶点 数据元素所构成的结点。

有向图 弧的顶点偶对是有序的。对弧 $\langle v_i, v_j \rangle$ 而言， v_i 是**弧尾**/初始点； v_j 是**弧头**/终端点。



无向图 弧的顶点偶对是无序的。

(v_i, v_j) 和 (v_j, v_i) 代表同一条**边**($i \neq j$)。



(无向)完全图 每个顶点与其余顶点都有边的无向图。

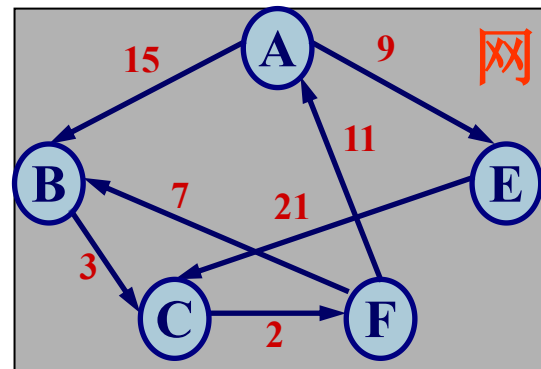
顶点数为 n 时，边数 $e = n(n-1)/2$

有向完全图 每个顶点与其余顶点都有弧的有向图。

顶点数为 n 时，弧数 $e = n(n-1)$

稀疏图 有很少边或弧的图。 ($e < n \log n$)

稠密图 有较多边或弧的图。



权 图中的边或弧具有一定的大小的概念。

网 边/弧带权的图。

邻接 有边/弧相连的两个顶点之间的关系。

存在 (v_i, v_j) ，则称 v_i 和 v_j 互为**邻接点**；

存在 $\langle v_i, v_j \rangle$ ，则称 v_i **邻接到** v_j ， v_j **邻接于** v_i

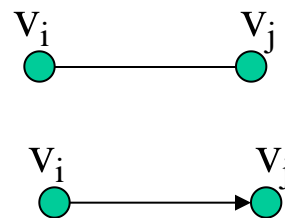
关联(依附) 边/弧与顶点之间的关系。

存在 $(v_i, v_j)/\langle v_i, v_j \rangle$ ，则称该边/弧关联于 v_i 和 v_j

顶点的度 与该顶点相关联的边的数目，记为 $D(v)$ 。

入度 $ID(v)$ ：有向图中，以该顶点为弧头的弧数目。

出度 $OD(v)$ ：有向图中，以该顶点为弧尾的弧数目。

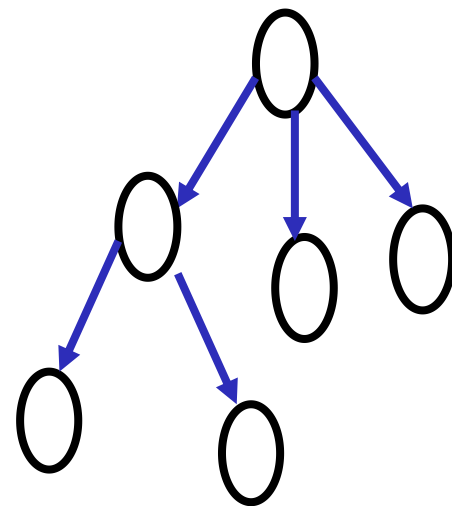


无向图中顶点数 n 、边数 e 和度数之间的关系：
$$e = \frac{1}{2} \sum_{i=1}^n D(v_i)$$

有向图中顶点数 n 、弧数 e 和度数之间的关系：
$$e = \sum_{i=1}^n OD(v_i) = \sum_{i=1}^n ID(v_i)$$

问：当有向图中仅1个顶点的入度为0, 其余顶点的入度均为1, 此时是何形状?

答：是树的形状。



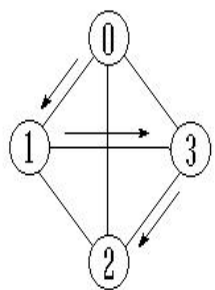
路径 接续的边构成的顶点序列。

路径长度 路径上边或弧的数目/权值之和。

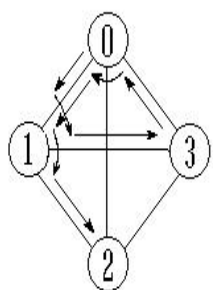
回路(环) 第一个顶点和最后一个顶点相同的路径。

简单路径 序列中顶点均不相同的路径。

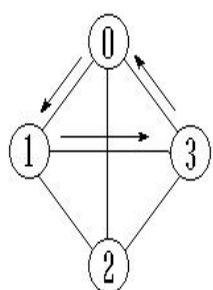
简单回路(简单环) 除路径起点和终点相同外，其余顶点均不相同的路径。



(a) 简单路径

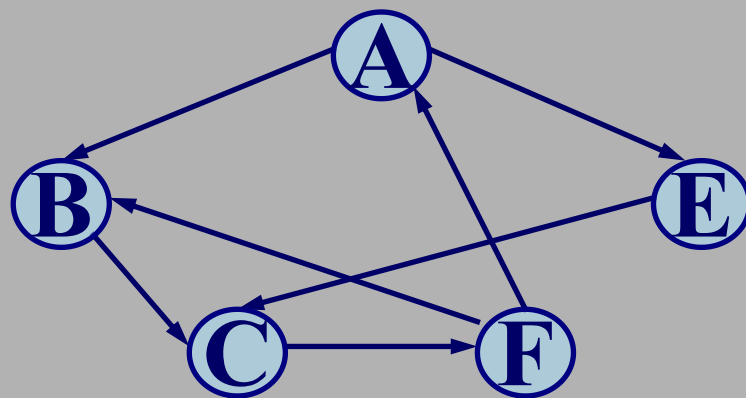


(b) 非简单路径



(c) 回路

从A到F长度为3的路径
 $\{A, B, C, F\}$

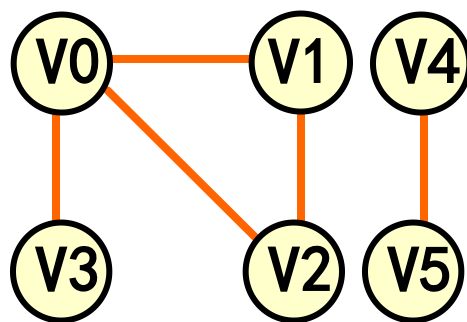
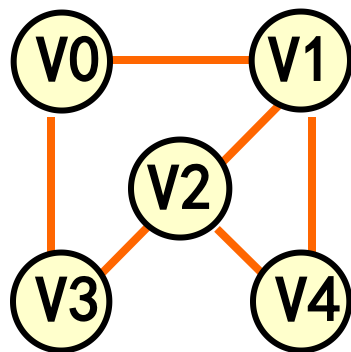


连通图（强连通图）

在**无向图** $G=(V, \{E\})$ 中，若对任何两个顶点 v 、 u 都存在从 v 到 u 的路径，则称 G 是**连通图**。

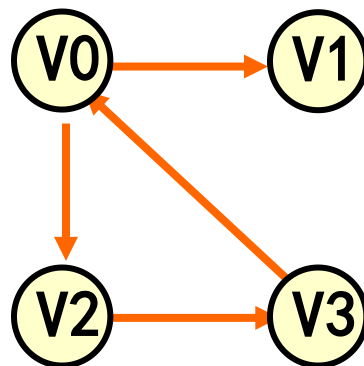
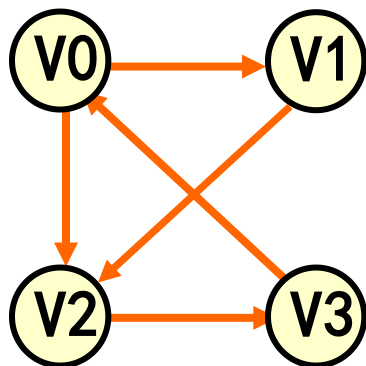
在**有向图** $G=(V, \{E\})$ 中，若对任何两个顶点 v 、 u 都存在从 v 到 u 的路径，则称 G 是**强连通图**。

连通图



非连通图

强连通图



非强连通图

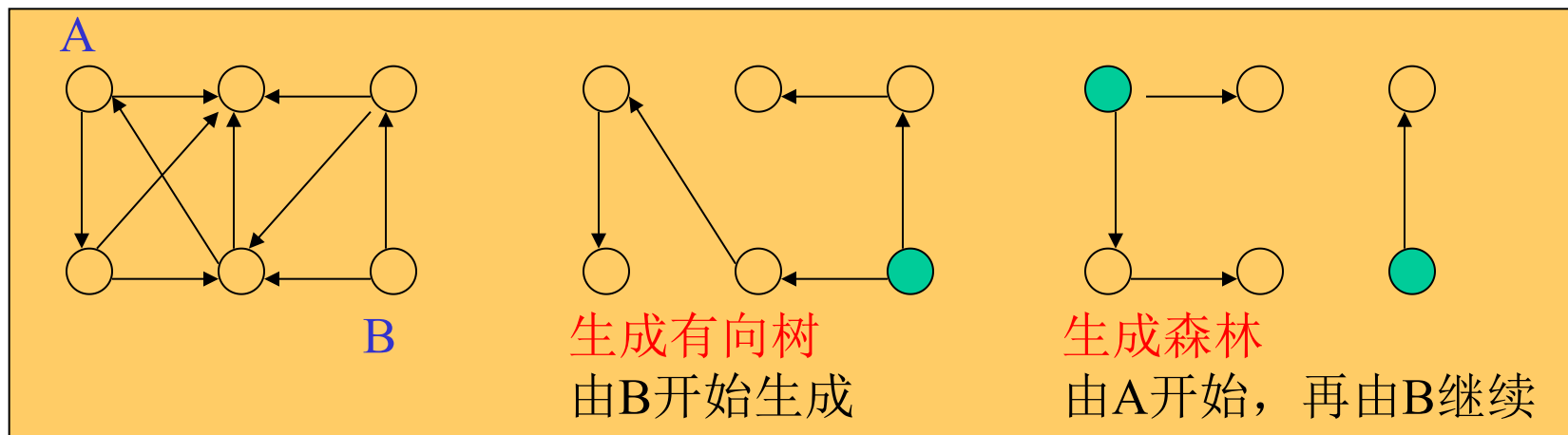
子图 对于图 $G=(V,E)$ 和 $G'=(V',E')$, 如果 $V'\subseteq V$, $E'\subseteq E$, 且 E' 关联的顶点都在 V' 中, 则称 G' 是 G 的子图。

生成子图 由图的全部顶点和部分边组成的子图称为原图的生成子图。

生成树 包含图中全部顶点的极小连通子图。

生成有向树 图中恰有一个顶点入度为0, 其余顶点入度均为1。

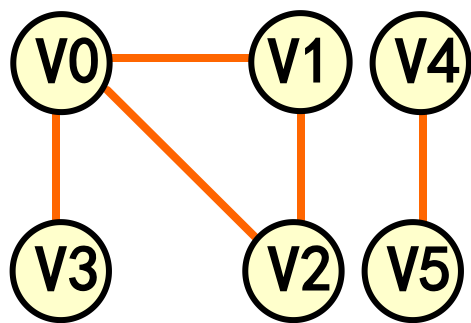
生成森林 有向图中, 包含所有顶点的若干棵有向树构成的子图。



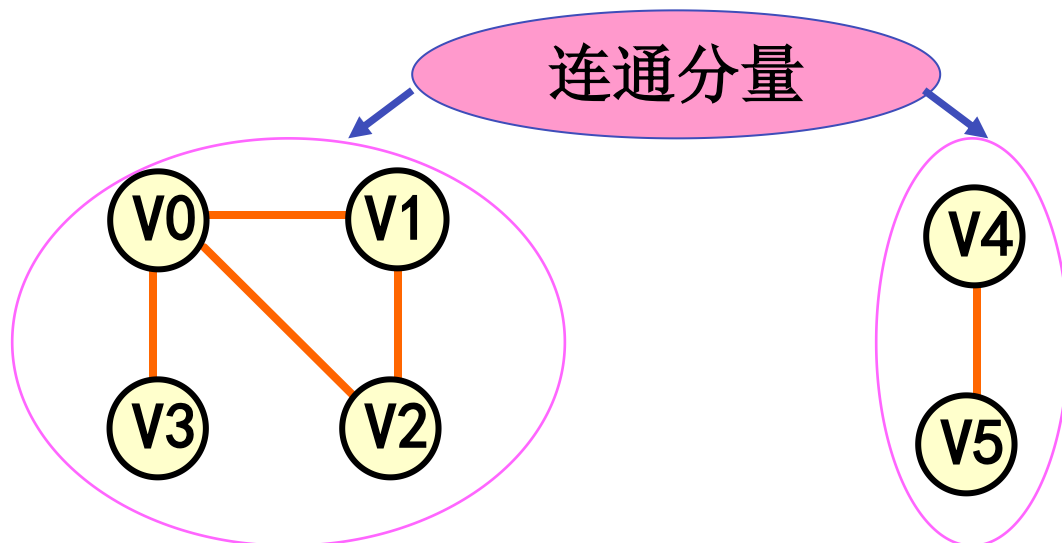
连通分量

无向图G 的**极大连通子图**称为G的**连通分量**。

极大连通子图意思是：该子图是 G 连通子图，将G 的任何不在该子图中的顶点加入，子图不再连通。

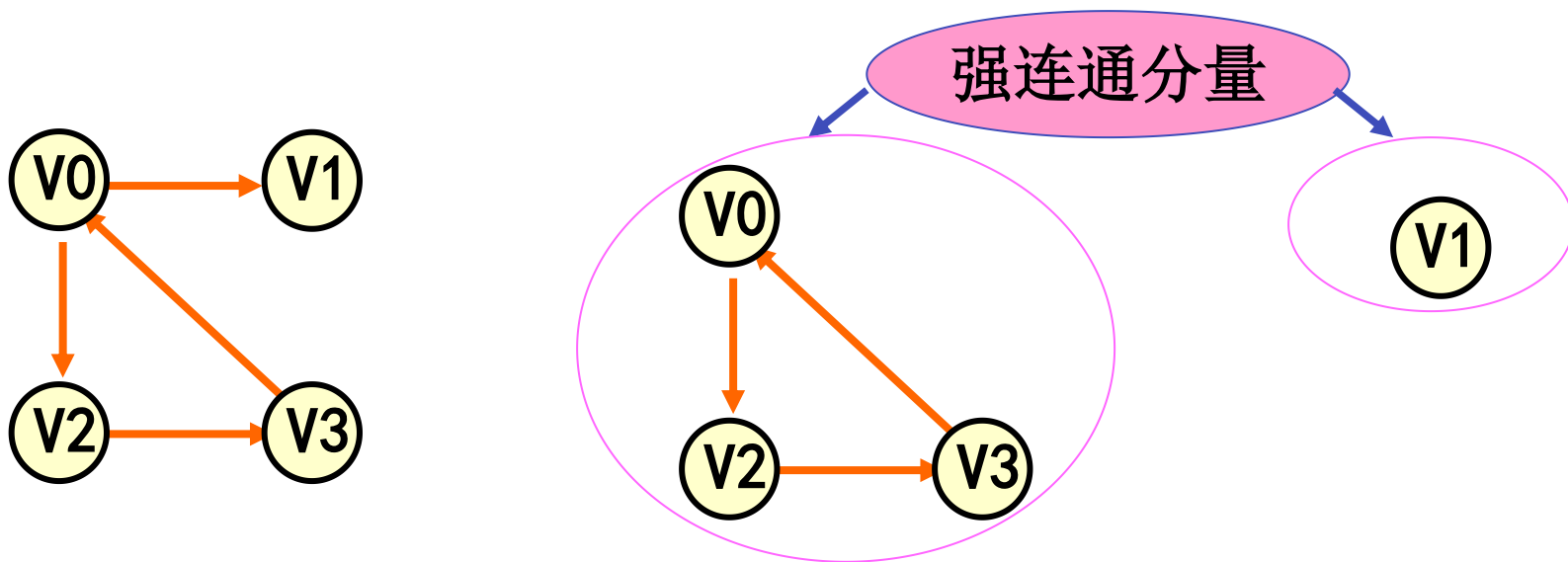


非连通图



强连通分量

有向图G 的**极大强连通子图**称为G的**强连通分量**。极大强连通子图意思是：该子图是G的强连通子图，将G的任何不在该子图中的顶点加入，子图不再是强连通的。

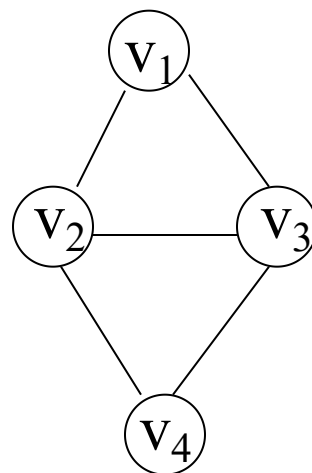


7.1.3 图的基本操作

参数中v、w为顶点编号

- | | |
|-------------|---------------------------------------|
| (1)图的生成 | <code>CreatGraph(&G, V,VR)</code> |
| (2)销毁图 | <code>DestroyGraph(&G)</code> |
| (3)顶点定位 | <code>LocateVex(G,value)</code> |
| (4)取顶点数据 | <code>Getvex(G,v)</code> |
| (5)对顶点赋值 | <code>Putvex(&G, v, value)</code> |
| (6)求第一个邻接顶点 | <code>FirstAdjVex(G,v)</code> |
| (7)求下一个邻接顶点 | <code>NextAdjVex(G,v,w)</code> |
| (8)插入顶点 | <code>InsertVex(&G,v)</code> |
| (9)删除顶点 | <code>DeleteVex(&G,v)</code> |
| (10)插入边/弧 | <code>InsertArc(&G,v,w)</code> |
| (11)删除边/弧 | <code>DeleteArc(&G,v,w)</code> |
| (12)图的遍历 | <code>Traverse(G)</code> |

为指定图中数据元素方便起见，逻辑上通常首先将顶点编号



图的存储结构

1、顺序存储结构：数组表示法（邻接矩阵）

❖ 建立一个**顶点表**（记录各个顶点信息）和一个**邻接矩阵**（表示各个顶点之间关系）。

❖ 设图 $A = (V, E)$ 有 n 个顶点，则图的邻接矩阵是一个二维数组 $A.Edge[n][n]$ ，定义为：

$$A.Edge[i][j] = \begin{cases} 1, & \text{如果 } \langle i, j \rangle \in E \text{ 或者 } (i, j) \in E \\ 0, & \text{否则} \end{cases}$$

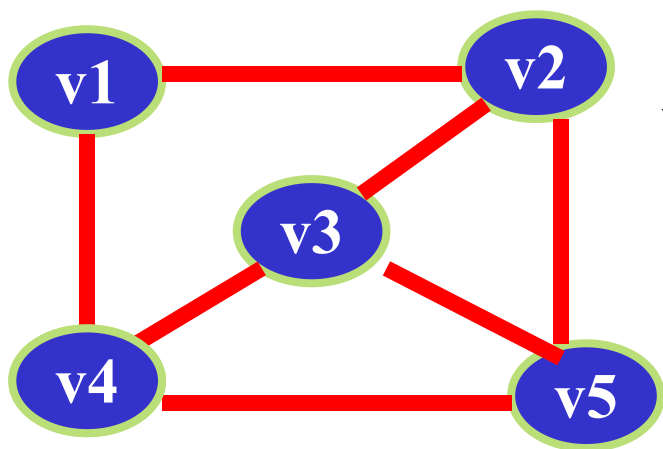
有向图

无向图

2、链式存储结构：多重链表 ➡

邻接表
邻接多重表
十字链表

无向图的邻接矩阵表示法



顶点表: (v1 v2 v3 v4 v5)

邻接矩阵:

$A_{Edge} =$

0	1	0	1	0
1	0	1	0	1
0	1	0	1	1
1	0	1	0	1
0	1	1	1	0

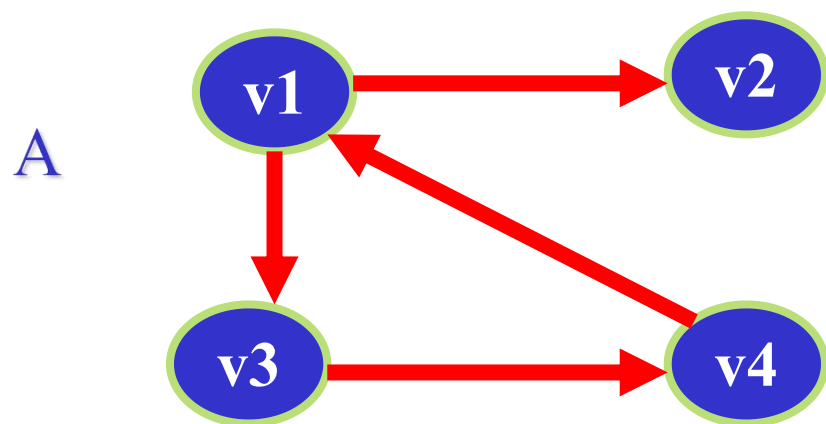
v1 v2 v3 v4 v5

分析1: 无向图的邻接矩阵是**对称**的;

分析2: 顶点 i 的**度** = 第 i 行 (或列) 中**1** 的个数;

特别: **完全图**的邻接矩阵中, 对角元素为0, 其余1。

有向图的邻接矩阵表示法



顶点表: (v1 v2 v3 v4)

邻接矩阵:

$$A.Edge = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{matrix} v1 \\ v2 \\ v3 \\ v4 \end{matrix}$$

v1 v2 v3 v4

注: 在有向图的邻接矩阵中,
第*i*行含义: 以结点 v_i 为尾的弧(即出度边);
第*i*列含义: 以结点 v_i 为头的弧(即入度边)。

分析1: 有向图的邻接矩阵可能是不对称的。

分析2: 顶点 v_i 的出度=第*i*行元素之和

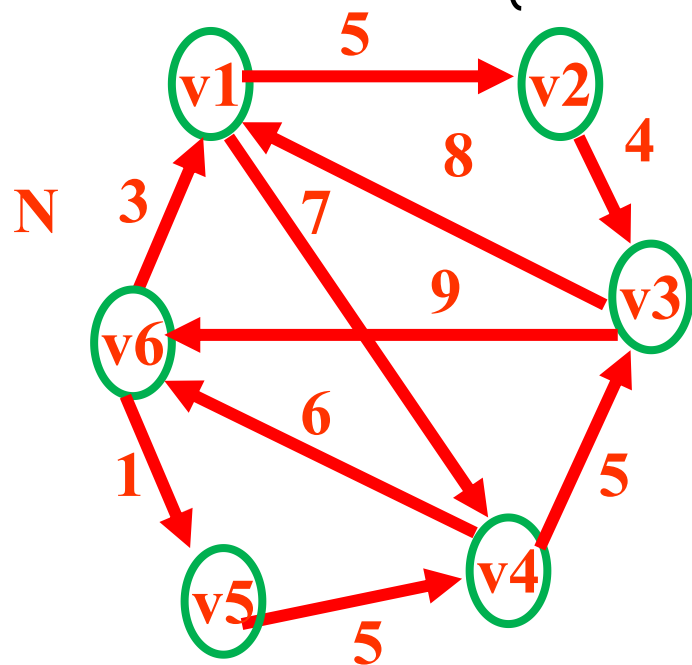
顶点 v_i 的入度=第*i*列元素之和

顶点 v_i 的度=第*i*行元素之和+第*i*列元素之和

网（即有权图）的邻接矩阵表示法

定义为：

$$N.Edge[i][j] = \begin{cases} W_{ij} & \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in VR \\ \infty & \text{无边（弧）} \end{cases}$$



顶点表： (v1 v2 v3 v4 v5 v6)

邻接矩阵：

$N.Edge =$

∞	5	∞	7	∞	∞
∞	∞	4	∞	∞	∞
8	∞	∞	∞	∞	9
∞	∞	5	∞	∞	6
∞	∞	∞	5	∞	∞
3	∞	∞	∞	1	∞

[邻接矩阵的存储结构定义]

```
#define INFINITY          INT_MAX          // 最大值 $\infty$ 
#define MAX_VERTEX_NUM    20              // 最大顶点个数
typedef enum{DG, DN, UDG, UDN} GraphKind;
    //有向图、有向网、无向图、无向网，四种图类型
typedef int VertexType;  //顶点携带信息的类型，假设为整数
typedef struct  ArcCell  {
    VRType      adj;  // 顶点关系类型，对无权图用0或1在矩阵中
    //的行号和列号对应的顶点是否相邻。对带权图，则为边或弧的权值。
    InfoType    * info; // 该弧的相关信息指针
}ArcCell, AdjMatrix[MAX_VERTEX_NUM ][MAX_VERTEX_NUM ];
typedef struct  {
    VertexType  vexs[MAX_VERTEX_NUM];  // 顶点向量
    AdjMatrix   arcs;                  // 邻接矩阵
    int         vexnum, arcnum;         // 图的当前顶点数和弧数
    GraphKind   kind;                  // 图的种类标志
}MGraph;
```

图的数组/邻接矩阵 表示法(Adjacency Matrix) 顺序存储方式

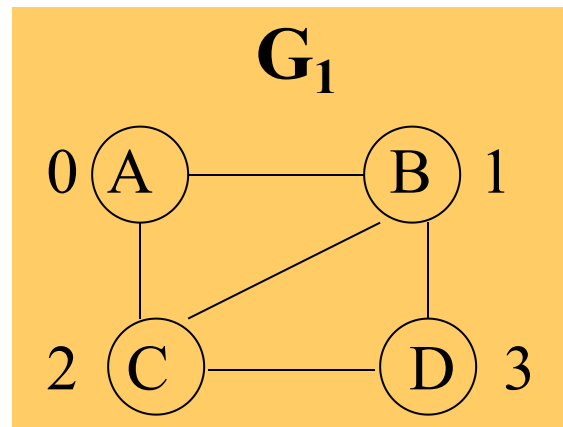
[例1] 无向图

顶点数组 **vexs**

0	A
1	B
2	C
3	D

邻接矩阵(边表) **arcs**

0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0
	0	1	2	3



无向图的邻接矩阵具有对称性

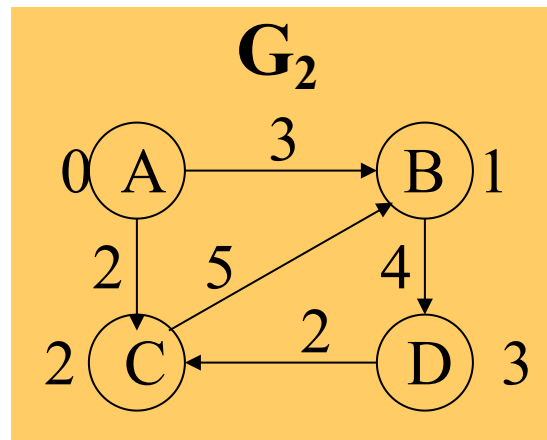
[例2] 有向网

顶点数组 **vexs**

0	A
1	B
2	C
3	D

邻接矩阵(边表) **arcs**

0	0	3	2	∞
1	∞	0	∞	4
2	∞	5	0	∞
3	∞	∞	2	0
	0	1	2	3



邻接矩阵表示法的特点

优点：容易实现图的操作，如：求某顶点的度、判断顶点之间是否有边、找顶点的邻接点等等。

缺点： n 个顶点需要 $n*n$ 个单元存储边；空间效率为 $O(n^2)$ 。

对稀疏图而言尤其浪费空间。

[算法示例] 建立无向网的数组型存储结构

步骤:

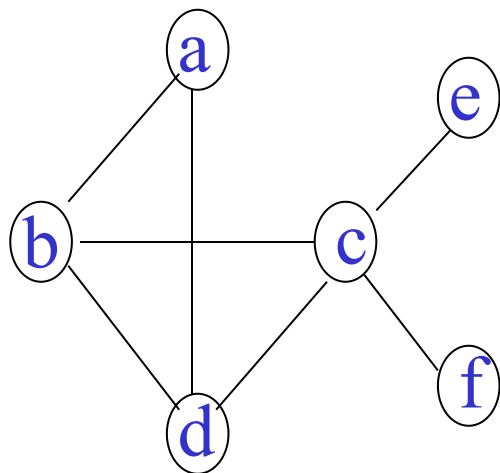
键盘输入:

```
4 5
A B C D
0 1
0 2
0 3
1 2
2 3
```

- 1) scanf(顶点数 n , 边数 e) $O(1)$
- 2) 依次读入每个顶点数据, 填入顶点表 $vexs$ $O(n)$
- 3) 邻接矩阵 $arcs$ 初始化:
全部置 ∞ ; 主对角线置0; $O(n^2)$
- 4) 建立邻接矩阵: $O(e)$
 - 4.1) 读入一条边的两个顶点编号 i 、 j 和权值 w
 - 4.2) 置 $arcs[i][j]=w$, $arcs[j][i]=w$若尚有未读入的边, 转4.1)

$O(n^2)$

邻接表(Adjacency List)是图的一种顺序存储与链式存储结合的存储方法。



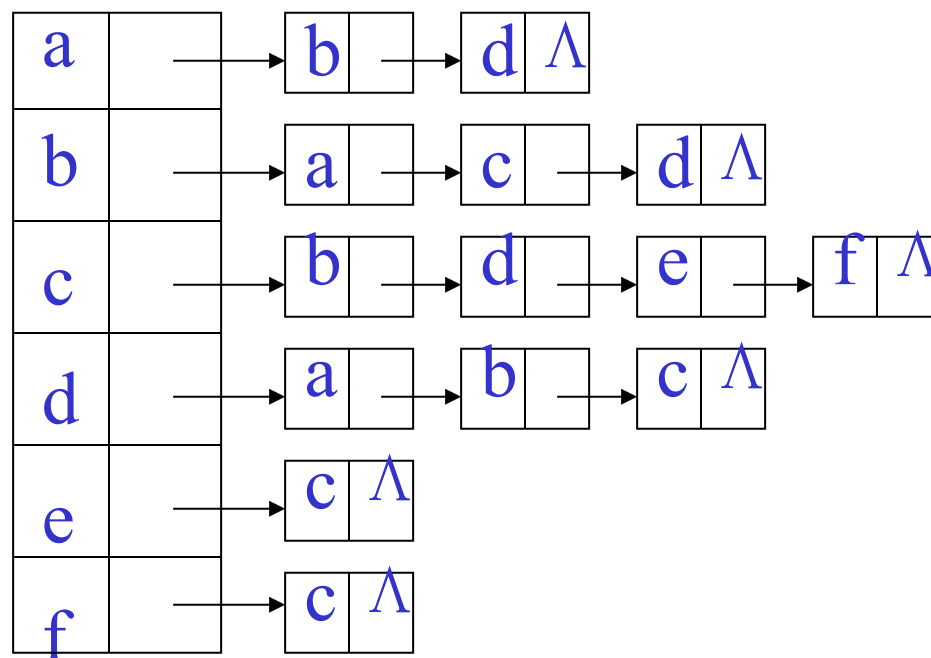
邻接表不唯一，因各个边结点的链入顺序是任意的

空间效率为 $O(n+2e)$ 。
若是稀疏图($e \ll n^2$)，
比邻接矩阵表示法 $O(n^2)$
省空间。

邻接表的逻辑概念：将顶点V的邻接点放入一个线性表(采用链式实现)存储

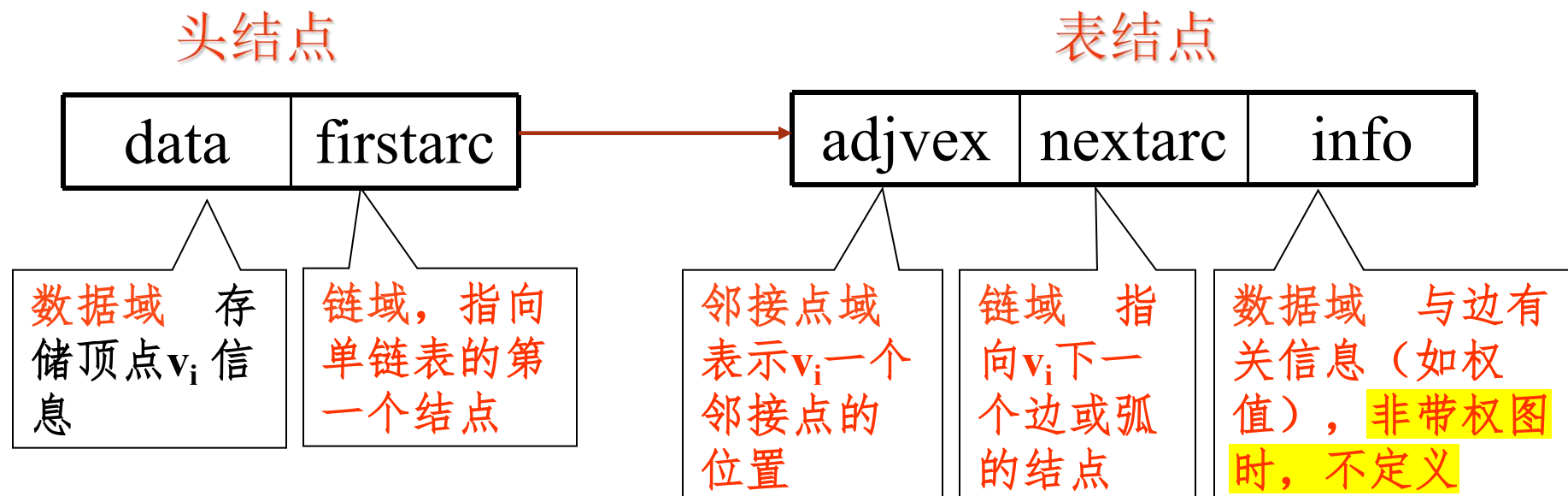
a	b, d
b	a, c, d
c	b, d, e, f
d	a, b, c
e	c
f	c

邻接表的顺序存储和链式存储结合的实现



邻接表（链式）表示法

- ❖ 对每个顶点 v_i 建立一个单链表，把与 v_i 有关联的边的信息链接起来，每个结点设为3个域；边表。



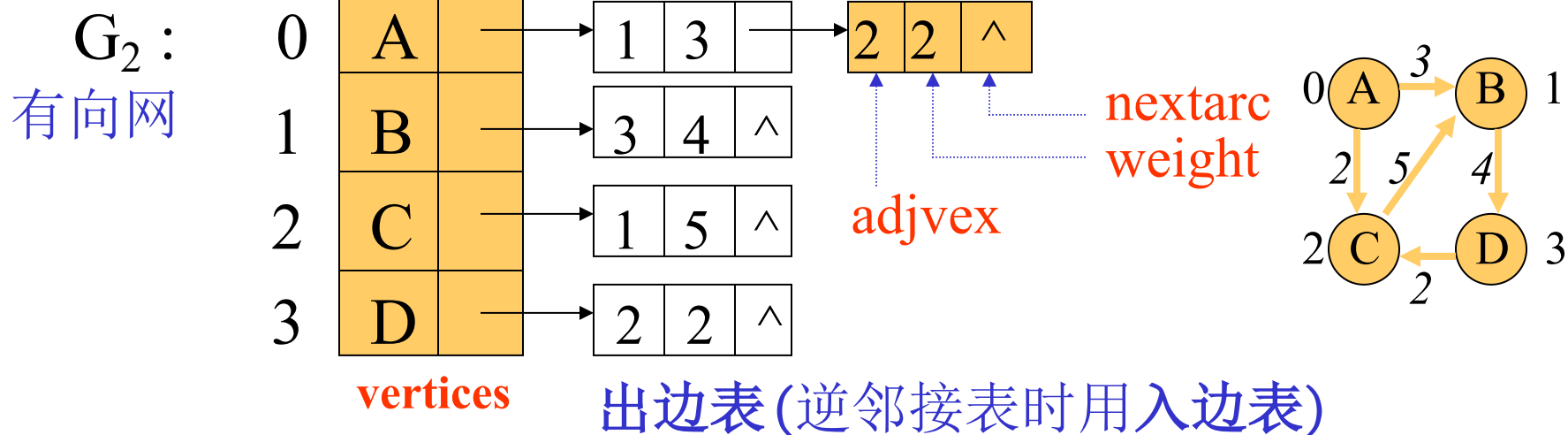
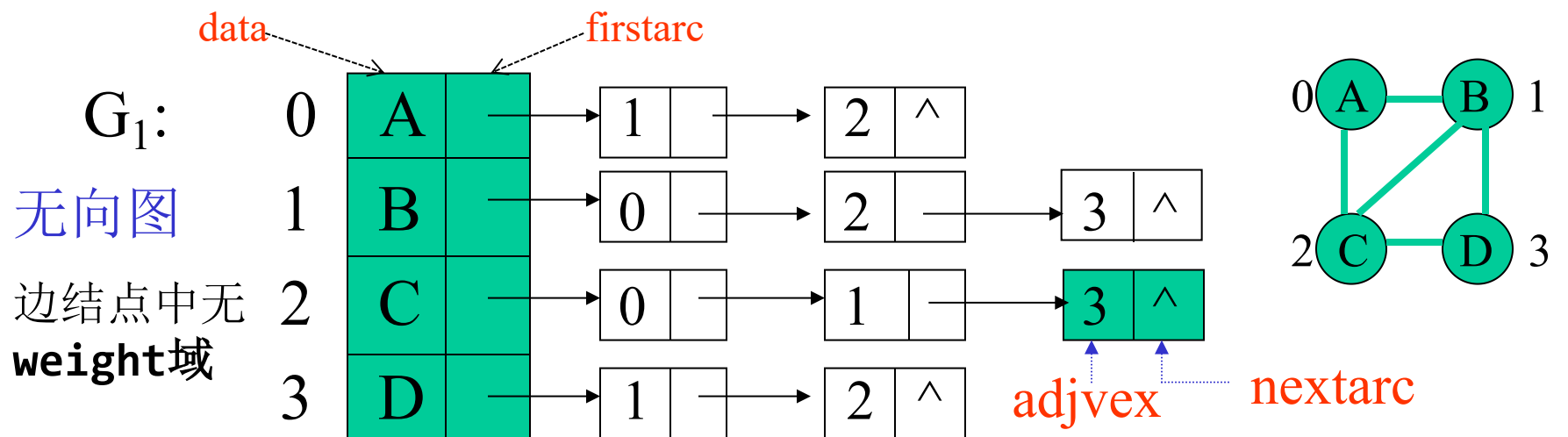
- ❖ 每个单链表有一个头结点（设为2个域），存 v_i 信息；
- ❖ 用来记录顶点信息的头结点，用顺序存储结构保存为一个数组，顶点顺序表（顶点表）。

[邻接表的存储结构定义]

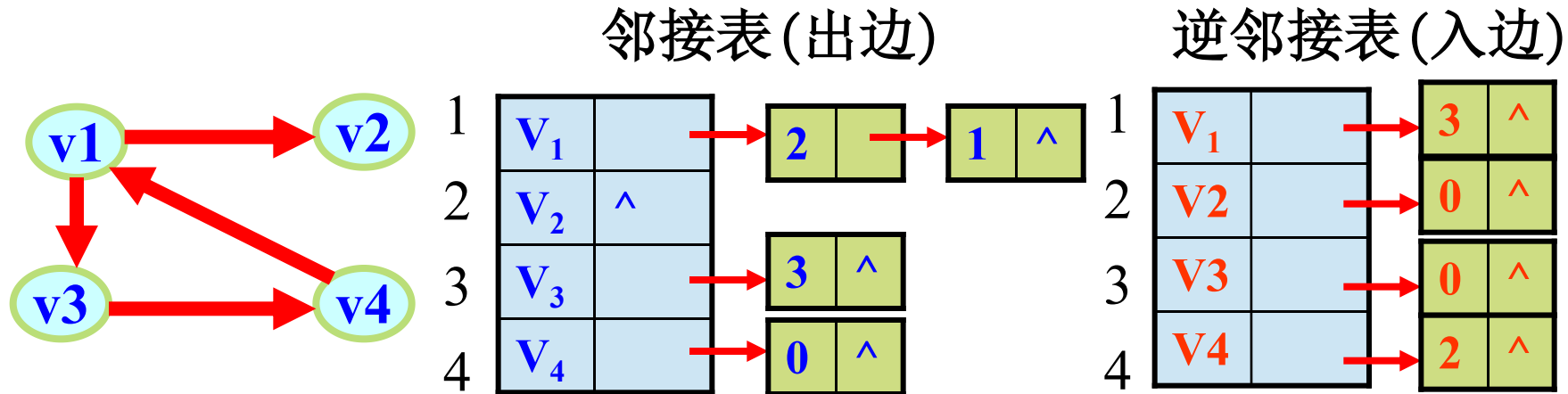
```
#define MAX_VERTEX_NUM    20                // 最大顶点个数
typedef enum{DG, DN, UDG, UDN} GraphKind;    // 四种图类型
typedef int VertexType;    //顶点携带信息的类型，假设为整数
typedef struct ArcNode {
    int                adjvex;    //该边的另外一个顶点
                                //或弧所指向的顶点的位置
    struct ArcNode * nextarc ;    //指向下一条边或弧的指针
    int                weight;    //边或弧的权重，网类型时使用
    InfoType           * info;    //该弧的其它相关信息的指针
}ArcNode;
typedef struct VNode {
    VertexType    data;    // 顶点信息
    ArcNode       * firstarc ;//指向第一条依附该顶点的弧
                                //或该顶点的第一条边的指针
                                //即头结点的指针域
}VNode, AdjList[MAX_VERTEX_NUM ];
typedef struct {
    AdjList        vertices;    // 顶点向量
    int            vexnum, arcnum;    // 图的当前顶点数和弧数
    GraphKind      kind;    // 图的种类标志
}ALGraph;
```

图的存储结构---邻接表(Adjacency List) 顺序存储+链式存储

顶点顺序表 邻接顶点的单链表(边表)



有向图的邻接表和逆邻接表



空间效率为 $O(n+e)$

出度: $OD(V_i) = \text{单链出边表中链接的结点数}$

入度: $ID(V_i) = \text{邻接点域为 } V_i \text{ 的弧个数}$

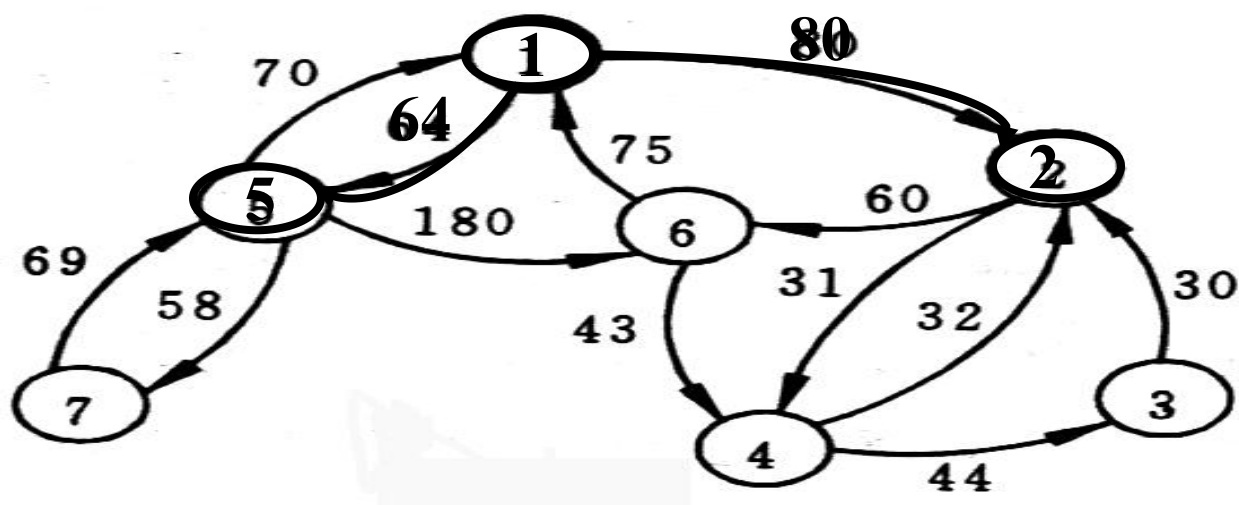
度: $TD(V_i) = OD(V_i) + ID(V_i)$

已知某网的邻接（出边）表，请画出该网络。

1		1	80	-	4	64	Λ			
2		3	31	-	5	60	Λ			
3		1	30	Λ						
4		1	32	-	2	44	Λ			
5		0	70	-	5	180	-	6	58	Λ
6		0	75	-	3	43	Λ			
7		4	69	Λ						

当邻
储结
图便

当邻接表的存储结构形成后，图便唯一确定！



[算法示例] 建立无向图的邻接表表示

步骤:

1) scanf(顶点数 n , 边数 e) $O(1)$

2) 依次读入每个顶点数据,
填入顶点表vertices的相应位置 $O(n)$

3) 建立边表: $O(e)$

3.1) 读入一条边的两个顶点编号 i 、 j 和权值 w

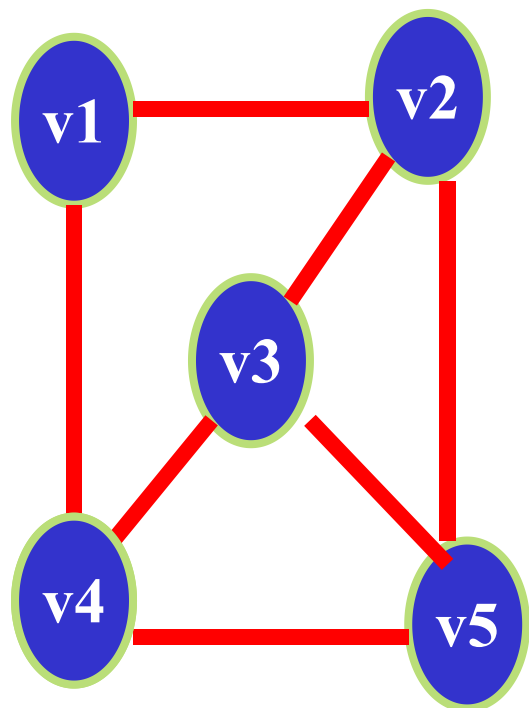
3.2) 申请边表结点, $adjvex$ 域= j , $weight$ 域= w ,
将此结点插入顶点 v_i 的边表头部

3.3) 申请边表结点, $adjvex$ 域= i , $weight$ 域= w ,
将此结点插入顶点 v_j 的边表头部

若尚有未读入的边, 转3.1)

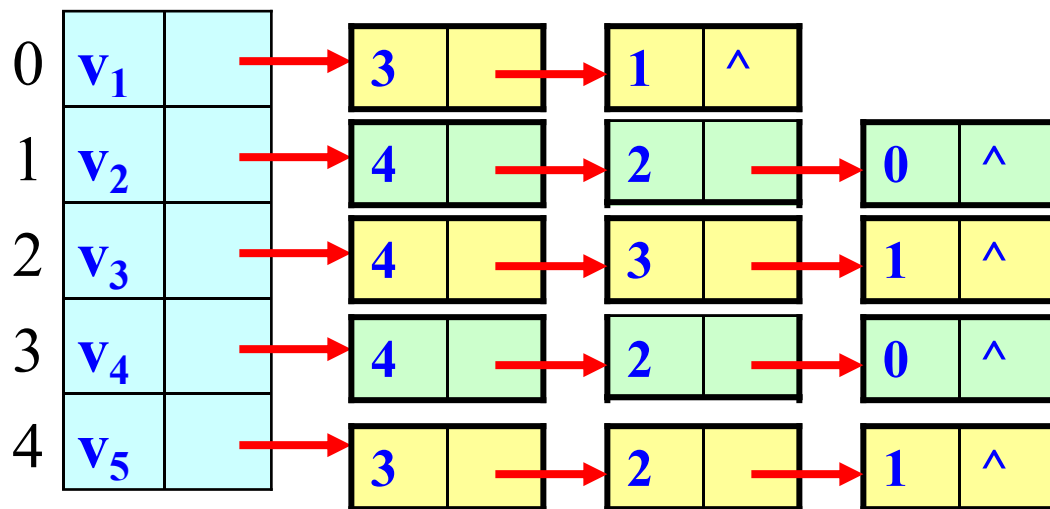
$O(n+e)$

邻接矩阵与邻接表表示法的关系



顶点表(v1 v2 v3 v4 v5)

0	1	0	1	0	v1
1	0	1	0	1	v2
0	1	0	1	1	v3
1	0	1	0	1	v4
0	1	1	1	0	v5



- 联系：**邻接表中每个链表对应于邻接矩阵中的一行，链表中结点个数等于一行中非零元素的个数。

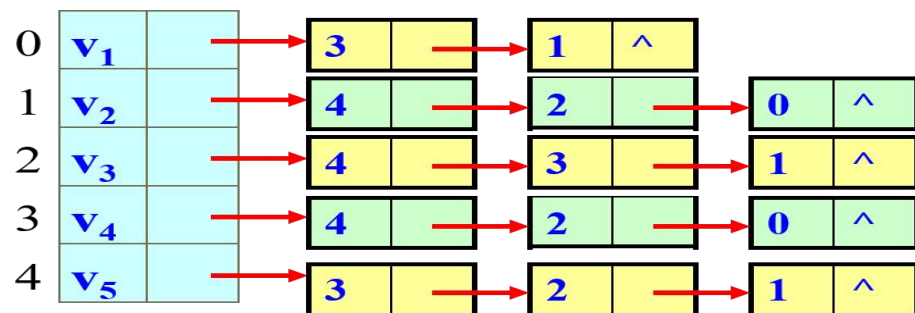
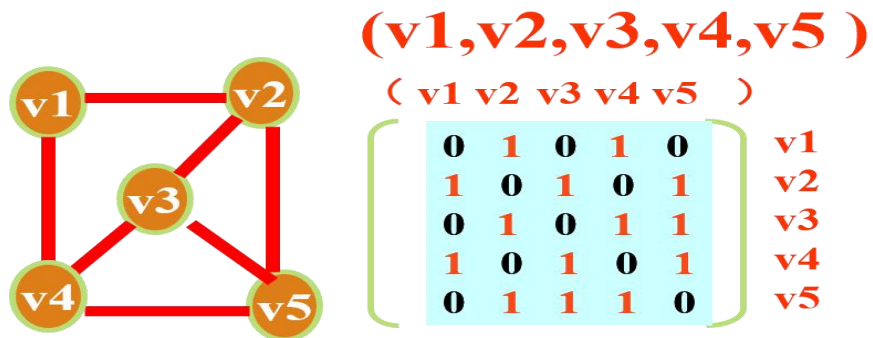
邻接矩阵与邻接表表示法的关系

2. 区别:

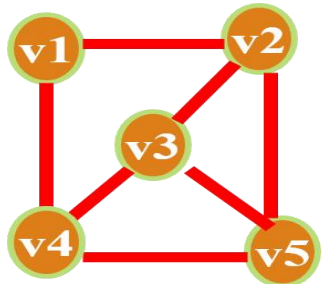
- (1) 对于任一确定的无向图，邻接矩阵是**唯一**的（行列号与顶点编号一致），但邻接表**不唯一**（链接次序与顶点编号无关）。
- (2) 邻接矩阵的空间复杂度为 **$O(n^2)$** ，而邻接表的空间复杂度为 **$O(n+e)$** 。
- (3) 邻接表空间效率高，容易寻找顶点的邻接点；
- (4) 邻接矩阵空间效率低，但是容易找顶点相关的边或弧；

3. 用途: 邻接矩阵多用于**稠密图**；而邻接表多用于**稀疏图**

	邻接矩阵			邻接表		
		无向图	有向图		无向图	有向图
空间对比	顶点表	n	n	顶点表	n	n
	邻接矩阵	$n^2/2$ (对称情况下只存一半)	n^2	边表	2e	e
	适合稠密图: $O(n^2)$			适合稀疏图: $O(n+e)$		



	邻接矩阵	邻接表
loc_verte x(G,v)	扫描顶点表	扫描顶点表
get_verte x(G,i)	扫描顶点表	扫描顶点表
first_adj (G,v)	扫描顶点表, 找到v的下标; 在邻接矩阵中按行查找第一个	扫描顶点表, 找到v, 在其边表查找第一个
next_adj (G,v,w)	扫描顶点表, 找到v的下标; 在邻接矩阵中按行查找w, 找到后再查找w 下一个返回	扫描顶点表, 找到v, 在其边表查找w, 找到则返回w的next
ins_verte x(G,v)	在顶点表的表尾插入, 在矩阵 中加入行和列, 置为空	在顶点表的表尾或表头插入, 边表置 为空



$(v_1, v_2, v_3, v_4, v_5)$

$(v_1 \ v_2 \ v_3 \ v_4 \ v_5)$

0	1	0	1	0
1	0	1	0	1
0	1	0	1	1
1	0	1	0	1
0	1	1	1	0

v_1

v_2

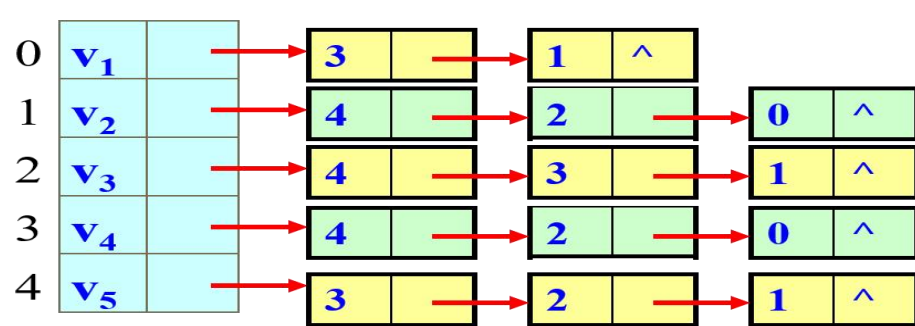
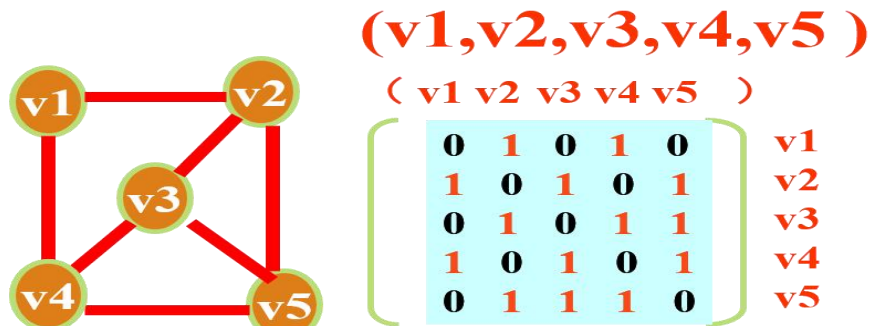
v_3

v_4

v_5

0	v_1		3		1	^
1	v_2		4		2	
2	v_3		4		3	
3	v_4		4		2	
4	v_5		3		2	

	邻接矩阵	邻接表
$del_vertex(G, v)$	扫描顶点表，找到v的下标；扫描矩阵，将所有和v有关的边删除；删除矩阵中v的行和列；	无向图：扫描顶点表，找到v；对边表的每个结点，找到对端顶点，在其边表中删除v结点；删除v的边表此结点。 有向图：对顶点表中的每个顶点，如果是v，则删除其边表；否则，扫描其边表，删除以v为结束的弧；
$ins_edge(G, u, v)$	扫描顶点表，找到u和v的下标，在矩阵的对应下标位置增加	无向图：扫描顶点表，找到v和u的下标；在两个结点的边表中增加结点。 有向图：扫描顶点表，找到v和u的下标；在u的边表中增加v结点。
$del_edge(G, u, v)$	扫描顶点表，找到u和v的下标，在矩阵的对应下标位置删除	无向图：扫描顶点表，找到v和u的下标；扫描两个结点的边表中删除对应结点。 有向图：扫描顶点表，找到v和u的下标；在u的边表中删除v结点。



邻接矩阵

无向图：扫描顶点表，找到v的下标；按行统计
有向图：扫描顶点表，找到v的下标；按对应行，统计出度；按对应列统计入度

扫描顶点表，找到u和v的下标，在矩阵的对应下标位置查找

扫描整个矩阵，统计边或弧的数量。

邻接表

无向图：扫描顶点表，找到v；统计边表结点数量
有向图：出度，同无向图。
入度：扫描顶点表，找到v的下标；对顶点表中所有结点，扫描其边表，统计结束结点为v的弧的数量。

无向图：扫描顶点表，找到v和u的下标；在v或u的边表中查找另外一个结点是否存在
有向图：扫描顶点表，找到u和v的下标；在u的边表中查找v是否存在。

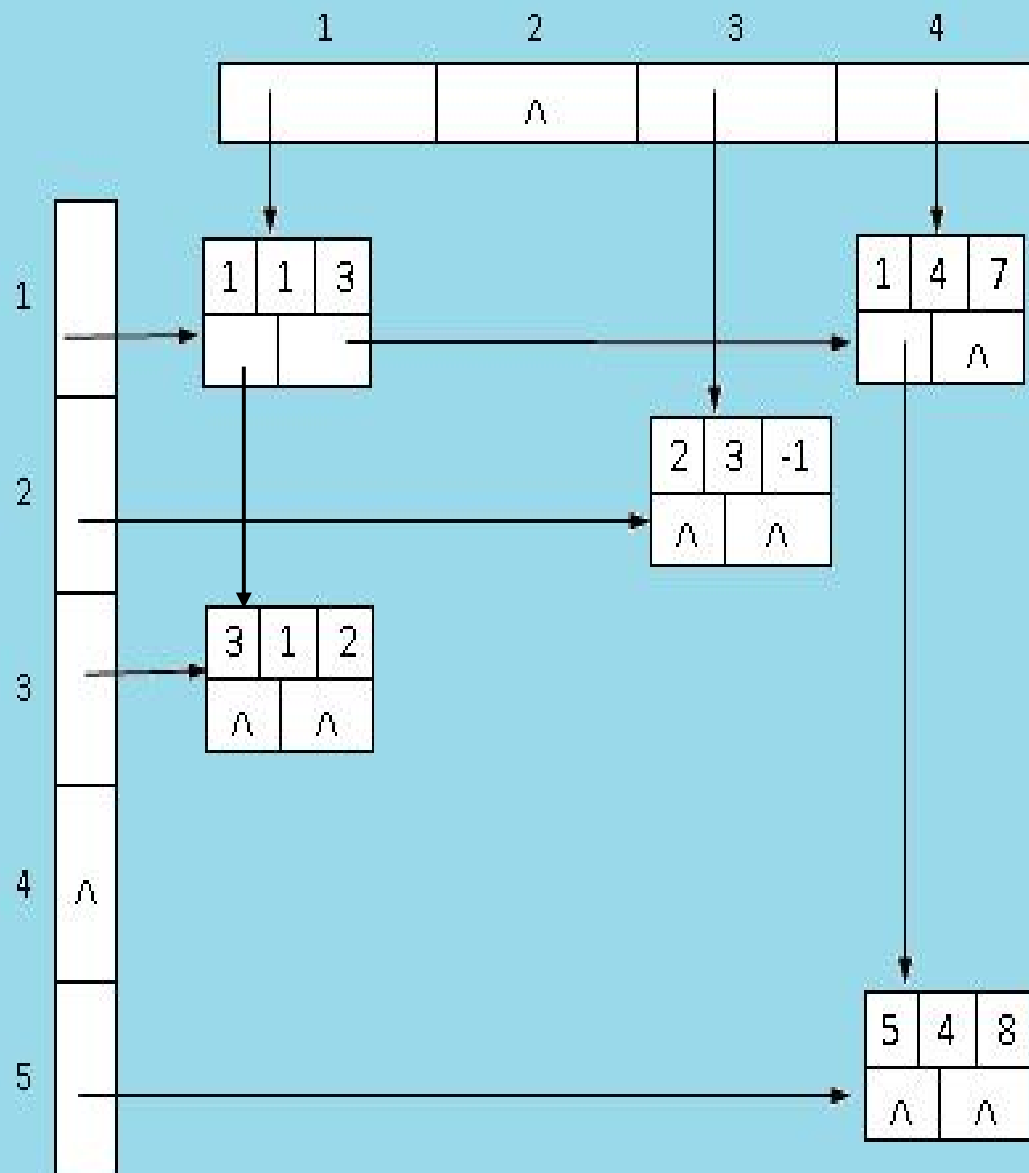
无向图：扫描顶点表中每个顶点的边表统计边数量后除以2
有向图：扫描顶点表中每个顶点的边表统计弧的数量

	邻接矩阵			邻接表		
		无向图	有向图		无向图	有向图
空间对比	顶点表	n	n	顶点表	n	n
	邻接矩阵	$n^2/2$	n^2	边表	2e	e
	适合稠密图: $O(n^2)$			适合稀疏图: $O(n+e)$		
时间复杂度比较	求某个顶点的度	扫描矩阵一行, $O(n)+O(n)$	出度: 扫描一行, $O(n)+O(n)$	求顶点的度	扫描此顶点的边表, $O(n)+O(n)$	出度: 扫描此顶点的边表, $O(n)+O(e)$
			入度: 扫描一列, $O(n)+O(n)$			入度: 扫描所有顶点的边表, $O(n)+O(n+e)$
	判定边(弧)是否存在	取矩阵对应下标的值, $O(n)+O(1)$	取矩阵对应下标的值, $O(n)+O(1)$	判定边(弧)是否存在	扫描边的两个顶点中某一个的边表 $O(n)+O(e)$	扫描弧的起点为顶点的边表 $O(n)+O(e)$
	求边(弧)的数目	扫描整个矩阵 $O(n)+O(n^2)$	扫描整个矩阵 $O(n)+O(n^2)$	求边(弧)的数目	扫描所有边表 $O(n+2e)$	扫描所有边表 $O(n+e)$

$$A = \begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -8 \end{pmatrix}$$

十字链表表示稀疏矩阵：

对矩阵中每个非零元素存储为一个结点，结点由5个域组成，其中：
 row域存储非零元素的行号，col域存储非零元素的列号，v域存储本元素的值，right，down是两个指针域，从行方向、列方向链接起来。



稀疏矩阵 A 的十字链表

7.2.3 十字链表：适用对象为有向图，顺序存储+链式存储

核心思路：一条链表记录结点指向了谁，另一条链表记录谁指向了当前结点。
邻接表和逆邻接表的结合，存储结构定义参教材第165页。

顶点结点

data	firstin	firstout
------	---------	----------

data: 顶点的数据域，保存顶点的数值

firstin: 顶点的指针域，给出自该顶点出发的第一条弧的弧结点的地址。

firstout: 顶点的指针场，给出进入该顶点的第一条弧的弧结点的地址。

边表中的弧结点

info	tailvex	headvex	hlink	tlink
------	---------	---------	-------	-------

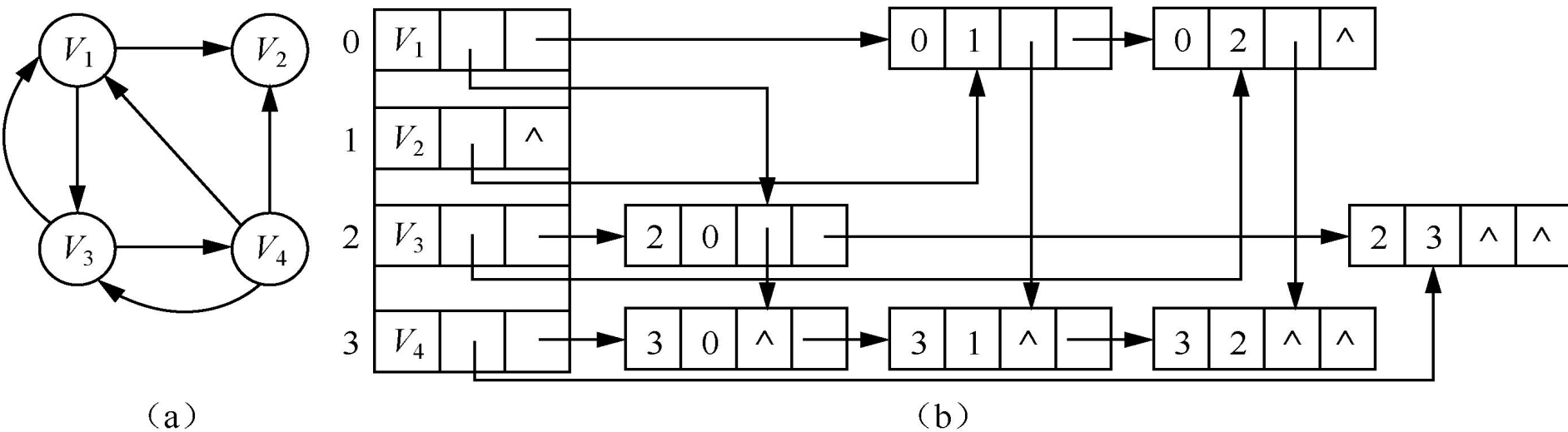
info:弧的数据域，保存弧的权值等（可以没有）

tailvex:本条弧的出发结点的地址。

headvex:本条弧的终止结点的地址。

hlink:终止结点相同的弧中的下一条弧的地址。

tlink:出发结点相同的弧中的下一条弧的地址。



7.2.4 邻接多重表：适用对象为无向图，顺序存储+链式存储

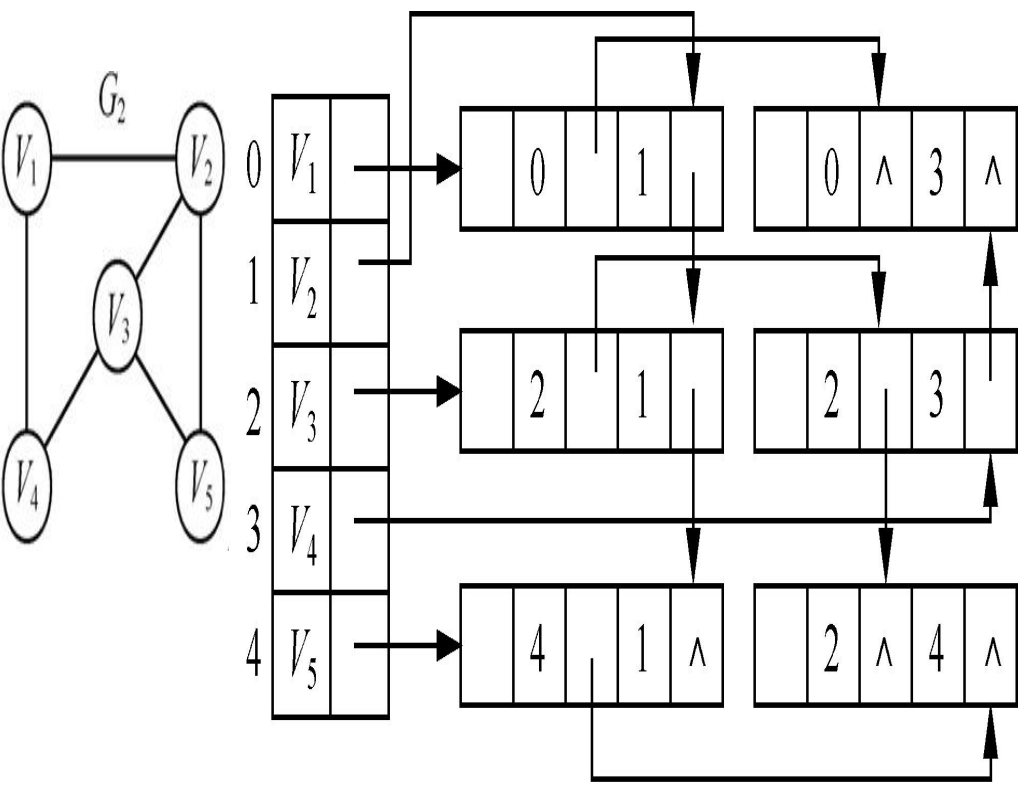
核心思路：每条边对应一个结点，方便无向图中对边的操作。

顶点结点

data	firstedge
------	-----------

data: 结点的数据域，保存结点的数据值。

firstedge: 结点的指针域，给出自该结点出发的第一条边的边结点的地址。



边表中的边结点

mark	ivex	ilink	jvex	jlink	info
------	------	-------	------	-------	------

info: 边的数据域，保存边的权值等（可以没有）

mark: 边结点的标志域，用于标识该条边是否被访问过。

ivex: 本条边依附的一个结点的地址（下标）。

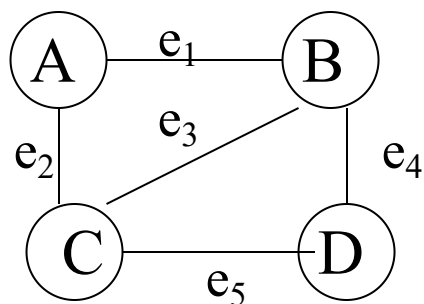
ilink: 依附于该结点（地址由 **ivex** 给出）的边中的下一条边的地址。

jvex: 本条边依附的另一个结点的地址。

jlink: 依附于该结点（地址由 **jvex** 给出）的边中的下一条边的地址

图的关联矩阵存储，用于无向图，有向图，顺序存储

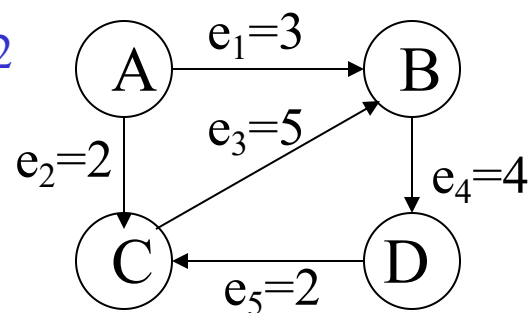
G_1



		e ₁	e ₂	e ₃	e ₄	e ₅	边
A	A	1	1	0	0	0	
B	B	1	0	1	1	0	
C	C	0	1	1	0	1	
D	D	0	0	0	1	1	

顶点

G_2

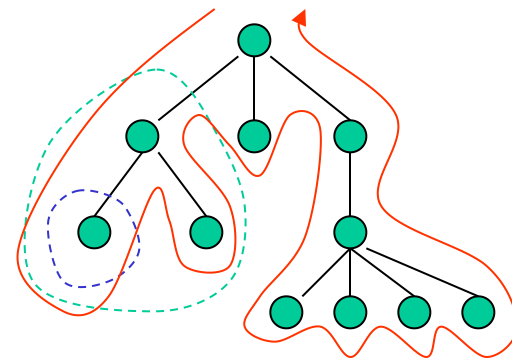


		e ₁	e ₂	e ₃	e ₄	e ₅	边
A	A	-3	-2	0	0	0	
B	B	3	0	5	-4	0	
C	C	0	2	-5	0	2	
D	D	0	0	0	4	-2	

顶点

7.3 图的遍历

- 深度优先遍历 (树的先根遍历的推广)
- 广度优先遍历 (树的按层次遍历的推广)



[例]

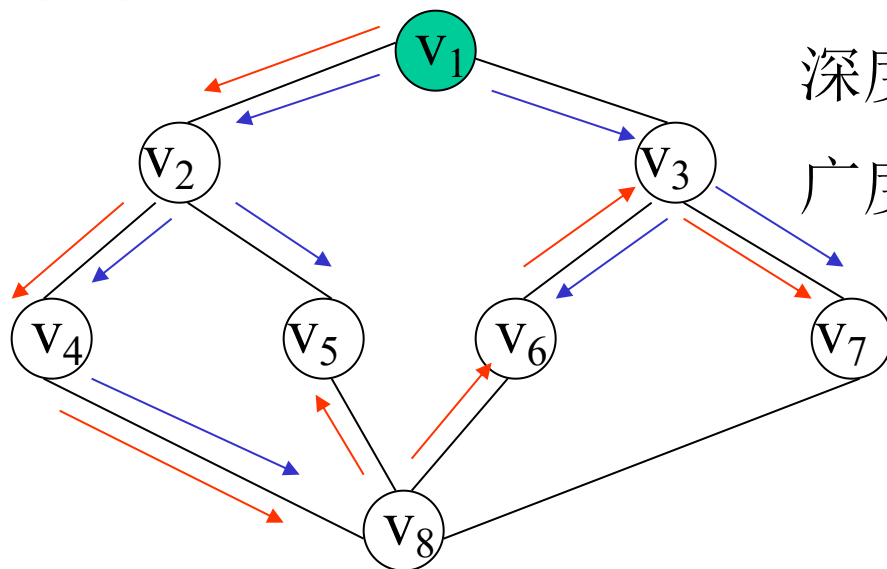
设从 v_1 出发遍历

深度: $v_1 v_2 v_4 v_8 v_5 v_6 v_3 v_7$

广度: $v_1 v_2 v_3 v_4 v_5 v_6 v_7 v_8$

为什么不简单的用顺序查找的方法对顶点表进行查找?

树存储结构中没有所有结点的一个简单的存储结构, 图有了顶点表, 为什么不直接遍历顶点表?

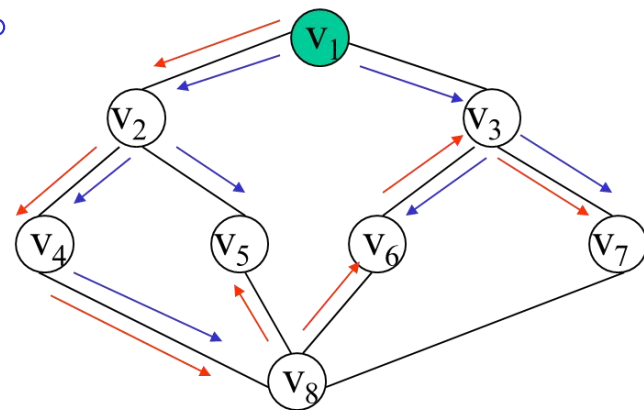


数据结构: 主——图的存储结构
辅——数组 $visited[0..n-1]$

7.3 图的遍历

遍历定义：从已给的连通图中某一顶点出发，**沿着一些边访遍图中所有的顶点**，且使每个顶点仅被访问一次，就叫做**图的遍历**，它是图的基本运算，是求解图的连通性、拓扑排序和求关键路径的基础。

遍历实质：找每个顶点的邻接点的过程。



图遍历的特点：由于图中可能存在**回路**，且图的任一顶点都可能与其它顶点相通，在访问完某个顶点之后可能会沿着某些边**又回到了曾经访问过的顶点**，需要在到达曾经访问过的点时回溯，算法实现时要考虑这一点和树的遍历不同的地方。

怎样避免重复访问？

解决思路：设置辅助数组 *visited* [n]，用来标记每个被访问过的顶点。

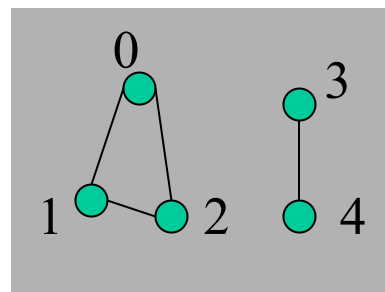
- ✓ *i* 结点的数组项初始状态 *visited* [*i*] 为 0 (false)
- ✓ *i* 被访问后，改 *visited* [*i*] 为 1 (true)，防止被多次访问

图的两种遍历的记录访问过的点的方式：

- ✓ 深度优先搜索，递归、或用栈记录访问过的结点
- ✓ 广度优先搜索，用队列记录访问过的结点

7.3.1 深度优先搜索

递归的算法思想



(1)访问顶点 v ，并记录 v 已被访问
(2)依次从 v 的未访问的邻接点出发，递归进行深度优先搜索图 G 。

2.1 先找到 v 的第一个邻接点 w

2.2 如果 w 存在并且 w 未访问过，对 w 进行递归的DFS
如果 w 不存在，递归函数返回。

2.3 再从 v 的邻接点 w 出发，找下一个邻接点赋值到 w

2.4 回到2.2

(3)在顶点表中是否还有未访问的顶点，有则将未访问结点赋值到 v ，回到(1)；没有则算法结束。

核心思路：只要将 v 的邻接点都递归访问完，那么 v 所在的连通分量中的顶点就都会被访问到。对非连通图，依次对每个连通分量进行就可以完成图的DFS

```
typedef enum{FALSE, TRUE} Boolean; //FALSE为0, TRUE为1
Boolean  visited[MAX_VERTEX_NUM]; //辅助访问标志向量

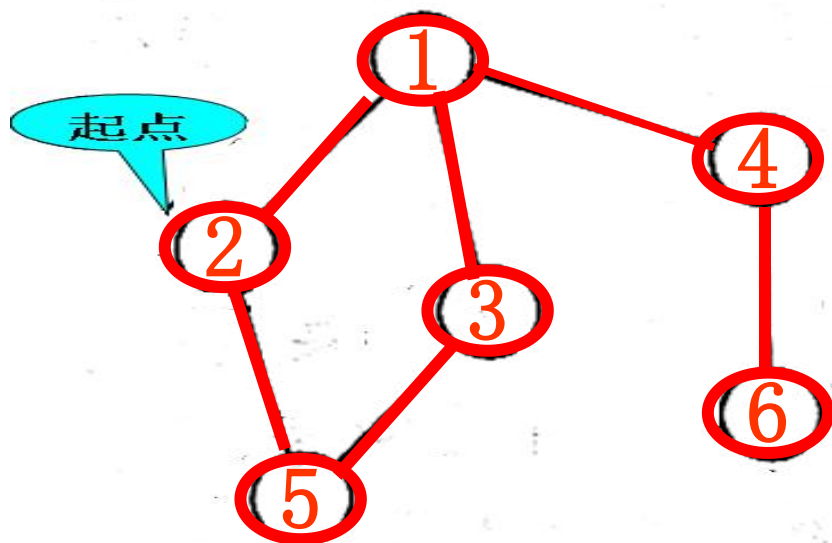
void  DFSTraverse(Graph G)
{
    for (v=0; v<G.vexnum; ++v)
        visited[v]=FALSE; //辅助访问标志向量初始化
    for (v=0; v<G.vexnum; ++v)
        if ( !visited[v] )  DFS(G, v);
} // DFSTraverse
```

```
void  DFS(Graph G, int  v)
{  visit(v);
    visited[v]=TRUE; //访问顶点v, 并标记
    for( w=FirstAdjVex(G,v); w>-1; w=NextAdjVex(G,v,w))
        if( !visited[w] )  DFS(G,w);
} // DFS
```

讨论1: 邻接矩阵实现DFS

邻接矩阵 A

	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0



——开辅助数组 $visited[n]!$

辅助数组 $visited[n]$

	1	2	3	4	5	6
1	0	0	1	1	1	1
2	0	1	1	1	1	1
3	0	0	0	1	1	1
4	0	0	0	0	0	1
5	0	0	0	0	1	1
6	0	0	0	0	0	1

DFS 结果

$2 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$

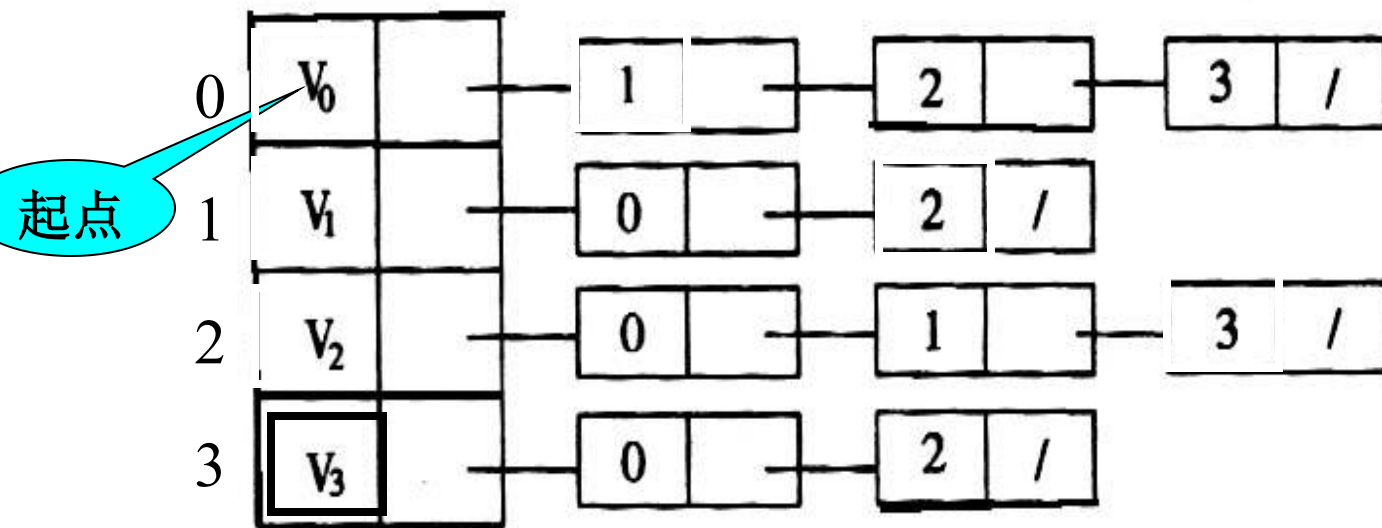
调用顺序

2, 1, 3, 5, (回退3), (回退1), 4, 6,
(回退4), (回退1), (回退2), 结束

```
void DFS(MGraph G, int v)
//深度优先遍历邻接矩阵表示的图
{
    visit(G.vexs[v]);
    visited[v]=TRUE;
    for ( j=0; j<G.vexnum; ++j )    //按行找
        if ( G.arcs[v][j]!=0 && !visited[j] ) DFS(G, j);
} // DFS
```

讨论2: 在图的邻接表中进行DFS

—借用 *visited* [n]



辅助数组 *visited* [n]

0	0	1	1	1
1	0	0	1	1
2	0	0	0	1
3	0	0	0	1

DFS 结果

$v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$

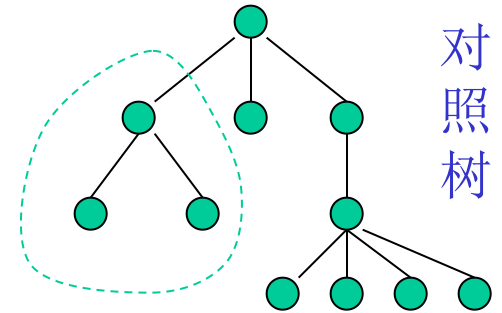
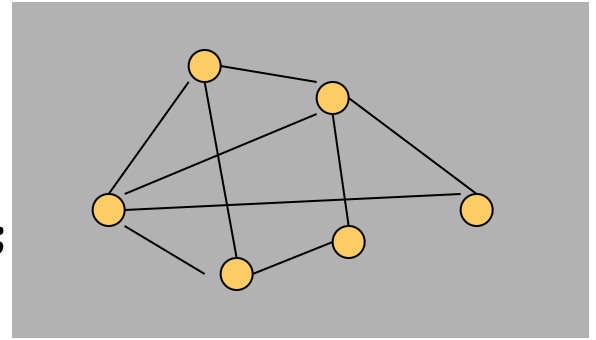
```

void DFS(ALGraph G, int v)
//深度优先遍历邻接表表示的图
{
    visit(G.vertices[v].data);
    visited[v]=TRUE;
    p=G.vertices[v].firstarc; //取v边表的头指针
    while (p)
    {
        if ( !visited[p->adjvex] ) DFS(G, p->adjvex);
        p=p->nextarc; //按边表找
    }
} // DFS

```

[非递归的算法思想]

- (1) 访问一个顶点，并记录它已被访问；
将它的所有未访问的邻接顶点入栈；
- (2) 如果栈空，则退出；
否则，栈中一顶点出栈；
- (3) 如果该顶点已被访问过，则转(2)
否则，转(1)



[算法时间复杂度分析]

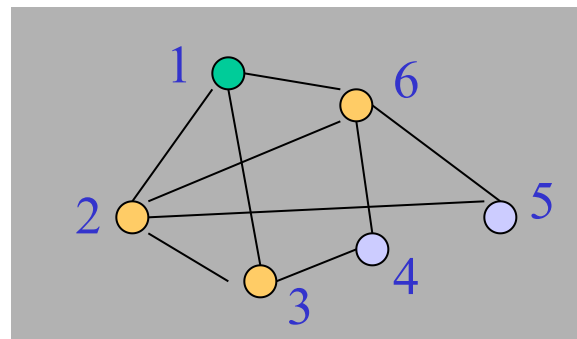
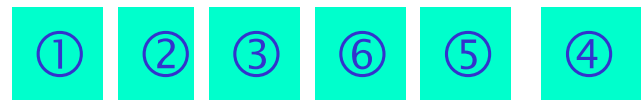
主要操作：查找每个顶点的所有邻接点

- 邻接矩阵 $O(n^2)$ 
- 邻接表 $O(n+e)$ $\begin{cases} \text{无向图结点数 } n+2e \\ \text{有向图结点数 } n+e \end{cases}$ 

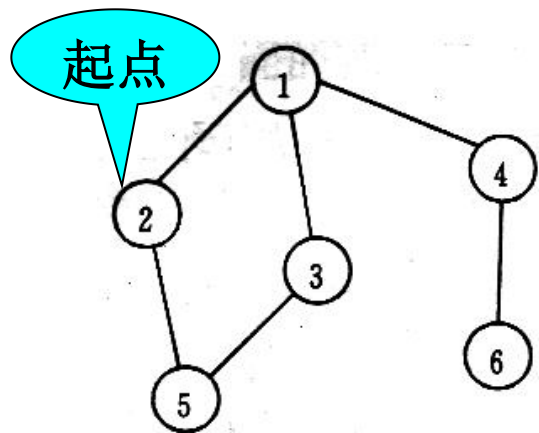
7.3.2 广度(宽度)优先遍历

[算法思想]

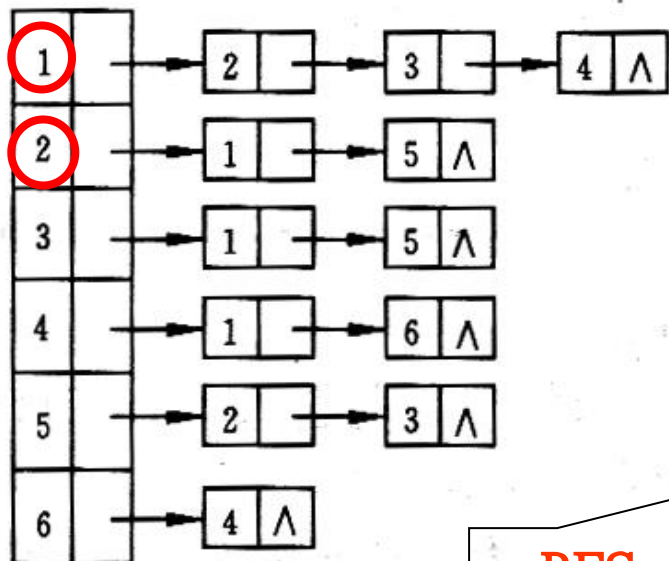
- (1)访问顶点 v ，并记录它已被访问；
顶点 v 入队列；
- (2)如果队列空，则退出；
否则，从队中取出一顶点；
- (3)求该顶点的一个邻接点；
如果此邻接点未被访问，
则访问它，并记录它已被访问，将其入队列；
- (4)如果该顶点还有下一个邻接点，则转(3)；
否则，转(2)



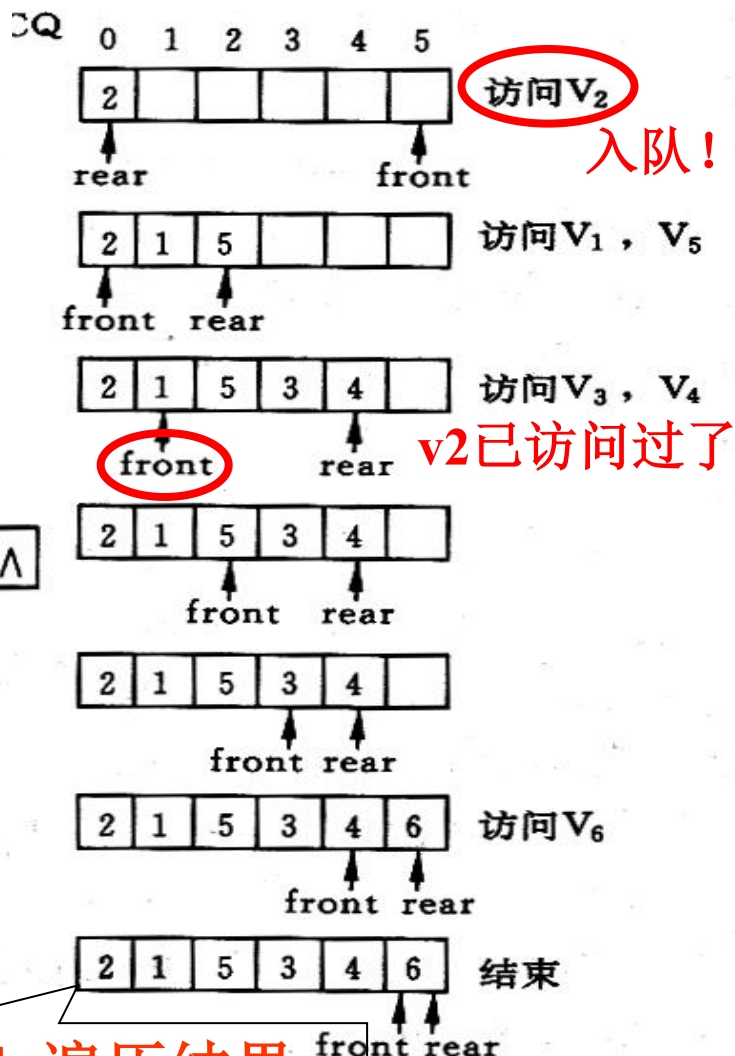
讨论1: 如何实现BFS? — 辅助数组 *visited* [n] 和辅助队列配合



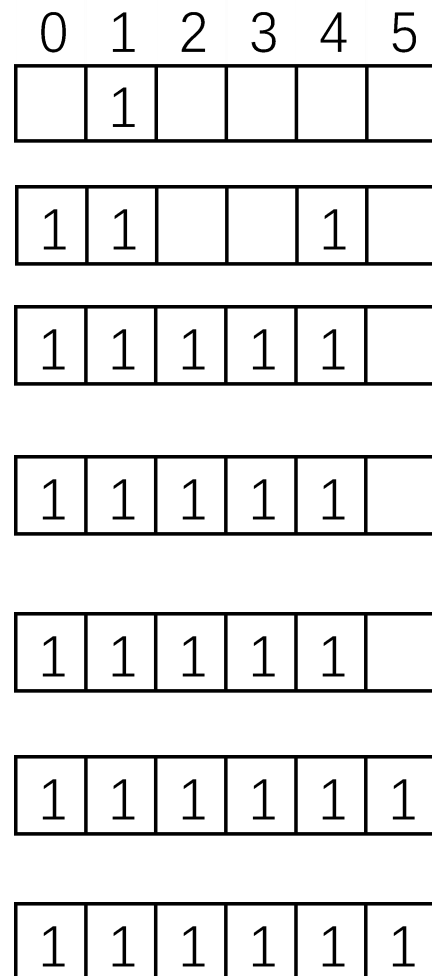
邻接表



辅助队列



辅助数组



BFS 遍历结果

```

typedef enum{FALSE, TRUE} Boolean; //FALSE为0, TRUE为1
Boolean visited[MAX_VERTEX_NUM]; //辅助访问标志向量
void BFSTraverse(Graph G)
{
    for (v=0; v<G.vexnum; ++v)
        visited[v]=FALSE; //辅助访问标志向量初始化
    for (v=0; v<G.vexnum; ++v)
        if ( !visited[v] ) BFS(G, v);
} // DFSTraverse

```

```

void BFS(Graph G, vtxptr v)
{
    InitQueue(Q); //辅助队列初始化
    visit(v); visited[v]=TRUE; EnQueue(Q, v);
    while (!QueueEmpty(Q)){
        DeQueue(Q, u);
        for(w=FirstAdjVex(G,u); w>-1; w=NextAdjVex(G,u,w)){
            if (!visited[w]){
                visited[w]=TRUE; visit(w); EnQueue(Q,w);
            }//if
        }//for
    }//while
} // BFS

```

算法描述

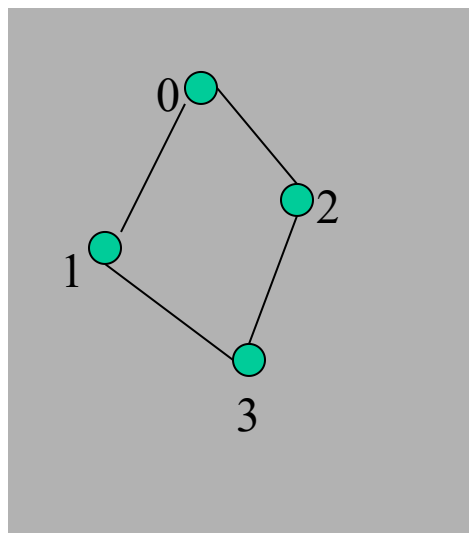
```
void BFSTraverse(Graph G)
{
    for (v=0; v<G.vexnum; ++v)
        visited[v]=FALSE; //标志向量初始化
    InitQueue(Q);          //辅助队列初始化
    for (v=0; v<G.vexnum; ++v){
        if (!visited[v]) {
            visit(v); visited[v]=TRUE; EnQueue(Q, v);
            while (!QueueEmpty(Q)) {
                DeQueue(Q, u);
                for(w=FirstAdjVex(G,u); w>-1; w=NextAdjVex(G,u,w))
                {
                    if (!visited[w]) {
                        visited[w]=TRUE; visit(w); EnQueue(Q,w);
                    } // if
                } // for
            } // while
        } // if
    } // for
} // BFSTraverse
```

[算法时间复杂度分析]

与深度优先遍历过程相同

7.3.3 图的遍历小结

- 深度优先遍历算法借助于栈结构实现；广度优先遍历算法借助于队列结构实现
- 图的遍历序列与算法和存储方式有关



	0	1	2	3
0	<u>0</u>	1	1	0
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

DFS 0 1 3 2

0		→ 2, 1
1		→ 3, 0
2		→ 3, 0
3		→ 2, 1

0 2 3 1

- 对于非连通图，通过遍历可求得各连通分量

7.3.4 图的遍历应用举例

[例1] 求一条从顶点 i 到顶点 s 的简单路径

$$b \dashrightarrow k$$

从b出发深度优先搜索遍历:

假设找到的第一个邻接点是c, 则得到的结点访问序列为:

b c h d a e **k** f g

假设找到的第一个邻接点是a, 且得到的结点访问序列为:

b a d h c e **k** f g

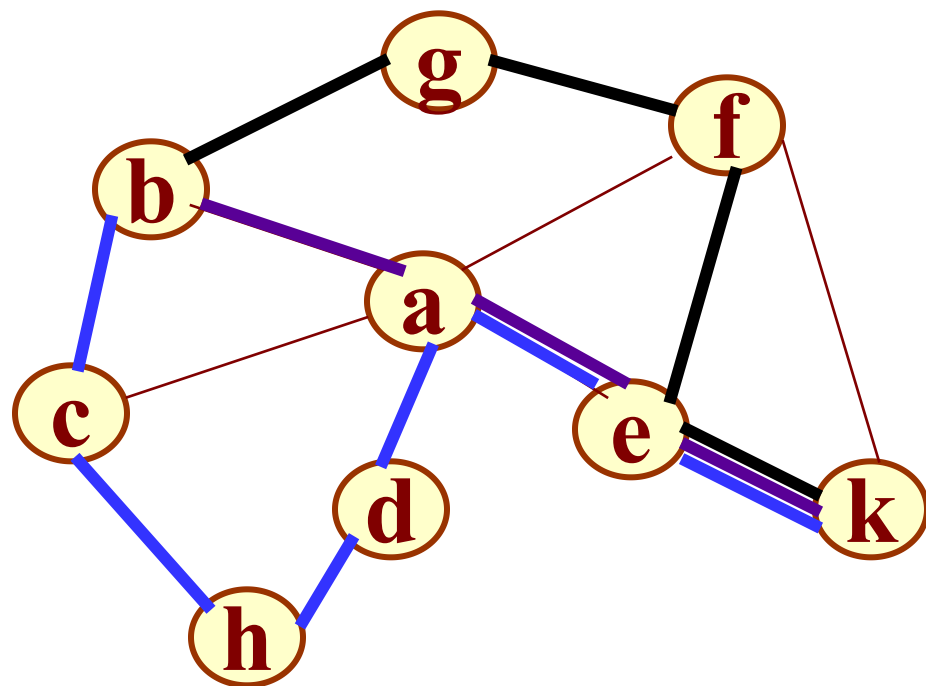


结论:

1. 从顶点 i 到顶点 s , 若存在路径, 则从顶点 i 出发进行深度优先搜索, 必能搜索到顶点 s 。
2. 遍历过程中搜索到的顶点 X 不一定是路径上的顶点。
3. 由 X 出发进行的深度优先遍历已经完成的顶点不一定是路径上的顶点。

```
void DFSearch( int v, int s, char *PATH, bool &found)
// 从第v个顶点出发递归地深度优先遍历图G,
// 求得一条从v到s的简单路径, 并记录在PATH中
{
    visited[v] = TRUE;    // 访问编号为 v 的顶点
    Append(PATH, getVertex(v)); // 编号为v的顶点加入路径
    for (w = FirstAdjVex(v);
        w != 0 && !found ;
        w = NextAdjVex(v) )
    {
        if (w==s) { found = TRUE; Append(PATH, w); }
        else if (!visited[w]) DFSearch(w,s,PATH,found);
    }
    if(!found) Delete(PATH,v); // 回退时从路径上删除顶点v
}
```

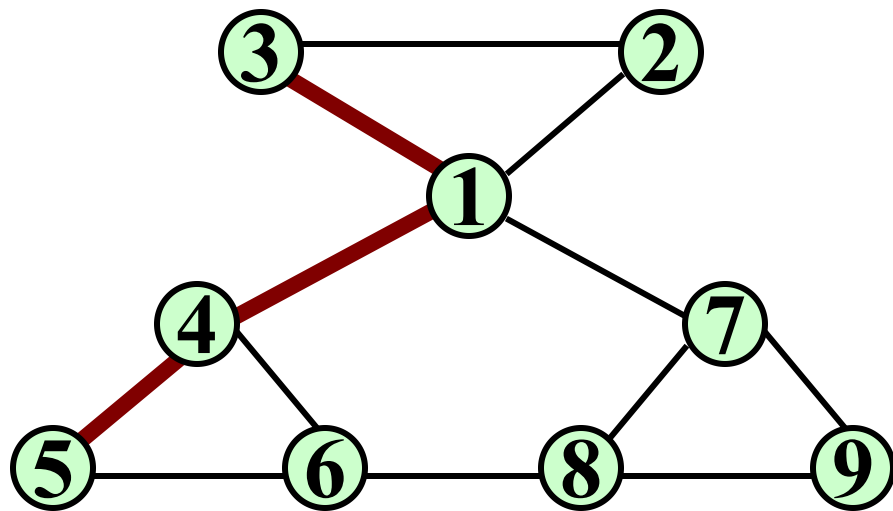
[例2] 求两个顶点之间的一条路径长度最短的路径



深度优先搜索访问顶点的次序取决于图的存储结构，而广度优先搜索访问顶点的次序是按“路径长度”渐增的次序。

因此，求路径长度最短的路径可以基于广度优先搜索遍历进行，但需要修改链队列的结点结构及其入队列和出队列的算法。

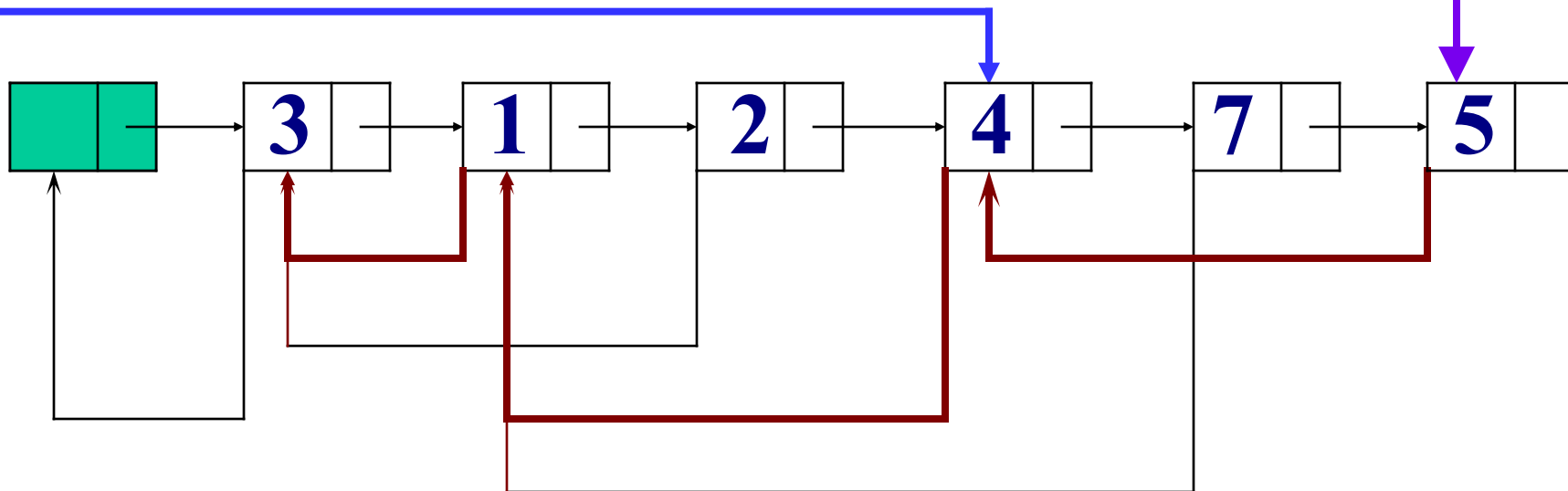
求右图中顶点 3 至顶点 5 的一条最短路径。



链队列的状态如下所示:

Q.front

Q.rear



- 1) 将链队列的结点改为“双链”结点。即结点中包含next 和prior两个指针;
- 2) 修改入队列的操作。插入新的队尾结点时, 令其prior域的指针指向刚刚出队列的结点, 即当前的队头指针所指结点;
- 3) 修改出队列的操作。出队列时, 仅移动队头指针, 而不将队头结点从链表中删除。

```
typedef DuLinkList QueuePtr;
void InitQueue(LinkQueue& Q)
{
    Q.front = Q.rear = new QNode;
    Q.front->next = Q.rear->next = NULL;
}
void EnQueue( LinkQueue& Q, QelemType e )
{
    p = new QNode;
    p->data = e;  p->next = NULL;
    p->prior = Q.front;
    Q.rear->next = p;  Q.rear = p;
}
void DeQueue( LinkQueue& Q, QelemType& e )
{
    Q.front = Q.front->next;
    e = Q.front->data
}
```

第七章 图

7.4 图的连通性问题

7.5 有向无环图及其应用

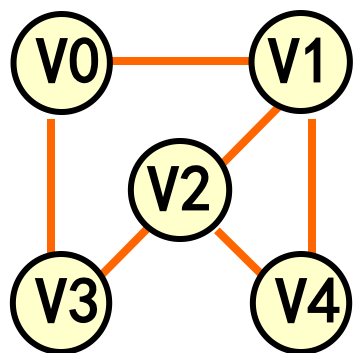
7.6 最短路径

本章学习要点

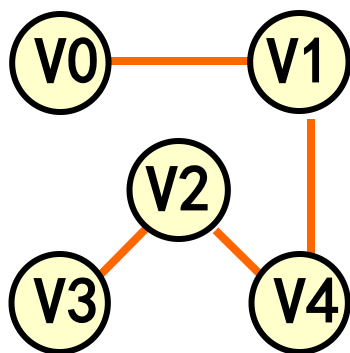
子图

设有两个图 $G=(V, \{E\})$ 、 $G_1=(V_1, \{E_1\})$ ，若 $V_1 \subseteq V$ ， $E_1 \subseteq E$ ，则称 G_1 是 G 的子图。

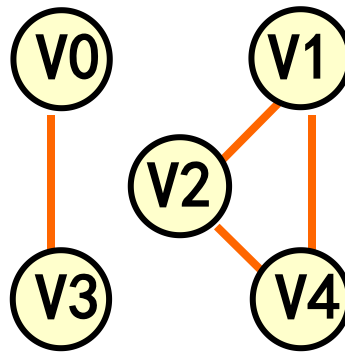
例：(b)、(c)、(d) 是 (a) 的子图



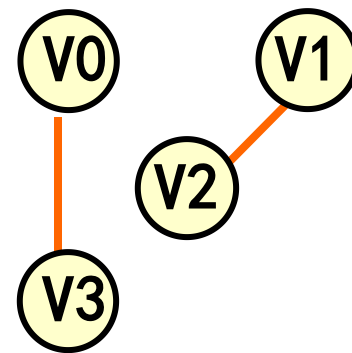
(a)



(b)



(c)



(d)

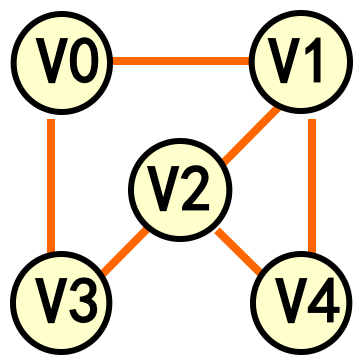
生成子图 由图的**全部顶点**和**部分边**组成的子图称为原图的生成子图。

例：(b)、(c) 是 (a) 的生成子图

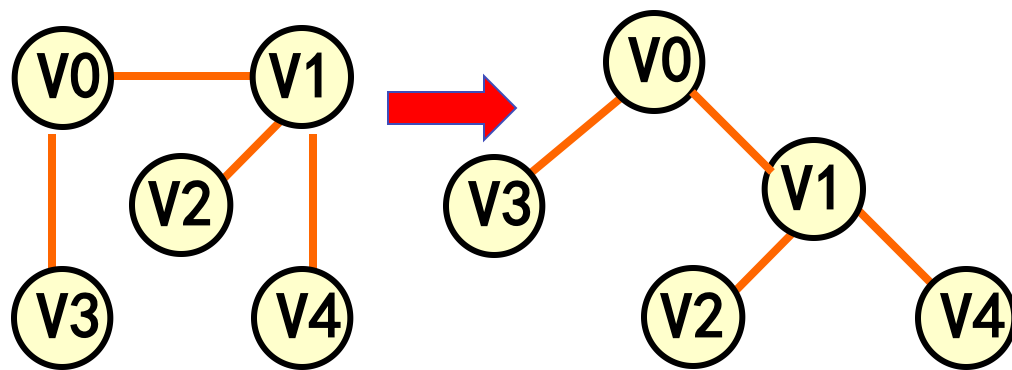
极小连通子图：该子图是 G 的连通子图，在该子图中删除任何一条边，子图不再连通。

生成树：包含无向图 G 所有顶点的极小连通子图。

生成森林：对非连通图，由各个连通分量的生成树的集合。



连通图 G_1



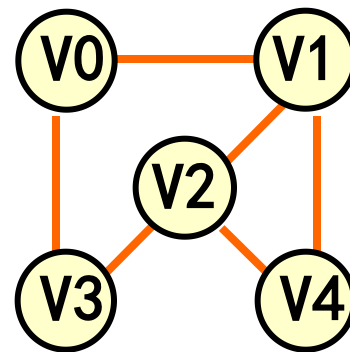
G_1 的生成树

生成树的概念（从遍历的角度看）：

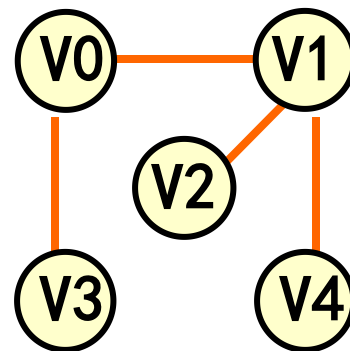
设 $G=(V_n, E_n)$ ，则从图中任一顶点进行遍历图时，必定将 E_n 分成两个集合 $T(G)$ 和 $B(G)$ ，其中 $T(G)$ 是遍历时经过的边的集合， $B(G)$ 是剩余边的集合。 $T(G)$ 是 G 的一棵生成树。

极小连通子图（从子图的角度）：该子图是 G 的连通子图，在该子图中删除任何一条边，子图不再连通。

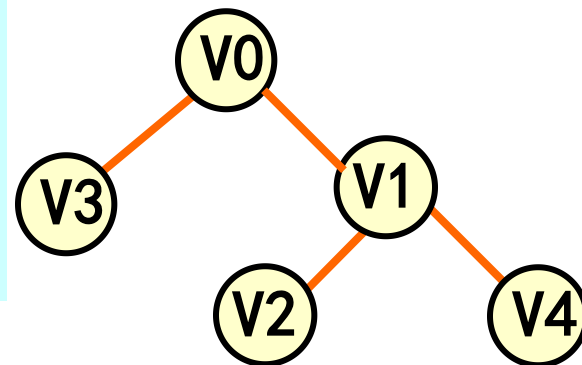
生成树：包含图 G 所有顶点的极小连通子图（ $n-1$ 条边）。



连通图 G_1



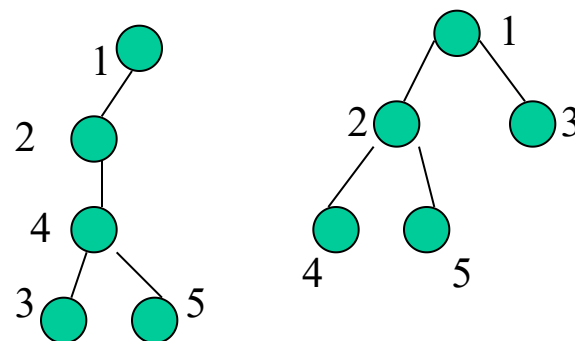
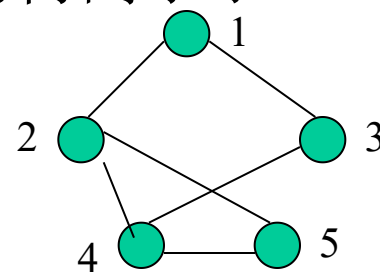
G_1 的生成树



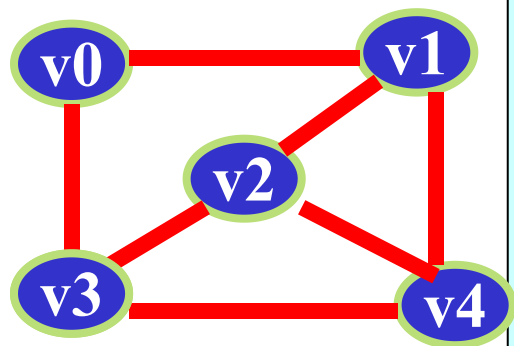
- 若连通图G有n个顶点，则其生成树必具有n-1条边。
- 生成树是包含图中所有顶点的极小连通子图。
- 生成树可以通过对图的深度/广度优先遍历而得到，称之为深度/广度优先生成树。

在算法中，访问一个结点时 v_j ，同时记录它的父结点 v_i ， (v_i, v_j) 是生成树的一条边。

- 一般情况，BFS(广度优先搜索)生成树的树高小于DFS(深度优先搜索)生成树的高度。
- 一个图的生成树是不唯一的
 - 从不同的顶点出发
 - 采用不同的存储结构和存储顺序



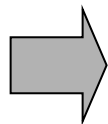
练习：画出下图的生成树



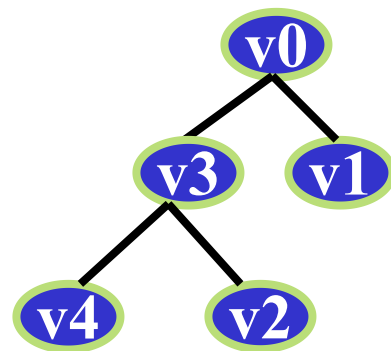
无向连通图，
从v0出发，画出其生成树

- 使用不同的遍历图的方法，可以得到不同的生成树
- 从不同的顶点出发，也可能得到不同的生成树。

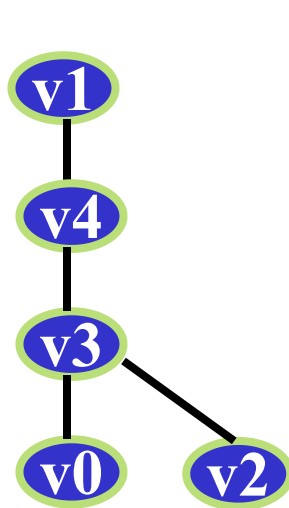
无向连通图，
从v1出发，画出其生成树



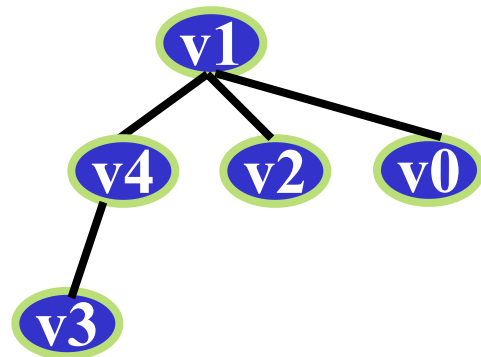
v0出发，DFS生成树



v0出发，BFS生成树

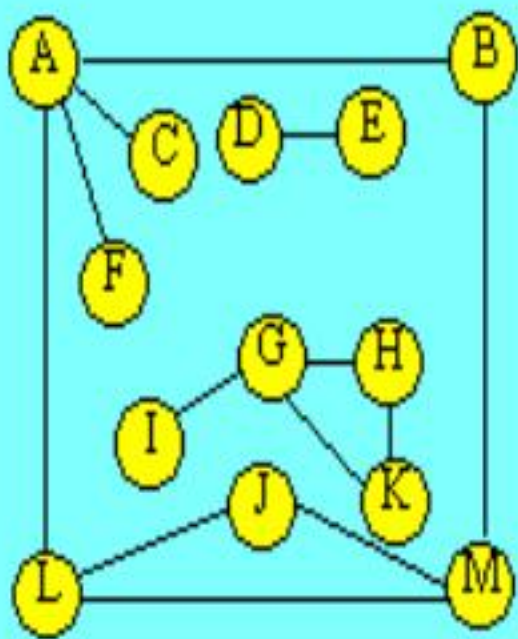


v1出发，DFS生成树

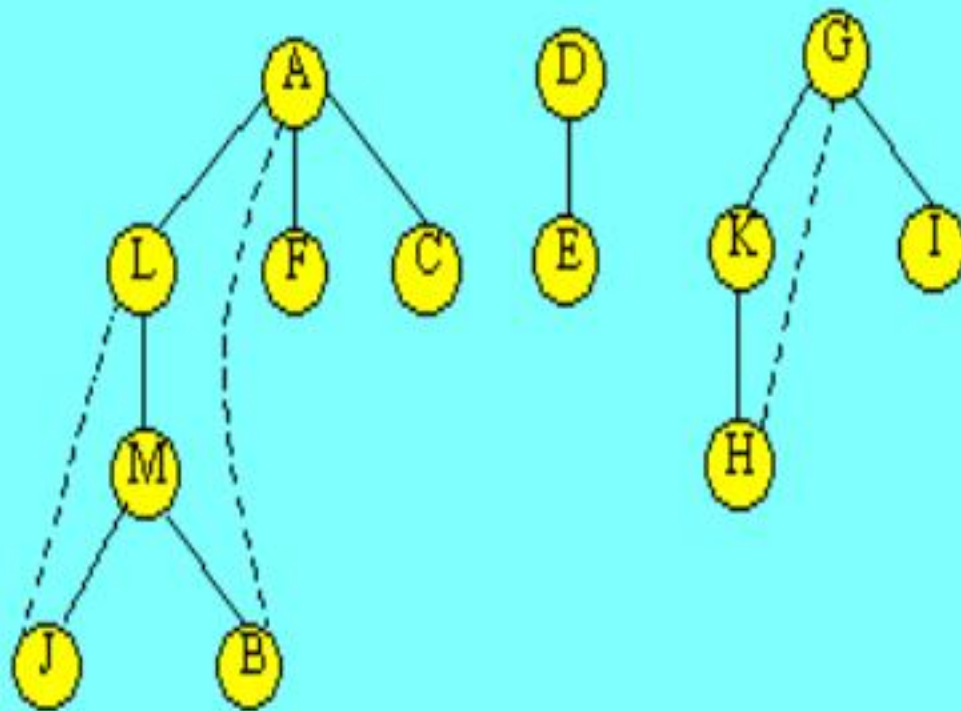


v1出发，BFS生成树

- 对于非连通图，通过这样的遍历，将得到的是生成森林。例如，下图所示为一个无向图及其深度优先生成森林，它由三棵深度优先生成树组成。



(a) 一个非连通图无向图 G6



(b) G6 的深度优先生成森林

求最小生成树

首先明确:

- 使用不同的遍历图的方法，可以得到不同的生成树
- 从不同的顶点出发，也可能得到不同的生成树。
- 按照生成树的定义， n 个顶点的**连通网络**的生成树有 n 个顶点、 $n-1$ 条边。

目标:

在**网**的多个生成树中，寻找一个**各边权值之和最小的生成树**，记为**MST**(Minimum Cost Spanning Tree)。

构造最小生成树的准则:

- ❖ 必须只使用该**网中的边**来构造最小生成树;
- ❖ 必须使用且仅使用 **$n-1$ 条边**来联结网络中的 **n 个**顶点
- ❖ 不能使用产生**回路**的边

最小生成树的典型用途

欲在 n 个城市间建立通信网，则 n 个城市应铺 $n-1$ 条线路；但因为每条线路都会有对应的经济成本，而 n 个城市可能有 $n(n-1)/2$ 条线路，那么，**如何选择 $n-1$ 条线路，使总费用最少？**

显然此连通网是一个**生成树**！

数学模型：

顶点——表示城市，有 n 个；

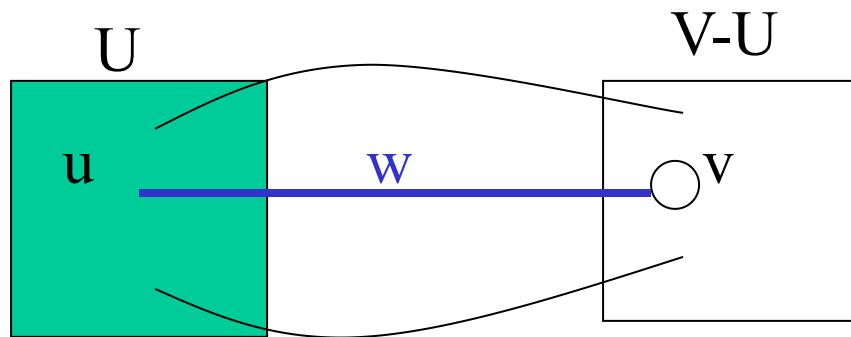
边——表示线路，有 $n-1$ 条；

边的权值——表示线路的经济代价；

连通网——表示 n 个城市间通信网。

MST性质

MST性质 设 $N=(V, \{E\})$ 是一个连通的网络， U 是 V 的真子集，若边 $(u, v)[u \in U, v \in V-U]$ 是 E 中所有一个端点在 U 内，一个端点不在 U 内的边中权值最小的一条边(轻边)，则一定存在 G 的一棵生成树包括此边。



如何求最小生成树

❖ Prim（普里姆）算法

❖ Kruskal（克鲁斯卡尔）算法

Prim算法： 归并顶点，与边数无关，适于稠密网

- 1、将图中顶点分为两个集合，其中集合 X 包含图的一个顶点 v_0 ，集合 Y 包含除 v_0 外的其它所有顶点；
- 2、将跨接这两个集合的权值最小的边加入生成树中，并将其依附在集合 Y 中的顶点 v_1 从 Y 中移入集合 X 中；(找可以连接到目前生成树上的最短边及此边连入的顶点)
- 3、反复过程 2，直到集合 Y 为空，所得生成子图即为最小生成树。

Kruskal算法： 归并边，适于稀疏网

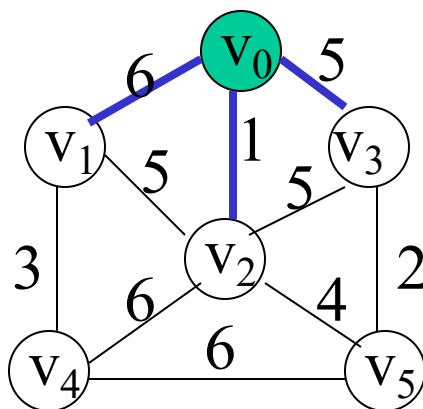
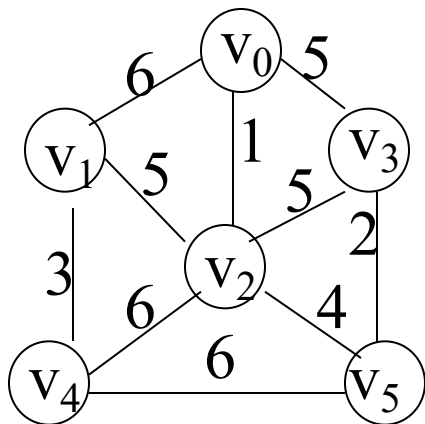
- 1、将图中所有边按权值从小到大排序；只使用所有的顶点形成图（ n 个顶点情况下，形成 n 个连通分量，即所有顶点互不连通）；
- 2、取权值最小的边，加入图中，判断是否形成了回路，未形成回路则保留此边；否则去掉该边，重取权值较小的边；（方法：选两个顶点落在不同的连通分量上的最小权值的边）
- 3、反复过程 2，直到全部顶点均连通为止。

7.4.2.1 Prim算法

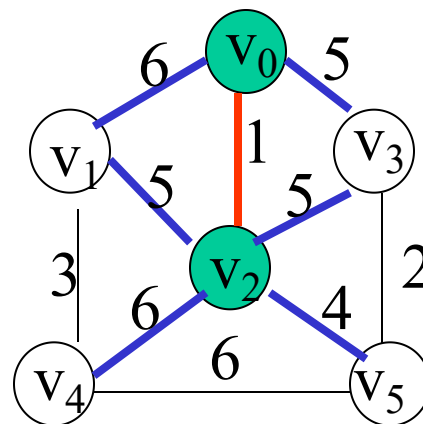
[示例]

将跨接这两个集合的权值最小的边加入生成树

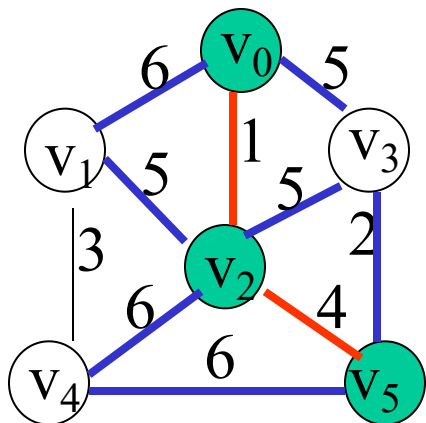
$n=6$



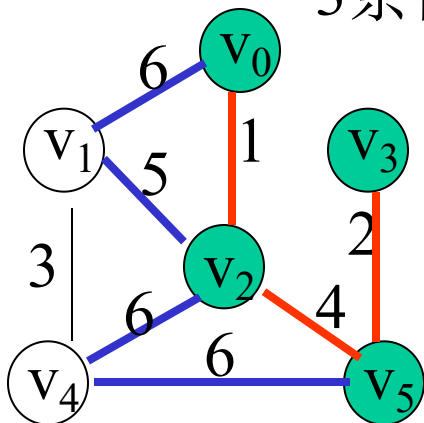
3条候选边



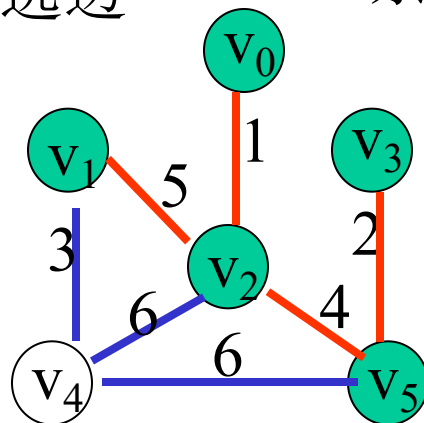
6条候选边



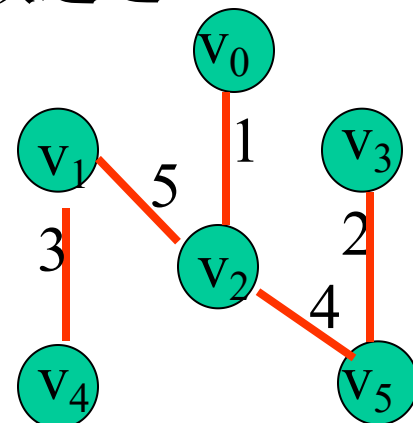
7条候选边



4条候选边



3条候选边



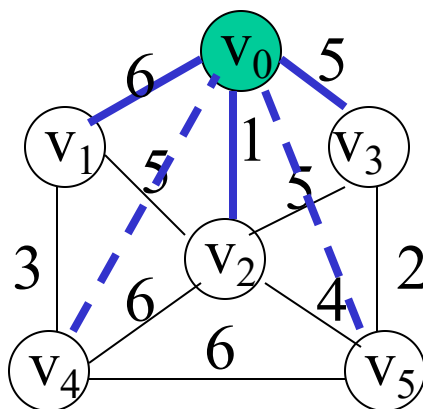
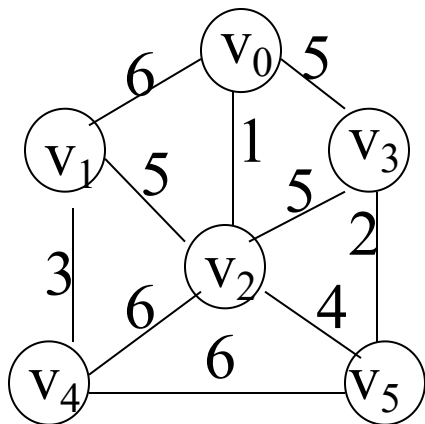
7.4.2.1 Prim算法

[示例]

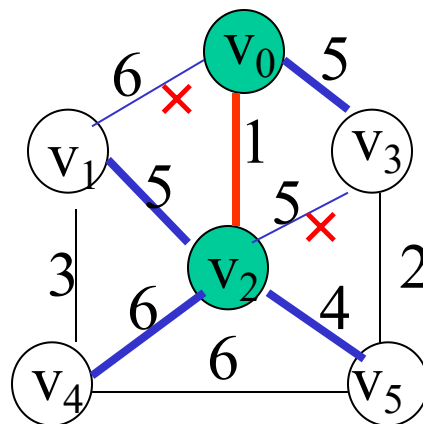
将跨接这两个集合的权值最小的边加入生成树

U中的多个顶点到V-U中一个顶点有多条边时，只选权值最小边做候选边，其他边可以放弃了

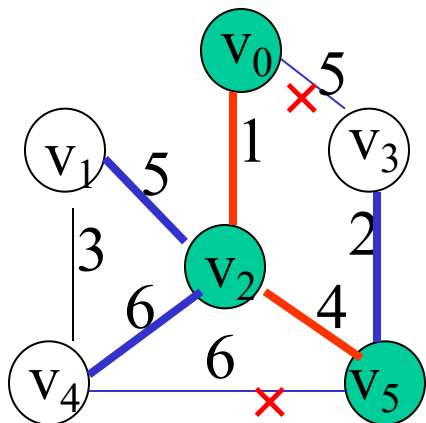
$n=6$



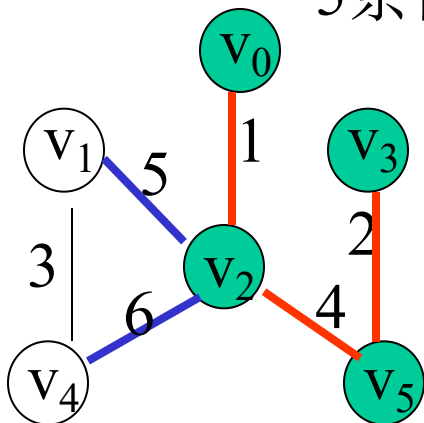
5条候选边



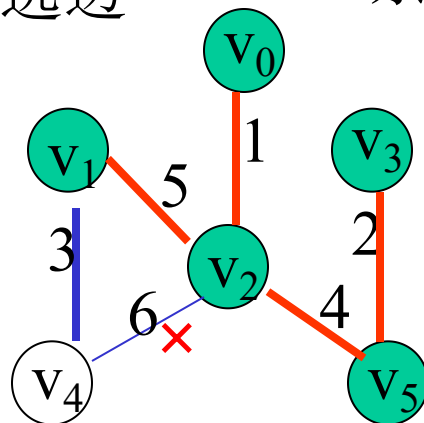
4条候选边



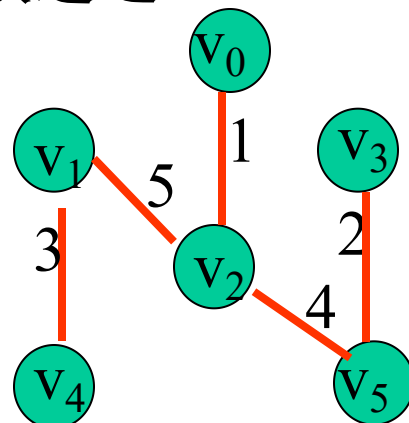
3条候选边



2条候选边



1条候选边



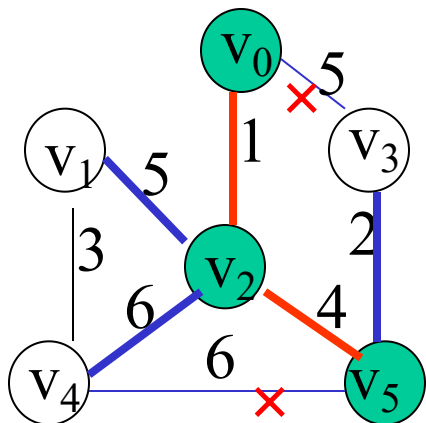
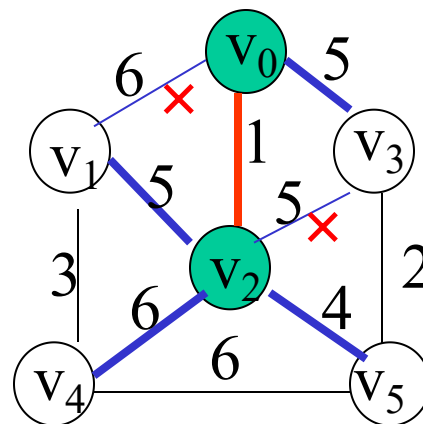
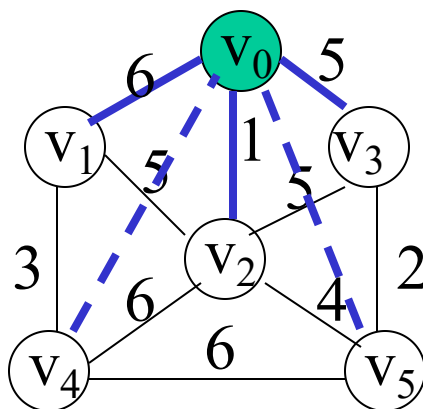
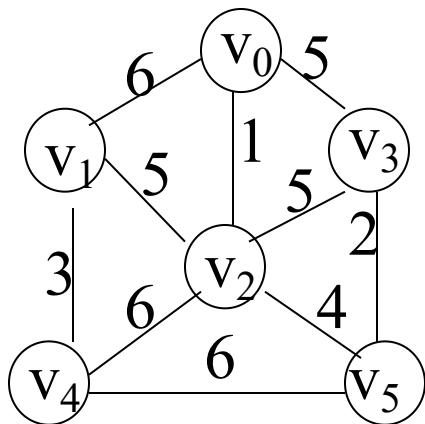
7.4.2.1 Prim算法

[示例]

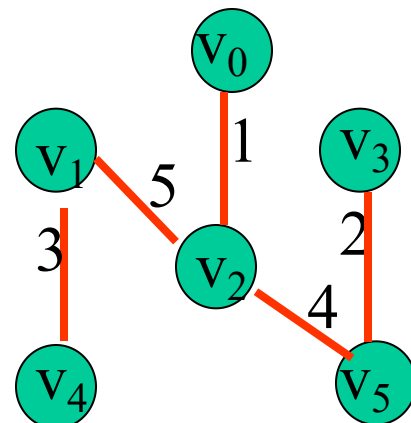
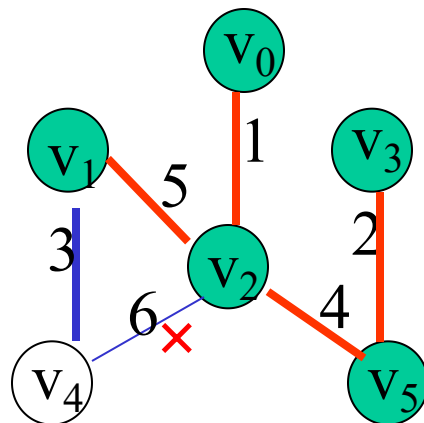
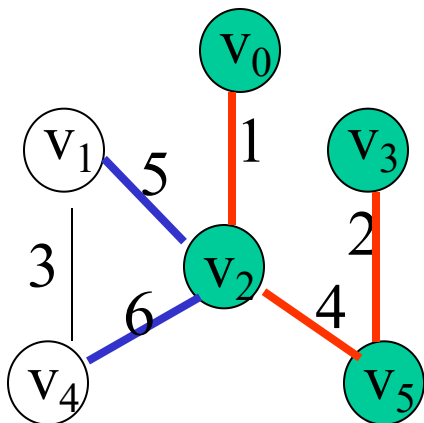
将跨接这两个集合的权值最小的边加入生成树

U中的多个顶点到V-U中一个顶点有多条边时，只选权值最小边做候选边，其他边可以放弃了

$n=6$



3条候选边



[算法步骤] 辅组数组: $\text{closedge}[i](i=0..n-1)$ —记录候选边, 已经纳入U集合的顶点在此数组中的 lowcost 设为0, 不参与候选边的筛选了

以 v 为出发点用prim算法生成最小生成树

1)以顶点 v 在邻接矩阵中的行向量值初始化 $\text{closedge}[j](j=0..n-1)$ $O(n)$

2)重复 $n-1$ 次找其他顶点, 进行以下操作:

2.1)在 $\text{closedge}[j](j=0..n-1)$ 中选择最小且非0的 lowcost , 记录其 j 值 (设为 k)和相应的 adjvex ; 即从候选边中找权值最小的边 $O(n^2)$

2.2)输出该边(adjvex, k); $O(n)$

2.3)顶点 k 并入U集: $\text{closedge}[k].\text{lowcost}=0$; $O(n)$

2.4)调整候选边集 $\text{closedge}[j](j=0..n-1)$: $O(n^2)$

若 $G.\text{arcs}[k][j] < \text{closedge}[j].\text{lowcost}$,

则更改 $\text{closedge}[j]$: $\text{adjvex}=k, \text{lowcost}=G.\text{arcs}[k][j]$

//调整方法: U中新增顶点 k 带来的和 j 关联的边(k, j) 权值小就替换已有候选边,

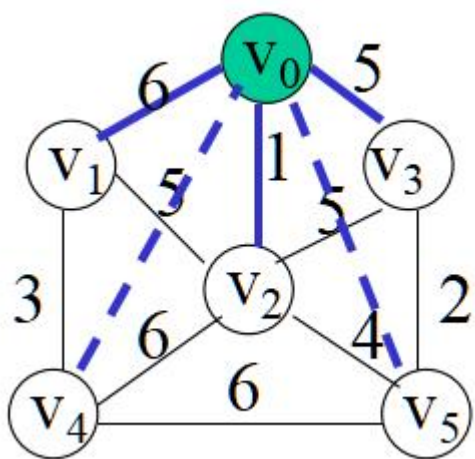
//因 $\text{closedge}[j]$ 中记录了现在为止权值最小边, 新增边与其比较选一个小的即可

prim算法复杂度: $O(n^2)$

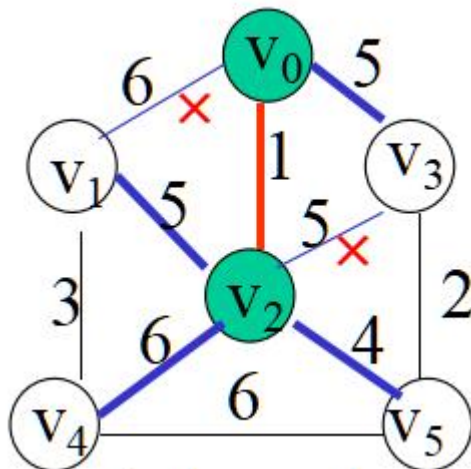
[算法描述]

```
void Prim(MGraph G, VertexType u )
{
    //用v和其他结点的边权值初始化辅助数组
    k=LocateVex(G,u);
    for(j=0; j<G.vexnum; ++j) //即为v结点在邻接矩阵的k行数据
        closedge[j]={k, G.arcs[k][j].adj};
    //在剩余结点中循环,
    //依次找出最小生成树的边权值最小并且连接U和V-U的边和结点
    for (i=1; i<G.vexnum; ++i) { //进行G.vexnum-1次循环
        k=minium(closedge); //选择权值最小的边对应的V-U中的顶点
        printf(closedge[k].adjvex, k);
        closedge[k].lowcost=0; //顶点k并入U集
        for (j=0; j<G.vexnum; ++j) //用k带来的边调整候选边
            if (G.arcs[k][j].adj<closedge[j].lowcost)
                closedge[j]={k, G.arcs[k][j].adj};
    }
}
} //Prim
```

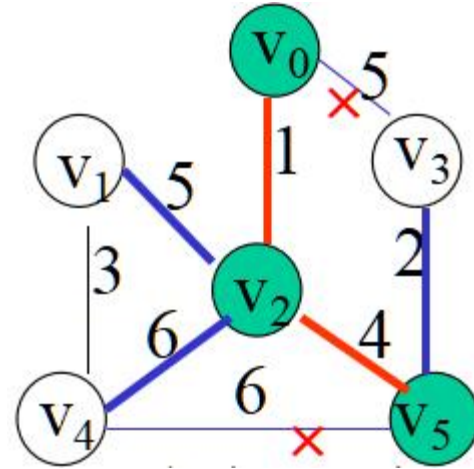
```
struct {
    int      adjvex;
    VRType   lowcost;
}closedge[MAX_VERTEX_NUM];
```



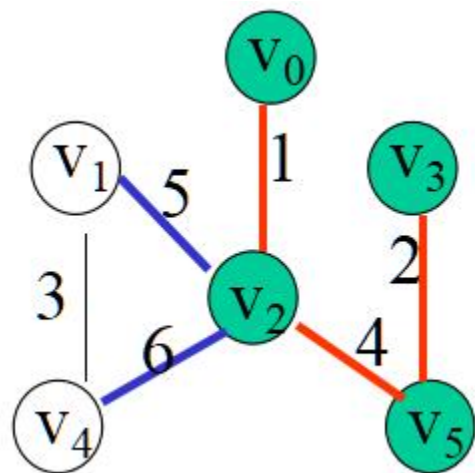
初始	lowcost	0	6	1	5	∞	∞
	adjvex	0	0	0	0	0	0



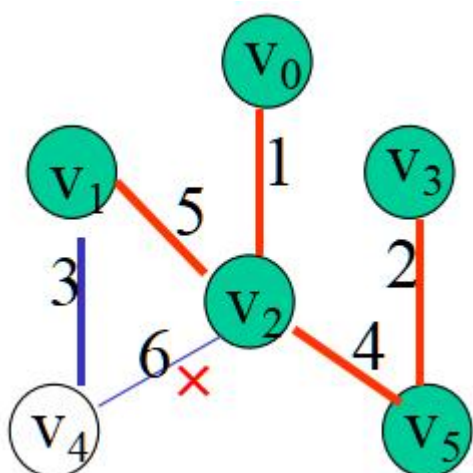
第一次 迭代后	lowcost	0	5	0	5	6	4
	adjvex	0	2	0	0	2	2



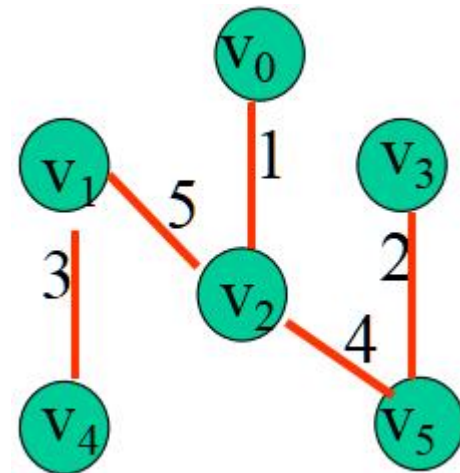
第二次 迭代后	lowcost	0	5	0	2	6	0
	adjvex	0	2	0	5	2	2



第三次 迭代后	lowcost	0	5	0	0	6	0
	adjvex	0	2	0	5	2	2



第四次 迭代后	lowcost	0	0	0	0	3	0
	adjvex	0	2	0	5	1	2



第五次 迭代后	lowcost	0	0	0	0	0	0
	adjvex	0	2	0	5	1	2

从顶点j向开始顶点回溯的一条路径

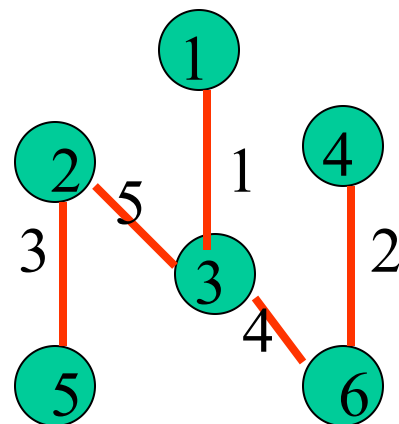
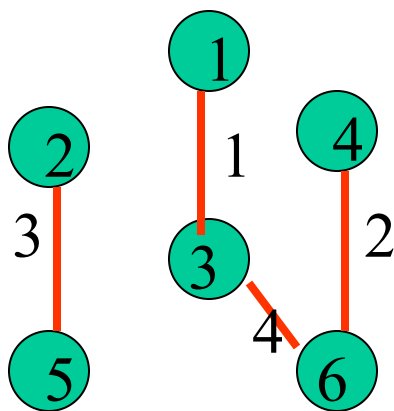
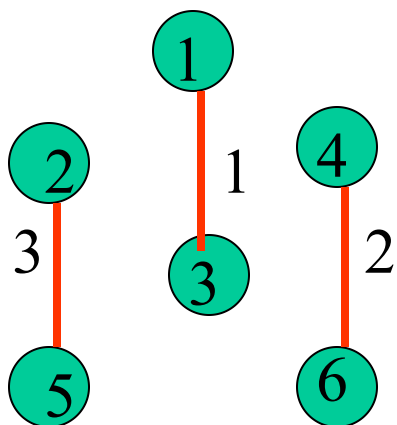
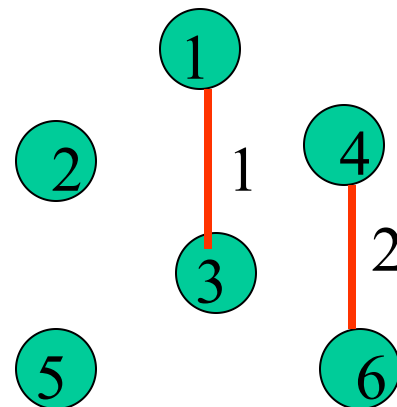
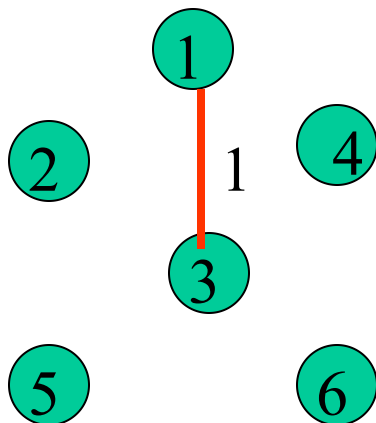
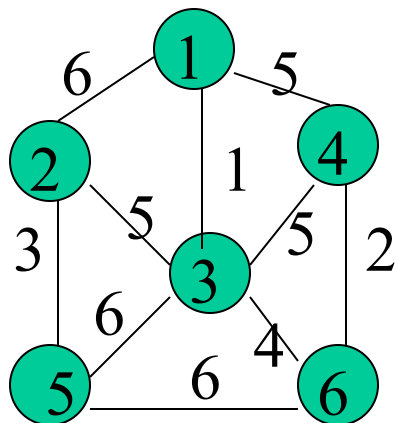
7.4.2.2 Kruskal算法

选两个顶点落在不同的连通分量上的最小权值的边

[示例] $n=6$

边表

v1	1	4	2	3	3	2	1	5	3	
v2	3	6	5	6	4	3	2	6	5	
w	1	2	3	4	5	5	6	6	6	



[算法步骤]

- 1) 初始化T: 顶点集=所有顶点, 每个独立的顶点作为一棵树, 边集= $\emptyset$; $O(n)$
 - 2) 依权值递增序对图G的边排序, 结果为 $E[0..e-1]$ $O(elge)$
 - 3) 依次检测E中的各边(u,v): $O(elge)$
 - 3.1) 若u和v分属于T中两棵不同的树, 则将该边加入T, 并合并u和v分属的两棵树
 - 3.2) 若T中所有顶点尚未属于一棵树, 转3)
-

$O(elge)$

其中3) 可参阅教材139页“树与等价问题”

[算法特点] 适用于稀疏图

比较两种算法

算法名	普里姆算法	克鲁斯卡尔算法
方法	基于MST性质	加边
时间复杂度	$O(n^2)$	$O(eloge)$
适应范围	稠密图	稀疏图

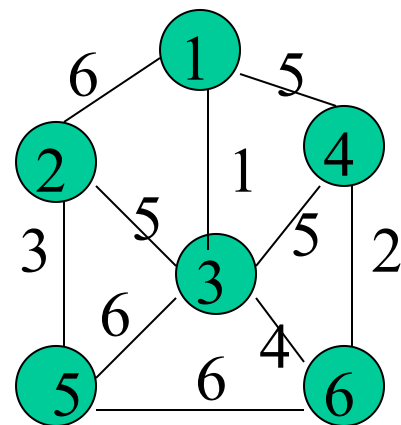
Prim算法内层循环是当新增加一个顶点到生成树后，A：需要更新所有未加入生成树的顶点j到树的边（原有边和新的经过k到j的边中选最小的）；B：在更新还需要在所有的j中选边长度最小的；两个操作都需要对数组进行遍历 $O(n)$ 。外层需要对所有顶点循环一次，总体为 $O(n^2)$ 。

换一种估算方法：整体算法的A需要对所有边操作一次($O(e)$)，B需要对所有顶点操作，因此是 $O(n^2)$ ，由于 $e < n^2$ 作，总体为 $O(n^2)$

Prim算法改进：使用一种“斐波那契堆”，可以使得B操作的总体时间复杂度降低到 $O(n \log_2 n)$ ，从而使得整个算法时间复杂度降低到 $O(n \log_2 n + e)$

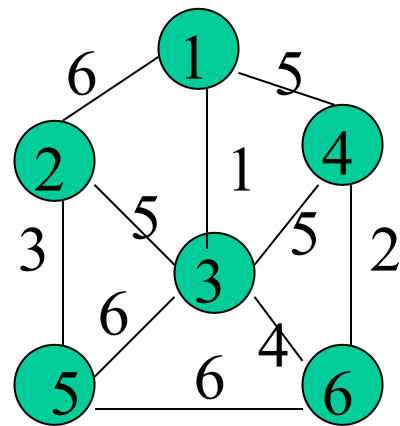
三、破圈法：

- 1、在图中找到一个回路；
- 2、去掉该回路中权值最大的边；
- 3、反复此过程，
直到图中不存在回路为止。



四、去边法：

- 1、将图中所有边按权值从大到小排序；
- 2、去掉权值最大的边，若图不再连通则保留此边，再选取下一权值较大的边去掉；
- 3、反复此过程，直到图中只剩下 $n-1$ 条边



7.5 有向无环图及其应用

(**DAG**, Directed Acyclic Graph)

7.5.1 拓扑排序



? 假设以有向图表示一个工程的施工图或程序的数据流图，则图中不允许出现回路。

[问题目标]

当一个任务可以划分为若干个子任务/子活动/子事件，其中的任何子任务可能又以另外的一些子任务作为先决条件时，如何排定子任务的执行顺序，达到整体任务的完成。

[什么是拓扑排序]

按照有向图给出的次序关系，将图中顶点排成一个**线性**序列。

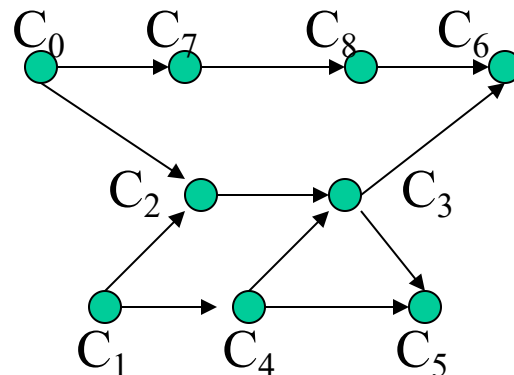
检查有向图中是否存在回路的方法之一，是对有向图进行**拓扑排序**。

[示例]

课程之间的依赖关系及其拓扑次序排列。

计算机专业的必修课

课程代号	课程名称	先修课
C ₀	高等数学	无
C ₁	程设基础	无
C ₂	离散数学	C ₀ , C ₁
C ₃	数据结构	C ₂ , C ₄
C ₄	程设语言	C ₁
C ₅	编译原理	C ₃ , C ₄
C ₆	操作系统	C ₃ , C ₈
C ₇	普通物理	C ₀
C ₈	计算机原理	C ₇



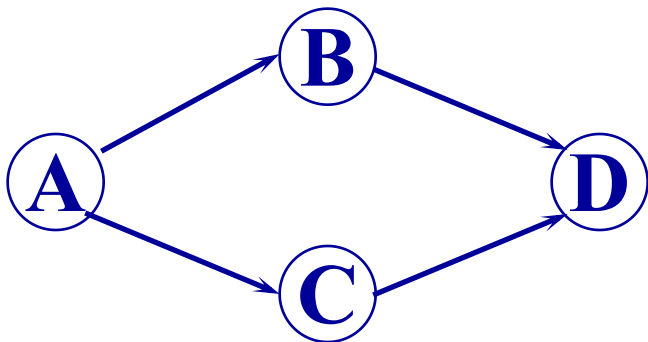
[问题的表示方法]

AOV网 (边活动图 activity on edge network)

- 顶点表示活动
- 弧表示活动间的优先关系

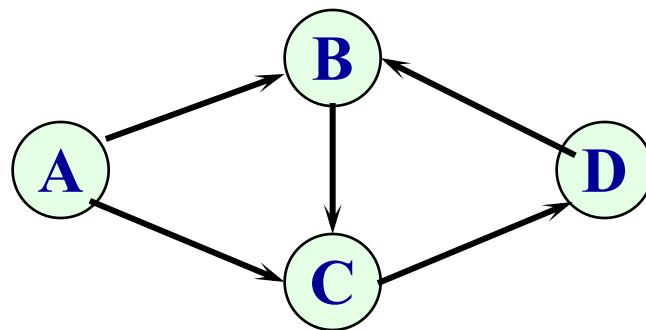
[可拓扑排序的条件]

有向无环图



可求得拓扑有序序列:

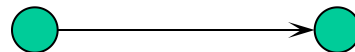
ABCD 或 ACBD



不能求得它的拓扑有序序列。

因为图中存在一个回路 {B, C, D}

[无前趋的顶点优先算法]



算法原理

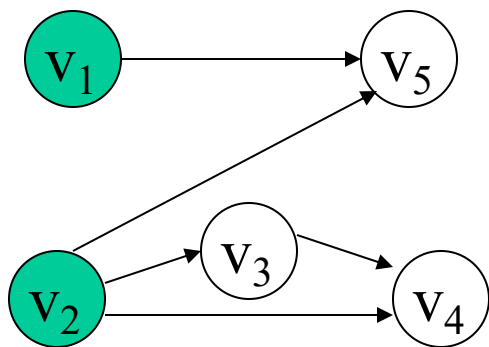
在一个拓扑序列中，每个顶点必定出现在它的所有前趋顶点之后。

算法思想

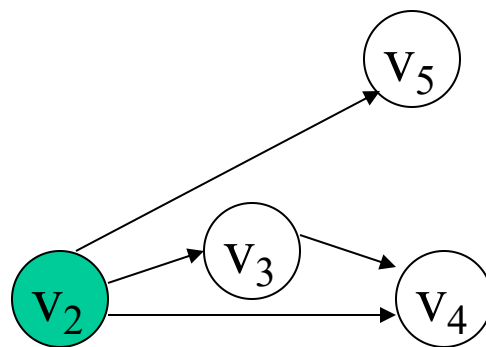
1. 选择一个入度为0的顶点(无前趋的顶点)，输出它
 2. 删去该顶点及其关联的所有出边
- 重复上述两步，直至图中不再有入度为0的顶点为止。
若所有顶点均被输出，则排序成功，
否则图中存在有向环。

若该有向图不是无环图，则不存在拓扑排序。可以用来判断一个有向图是否存在回路。

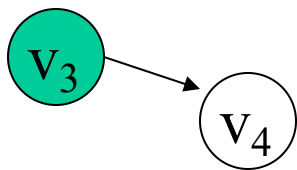
例



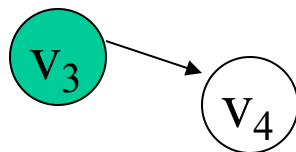
V_1



$V_1 V_2$



$V_1 V_2 V_5$



$V_1 V_2 V_5 V_3$



$V_1 V_2 V_5 V_3 V_4$

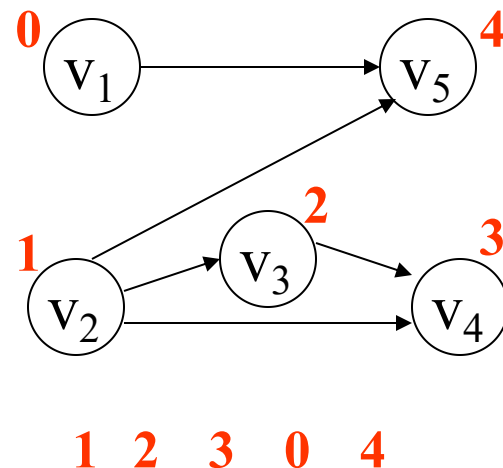
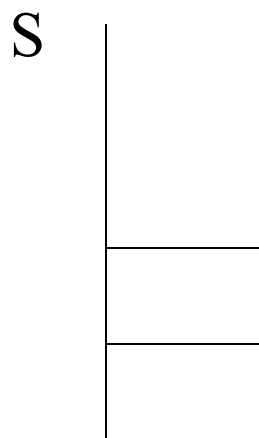
有向无环图的拓扑序列不唯一

数据结构

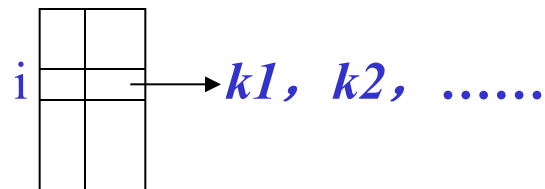
- 主：邻接表G（或者邻接矩阵）
- 辅：
 - 一维数组indegree[0..n-1]：记录每个顶点的当前入度
 - 栈S（或者队列Q）：暂存当前所有待输出的入度为0的顶点。作用是避免每次扫描indegree，提高算法效率。

例中各辅助数据结构初值如下：

indegree	0	0
	1	0
	2	0
	3	0
	4	0



算法步骤



1)初始化辅助数据结构:

1.1)扫描邻接表G的各出边表, 在indegree中计数; $O(n+e)$

1.2)检查indegree数组, 将入度为0的顶点入栈; $O(n)$

1.3)输出顶点计数器 $count = 0$ $O(1)$

2)若栈不空, 反复执行以下操作:

2.1)栈顶元素*i*出栈; $O(n)$

2.2)输出顶点*i*, $count$ 加 1; $O(n)$

2.3)扫描顶点*i*的出边表: $O(n+e)$

修改indegree, 将出边表上顶点的入度减 1;

若其入度为 0, 则令其入栈

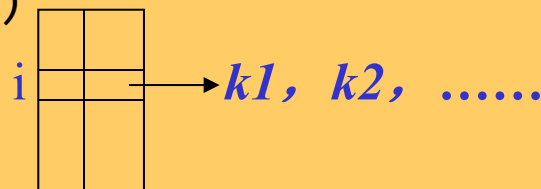
3)若 $count < \text{顶点数 } vexnum$, write (“排序失败! ”) $O(1)$

$O(n+e)$

[算法实现---图采用邻接表数据结构]

Status **TopologicalSort**(ALGraph G)

```
{ FindInDegree(G,indegree); //求各顶点入度indegree[G.vexnum]
  InitStack(S); // InitQueue(Q);
  for (i=0; i<G.vexnum; ++i)
    if (!indegree[i]) push(S, i); // EnQueue(Q,i)
  count=0;
  while (! StackEmpty(S)) // (!QueueEmpty(Q))
  {
    pop(S, i); //DeQueue(Q, i);
    printf(i, G.vertices[i].data); ++count;
    for(p=G.vertices[i].firstarc;p;p=p->nextarc)
    { //修改入度
      k=p->adjvex;
      //入度为0则入栈(入队列)
      if(!(--indegree[k])) push(S,k); //EnQueue(Q,i)
    } //for
  } //while
  if (count<G.vexnum) return ERROR;
  else return OK;
} //TopologicalSort
```



[无后继的顶点优先]

算法原理

在一个拓扑序列中，每个顶点必定出现在它的所有后继顶点之前。

方法一

（按逆拓扑次序生成顶点序列）

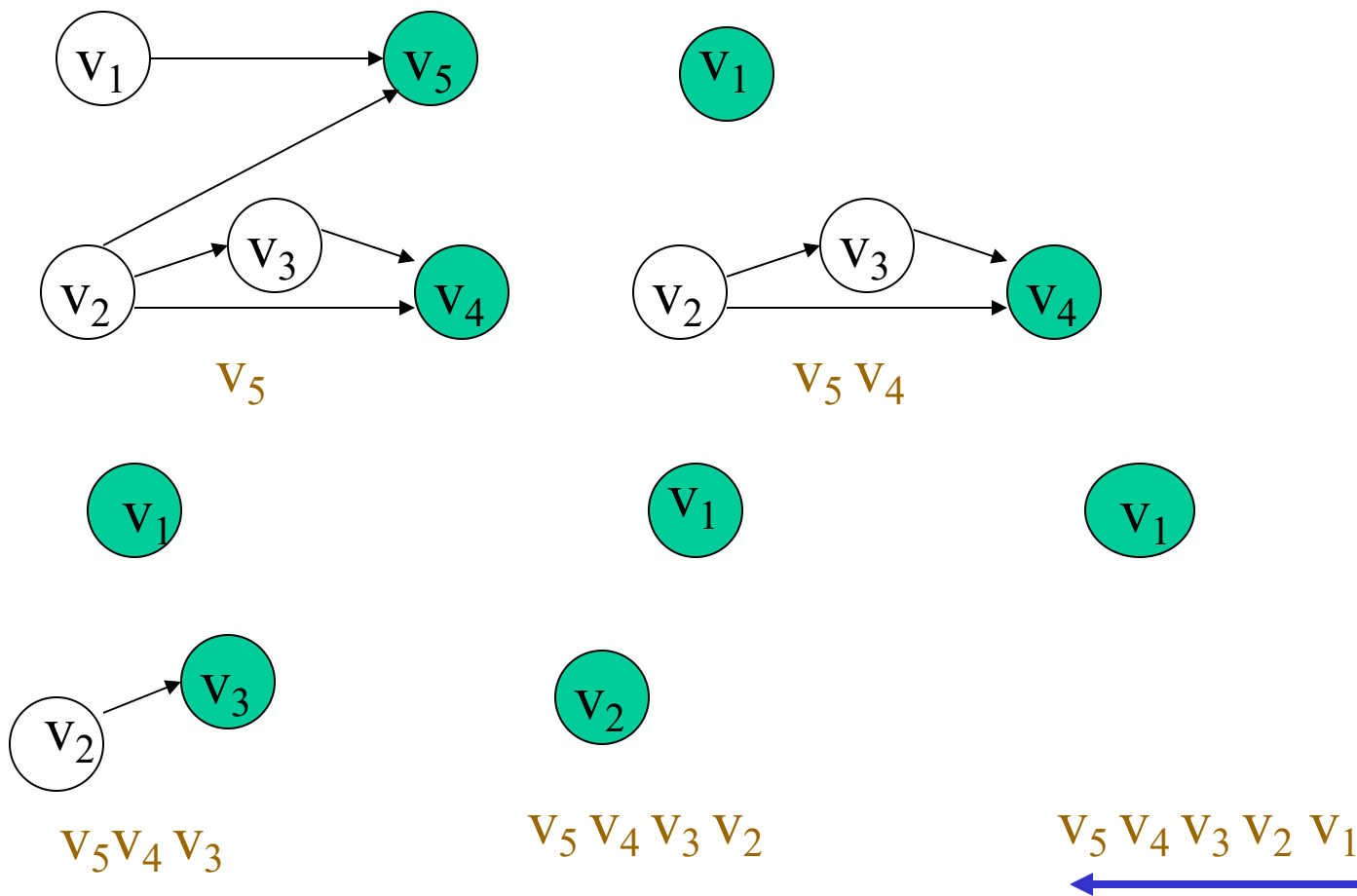
1. 选择一个出度为0的顶点(无后继的顶点)，输出它
2. 删去该顶点及其关联的所有入边

重复上述两步，直至图中不再有出度为0的顶点为止。

若所有顶点均被输出，则排序成功，

否则图中存在有向环。

例



[算法实现]

Status TopologicalSort(ALGraph G)

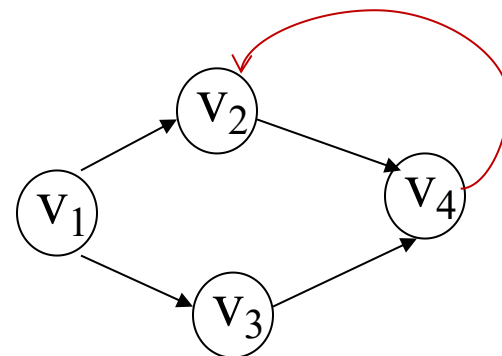


```
{
    FindInDegree(G, outdegree); //求各顶点出度 outdegree[G.vexnum]
    InitStack(S);
    for (i=0; i<G.vexnum; ++i)
        if (! outdegree[i]) push(S, i);
    count=0;
    while (! StackEmpty(S))
    {
        pop(S, i); printf(i, G.vertices[i].data); ++count;

        //对 G.vertices[i] 顶点, 找其“弧尾/初始点” k
        //修改k的出度
        //若k的出度为0, k入栈

    } //while
    if (count<G.vexnum) return ERROR;
    else return OK;
} //TopologicalSort
```

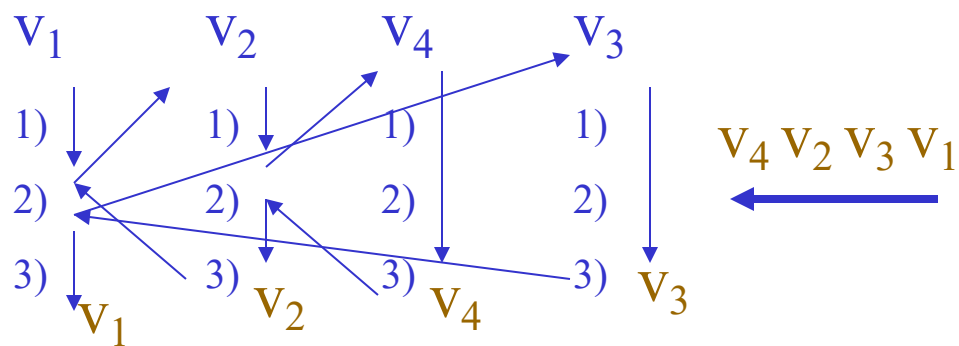
方法二 利用深度优先遍历 (从入度为0的顶点出发)



- 1) 访问顶点 v ，并记录 v 已被访问
- 2) 依次从 v 的未访问的邻接点出发，深度优先拓扑排序图 G 。
- 3) 输出顶点 v （此时 v 相当于无后继的顶点）

注意：此方法仅适用于有向无环图

（此算法不能检测出有向环，得到虚假拓扑序列）



7.5.2 关键路径

7.5.2.1 概述

[问题目标]

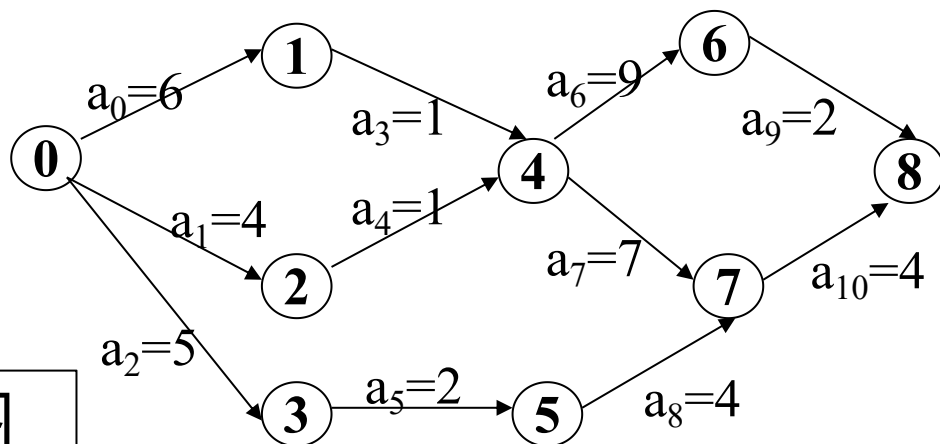
当一个任务/工程可以划分成若干个子任务/子活动/子工程，其中的任何子任务可能又以另外的一些子任务作为先决条件，此时，我们通常会关心两个问题：

- 1、整个工程至少需要多少时间
- 2、哪些活动是影响进度的关键

这类问题，可以用有向无环图来表示：顶点(vertex)用来表示事件(event)，弧(edge)用来表示活动(activity)，权值表示活动所需时间(dut)，称为边活动图(activity on edge network)，简称AOE网(AOE图)。

图中整个工程所需时间为：18

关键活动为： a_0 、 a_3 、 a_6 、 a_7 、 a_9 、 a_{10}



[AOE网的性质]

进入 V_i 的活动都结束， V_i 所代表的事件才能发生

顶点 V_i 所代表的事件发生，从 V_i 出发的活动才能开始

[相关概念和术语]

源点 入度为0的顶点，即工程的开始点

汇点 出度为0的顶点，即工程的完成点

活动持续时间 活动 a_i 由弧 $\langle j, k \rangle$ 表示，用 $\text{dut}(\langle j, k \rangle)$ 表示持续时间

关键路径 从源点到汇点的最长路径，关键路径长度=最短工期

关键活动 关键路径上的活动，关键活动的加快可以缩短工期

关键活动的判断方法： $e(i)=l(i)$

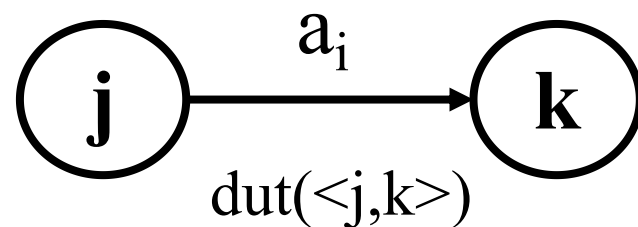
[确定关键路径时涉及的几个变量]

$ve(j)$: 事件 v_j earliest time of vertex

$vl(j)$: 事件 v_j latest time of vertex

$e(i)$: 活动 a_i earliest time of edge

$l(i)$: 活动 a_i latest time of edge



由变量含义可得:

$$e(i) = ve(j)$$

$$l(i) = vl(k) - \text{dut}(\langle j, k \rangle)$$

[求关键路径算法步骤]

主数据结构：图—邻接矩阵或邻接表

辅助数组： $ve(0..n-1)$, $vl(0..n-1)$, $e(0..m-1)$, $l(0..m-1)$

{ n 为图的顶点数目； m 为图的弧数目}

T —栈，存储拓扑排序的顶点序列

(1)输入所有弧的信息，建立AOE网的存储结构

(2)计算 $ve[]$ 数组值，事件的最早开始时间：

先进行拓扑排序；

每求得一个顶点 j ，则使 j 入栈 T ，并调整 j 的所有后继的 ve 值

(3)计算 $vl[]$ 数组值，事件的最迟开始时间：

按栈 T 的从栈顶到栈底的顺序求顶点的 vl 值

(4)计算 $e[]$ 和 $l[]$ 数组值，活动的最早开始时间和最迟开始时间

求每个活动 i 的 $e(i)$ 和 $l(i)$ 值；

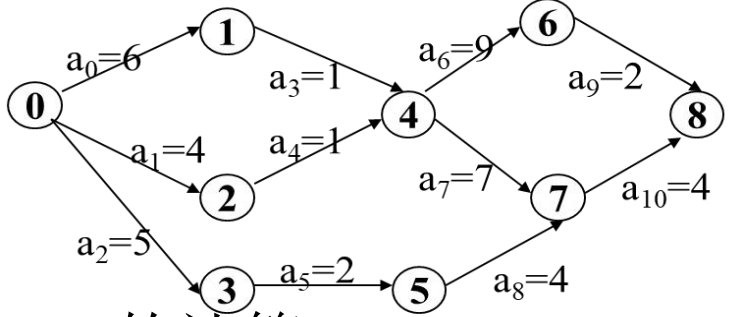
依据 $e(i)=l(i)$ 确定关键事件，再确定关键路径

1) 求事件 v_j 的最早发生时间 $ve(j)$

$$\begin{cases} ve(0)=0 \\ ve(j)=\max\{ve(i)+dut(<i,j>)\} (i \in j \text{的前驱}) \end{cases}$$

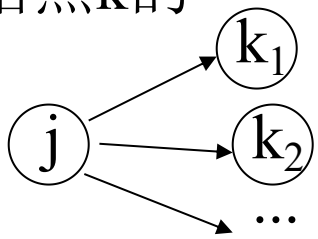
[算法思想]: 在求拓扑排序的过程中数组 $ve[]$ 的计算:

- 1)初值 $ve[j]=0 \quad 0 \leq j \leq n-1$
- 2)每输出一个无前驱的顶点 j 后, 修正 j 的所有后继结点 k 的



例图的拓扑序列1:

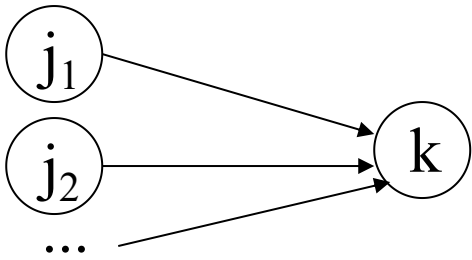
	0	1	2	3	4	5	6	7	8
j	0	1	2	3	4	5	6	7	8
ve(j)	0	6	4	5	7	7	16	14	18



$$ve[k_n]=\max(ve[j] + dut(<j,k_n>))$$

例图的拓扑序列2:

	0	1	2	3	4	5	6	7	8
j	0	1	2	3	4	5	6	7	8
ve(j)	0	6	4	5	7	7	16	11	18



$ve[k]$ 由不同的前驱 j_n 算出时, 每次保留最大的

2) 求事件 v_j 的最迟发生时间 $vl(j)$

$$\begin{cases} vl(n-1)=ve(n-1) & \{\text{保证}ve(n-1)\text{的前提下}\} \\ vl(j)=\min\{vl(k)-dut(<j,k>)\} & (k\in j\text{的后继}) \end{cases}$$

[算法思想]：按逆拓扑序列逐一计算。

1)初值 $vl[j]=ve[j]$ $0\leq j\leq n-1$

2)按拓扑排序结果，逆序计算 $\min\{vl(k)-dut(<j,k>)\}$

例图的拓扑序列1:

012345678

j

0

1

2

3

4

5

6

7

8

vl(j)

0

6

6

8

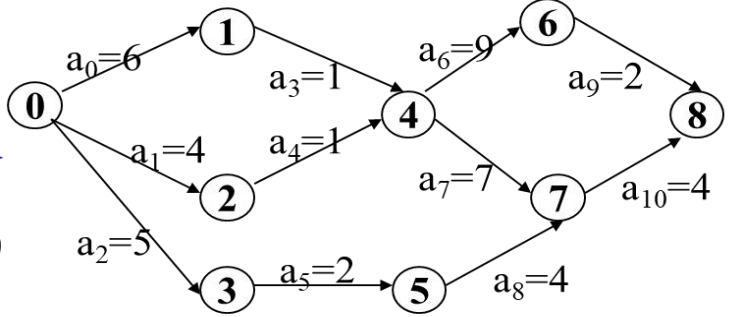
7

10

16

14

18



j

k₁

k₂

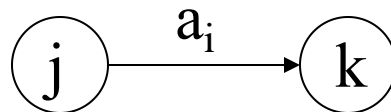
...

$vl[j]=\min(vl[k_n]$
 $- dut(<j,k_n>))$

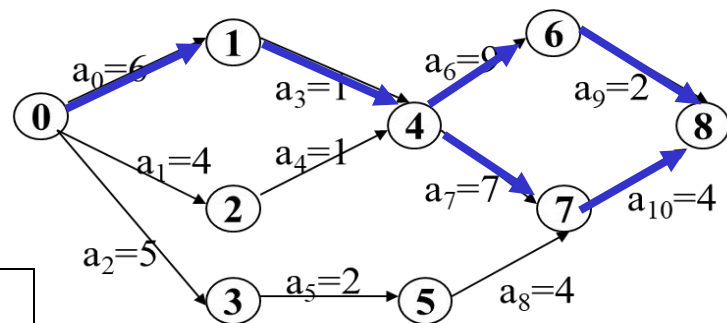
$vl[j]$ 由不同的后继 k_n 算出时，每次保留最小的

3) 求活动 a_i 的最早开始时间 $e(i)$ 和最迟开始时间 $l(i)$ 确定关键活动 $e(i)=l(i)$

$$\begin{cases} e(i)=ve(j) \\ l(i)=vl(k)-dut(<j,k>) \end{cases}$$



j	0	1	2	3	4	5	6	7	8
ve(j)	0	6	4	5	7	7	16	14	18
vl(j)	0	6	6	8	7	10	16	14	18

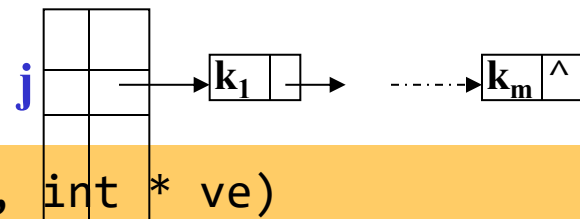


i	0	1	2	3	4	5	6	7	8	9	10
e(i)	0	0	0	6	4	5	7	7	7	16	14
l(i)	0	2	3	6	6	8	7	7	10	16	14

两条关键路径
最短工期：18

依据 **$e(i)=l(i)$** 确定关键事件，再确定关键路径

[算法描述-Part1]



Status **TopologicalOrder**(ALGraph G, Stack &T, int * ve)

//计算各顶点的最早发生时间保存在ve中，并用栈T返回G的一个拓扑序列
{

FindInDegree(G, indegree); //求各顶点入度indegree[G.vexnum]

InitStack(S); count=0; ve[0..G.vexnum-1]=0;

for (i=0; i<G.vexnum; ++i) if (! indegree[i]) push(S, i);

InitStack(T);

while (! StackEmpty(S)) {

pop(S, j); push(T, j); ++count;

for (p=G.vertices[j].firstarc; p; p=p->nextarc){

k=p->adjvex;

if (! (--indegree[k])) push(S,k); //若入度为0则入栈

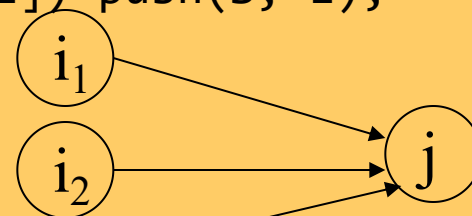
if (ve[j]+(p->weight) > ve[k]) ve[k]= ve[j]+(p->weight);

}//for

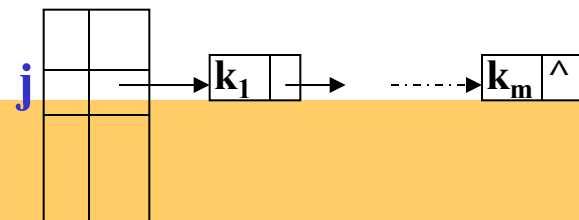
}//while

if (count<G.vexnum) return ERROR; else return OK;

}//TopologicalOrder



[算法描述-Part2]



Status **CriticalPath**(ALGraph G)

//G为有向网，输出G的各项关键活动

```
{ if (! TopologicalOrder(G,T,ve) ) return ERROR;
  vl[0.. G.vexnum-1] = ve[G.vexnum-1] ;
  while (! StackEmpty(T)){    //按拓扑逆序求各顶点的vl值
    for ( pop(T, j), p=G.vertices[j].firstarc; p; p=p->nextarc) {
      k=p->adjvex;
      if (vl[k]-(p->weight) < vl[j]) vl[j]= vl[k]-(p->weight);
    }//for
    for (j=0; j<G.vexnum; ++j){  //求各活动的ee、el和确定关键活动
      for (p=G.vertices[j].firstarc; p; p=p->nextarc) {
        k=p->adjvex;  dut=p->weight;
        ee=ve[j]; el=vl[k]-dut;
        tag=(ee==el) ? '*' : ' ';
        printf(j, k, dut, ee, el, tag);
      } //for
    } //for
  } //while
} //CriticalPath
```

用变量ee(earliest time
of edge)来代替e[j]

用变量el(latest time of
edge)来代替l[j]

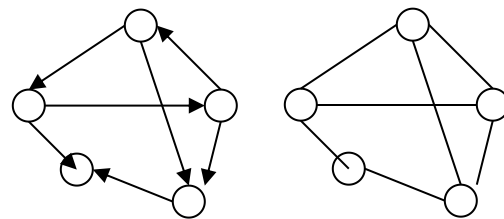
7.6 最短路径

7.6.1 概述

交通网、通信网中要解决的应用问题：两地之间是否有路相通？在多条通路的情况下，哪一条最短？

[问题对象]

有向网络（可扩展到无向网络）



[最短路径定义]

单源最短路径：从某源点到其它所有顶点间的最短路径。

每一对顶点间的最短路径：各顶点到其它所有顶点间的最短路径。

[两个典型算法]

- 单源最短路径（可扩展到多源问题）

迪杰斯特拉(Dijkstra)算法

- 每一对顶点间的最短路径

弗洛伊德(Floyd)算法

7.6 最短路径

7.6.1 概述

有路

FOCS'24最佳论文

时隔近70年，那个用来解决最短路径问题的经典算法——Dijkstra，现在有了新突破：

“ 被证明具有普遍最优性 (Universal Optimality) 。

什么意思？

这就意味着不论它面对多复杂的图结构，即便在最坏情况下都能达到理论上的最优性能！

最短路径。

本科经典算法Dijkstra，被证明是普遍最优了：最坏情况性能也最优！

<https://www.36kr.com/p/3010282055443718>

Universal Optimality of Dijkstra Via Beyond-Worst-Case Heaps(This paper proves that **Dijkstra's shortest-path algorithm is universally optimal in both its running time and number of comparisons when combined with a sufficiently efficient heap data structure.**)

<https://ieeexplore.ieee.org/document/10756107>

最短路径

我们经常需要求解从某点（源点--source）出发到达另一点（终点--destination）的最短路径。当然，我们可以先求出这两点间的所有简单路径，再从中选出最短路径，但这种方法未免太低效率了，有没有更好的办法呢？

Dijkstra

单源最
短路径

从源点的所有邻接点中找出距源点最近的邻接点，这即是源点到该点的最短路径(S直达第一个顶点)；

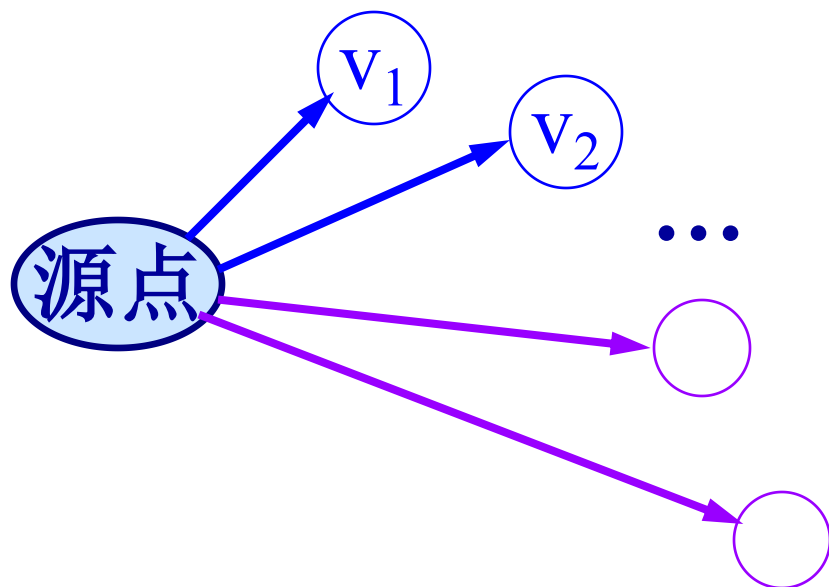
以该邻接点为中间点，比较其它各顶点与源点的路径通过此中间点后是否更短，若是则用新路径代替老路径，再从中找出其它各点距源点最近的顶点作为新的中间点，这也是这个新的中间点的最短路径；

反复此过程，直至全部顶点距源点的最短路径均找到为止。

7.6.2 从某个源点到其余各顶点的最短路径

[Dijkstra算法原理]

按路径长度递增次序产生源点S到其余各顶点的最短路径。



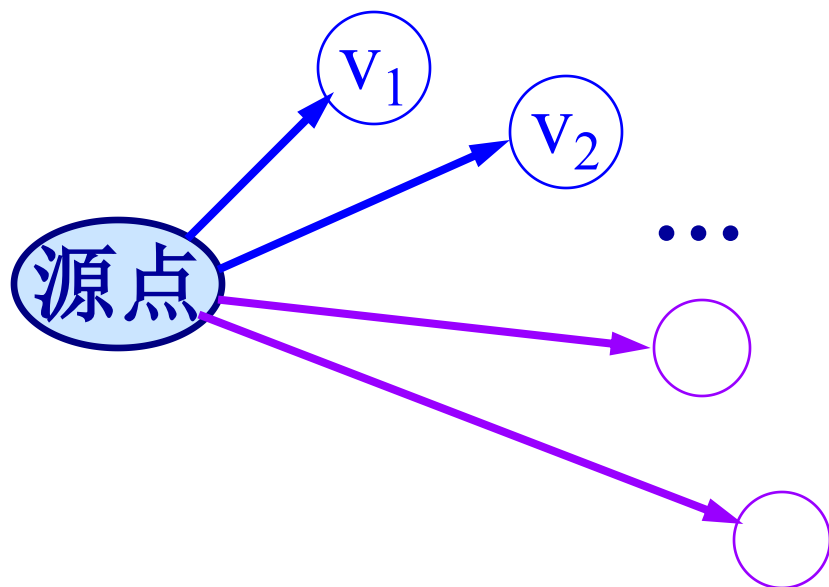
设从源点到顶点 v_1 的最短路径是从源点到所有顶点的最短路径中长度最短者。

路径长度最短的最短路径的特点:在这条路径上,必定只含一条弧,并且这条弧是所有从源点出发的弧中权值最小。(用反证法可证)

7.6.2 从某个源点到其余各顶点的最短路径

[Dijkstra算法原理]

按路径长度递增次序产生源点S到其余各顶点的最短路径。



设从源点到顶点 v_1 的最短

路径长度为 d_1 ，从源点到所有其他

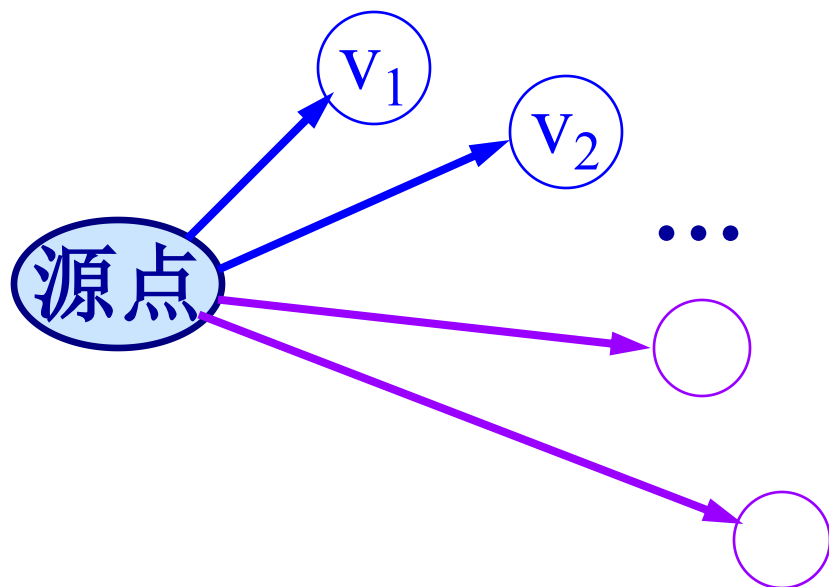
下一条路径长度次短的最短
路径的特点：

它只可能是下列两种情况之一：或者是直接从源点到该点（只含一条弧）；或者是，从源点经过顶点 v_1 ，再到达该顶点（由两条弧组成）。

7.6.2 从某个源点到其余各顶点的最短路径

[Dijkstra算法原理]

按路径长度递增次序产生源点S到其余各顶点的最短路径。



设从源点到顶点 v_1 的最短

路径已经求得，下一条

路径长度次短的最短

路径递推可得，其余最短路径的

特点：

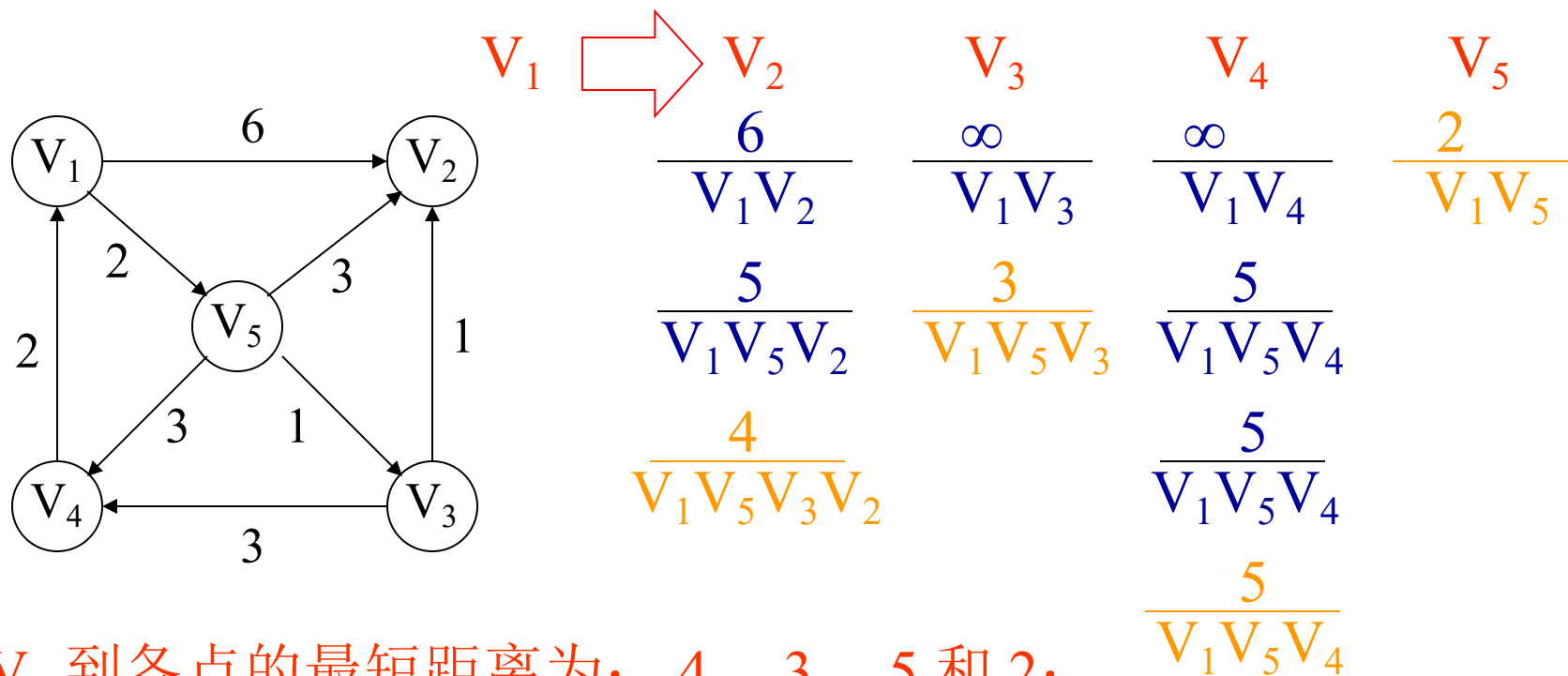
它或者是直接从源点到该点

(只含一条弧)；或者是，

从源点经过已求得最短路径

的顶点，再到达该顶点。

求下图中 v_1 到各顶点的最短路径：



V_1 到各点的最短距离为： 4 、 3 、 5 和 2；

V_1 到各点的最短路径为： $V_1 V_5 V_3 V_2$ 、 $V_1 V_5 V_3$ 、
 $V_1 V_5 V_4$ 和 $V_1 V_5$ 。 该方法的时间复杂度为 $O(n^2)$ 。 其实，
 此法用的也是贪心法。

从另外一个角度理解**Dijkstra**算法

- 最短路径的**Dijkstra**算法的思路和最小生成树的Prim算法的思路很像，两者都是从一个顶点出发，每次增加一条弧并带入了一个顶点，最终生成一棵树：

- **Prim**算法生成一个最小生成树（MST）T

- **Dijkstra**算法生成一个最短路径树（shortest path tree）T

- Prim**算法找的是当前树T外的顶点中，距离整棵树最近的顶点。

- Dijkstra**算法找的是当前树T外的顶点中，距离源点S最近的顶点。

设置辅助数组Dist，其中每个分量Dist[k] 表示 当前所求得从源点 v_0 到其余各顶点 k 的最短路径。

1) 初值: $\text{Dist}[k] = \langle \text{源点到顶点 } k \text{ 的弧上的权值} \rangle$

$$G.\text{arcs}[v_0][k]$$

2) 在所有从源点出发的弧中选取一条权值最小的弧，即为第一条最短路径。假设求得最短路径的顶点为u。

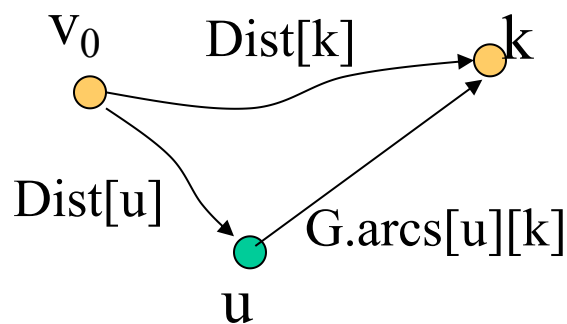
3) 修改其它各顶点的Dist[k]值。

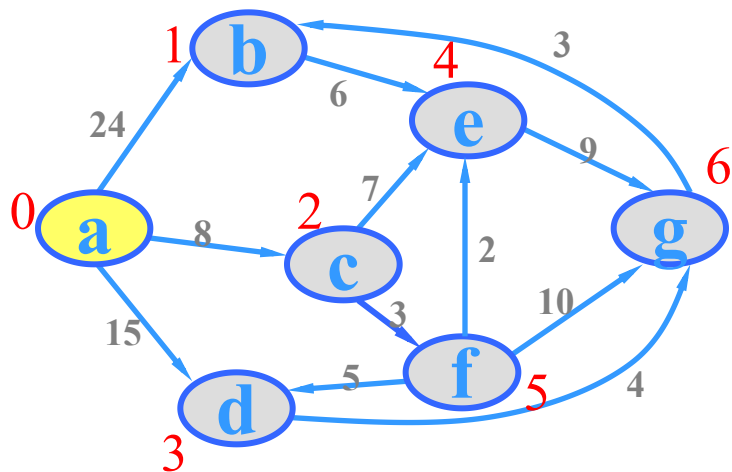
若 $\text{Dist}[u] + G.\text{arcs}[u][k] < \text{Dist}[k]$

则将 Dist[k] 改为

$$\text{Dist}[u] + G.\text{arcs}[u][k]$$

4) 在没有确定最短路径的顶点中选择一条路径最短的，被选中的顶点再设为u，转3)，直到所有顶点被选完。



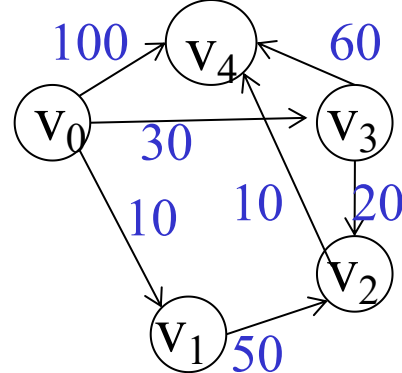


	a	b	c	d	e	f	g
a	0	24	8	15	∞	∞	∞
b	∞	0	∞	∞	6	∞	∞
c	∞	∞	0	∞	7	3	∞
d	∞	∞	∞	0	∞	∞	4
e	∞	∞	∞	∞	0	∞	9
f	∞	∞	∞	5	2	0	10
g	∞	3	∞	∞	∞	∞	0

选出 dist 中的最小值在 $i=1$ ，求得第 6 条最短路径 {adgb}, 顶点 b 并入集合 S

	a	b	c	d	e	f	g	
i	0	1	2	3	4	5	6	S
dist[i]		22	8	15	13	11	19	a c f e
path[i]		adgb	ac	ad	acfe	acf	adg	d g b

[数据结构] 设图中的顶点个数为 n



- 主：邻接矩阵 $G.arcs[n][n]$ (或者邻接表)
- 辅：
 - 一维数组 $S[0..n-1]$ ：记录相应顶点是否已被确定最短距离
 - 一维数组 $D[0..n-1]$ ：记录源点到相应顶点路径长度 *dist*
 - 一维数组 $P[0..n-1]$ ：记录相应顶点的双亲顶点 *Path* 变异

例中各辅助数据结构初值如下：

S:	0	FALSE
		...
	n-1	FALSE

D:	0	0
	1	10
	2	∞
	3	30
	n-1	100

P:	0	-1
	1	0
	2	-1
	3	0
	n-1	0

[算法步骤]

设确定 v 为源点

1)初始化辅助数据结构: ($i=0..n-1$) $O(n)$

1.1) $S[i]=FALSE$;

1.2) $D[i]=G.arcs[v][i]$;

1.3) 若 $D[i]$ 不为 ∞ , 则 $P[i]=v$, 否则 $P[i]=-1$

2)确定源点自身的最短距离: $S[v]=TRUE, D[v]=0$ $O(1)$

3)循环确定其余 $n-1$ 个顶点到 v 的最短距离: $O(n^2)$

3.1)找出 D 中未被确定最短距离的顶点中距离最小的顶点,
设该顶点序号为 k , 且 $D[k]=min$;

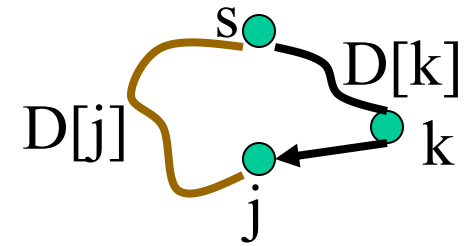
3.2)若 min 为 ∞ , 退出本算法;

3.3)赋值 $S[k]=TRUE$;

3.4)调整尚未被确定最短距离的顶点的估算距离: ($j=0..n-1$)

若 $S[j]=\text{FALSE}$ 且 $D[j]>D[k]+G.\text{arcs}[k][j]$

则 $D[j]=D[k]+G.\text{arcs}[k][j]$, $P[j]=k$



4)依次打印各顶点的最短路径及其距离: $O(n^2)$

($i=0..n-1$)

4.1) `printf(i, D[i]);`

4.2)确定终点i的前趋: $\text{pre}=P[i]$

4.3)若 $\text{pre}=-1$, 则转4)处理下一顶点

否则 `print(pre)`, $\text{pre}=P[\text{pre}]$, 转4.3)

$O(n^2)$

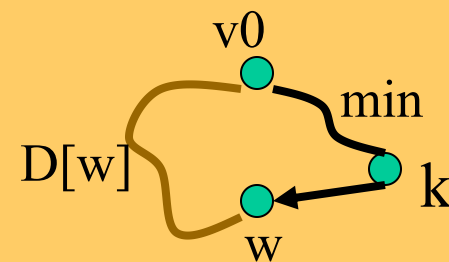
说明: 本算法与教材189页算法7.15使用的P结构不同, 算法7.15不能得到最短路径上顶点的顺序。

[教材189页算法7.15]

```
void ShortestPath_DIJ( Mgraph G, int v0,
                      PathMatrix &P, ShortPathTable &D)
{
    for (v = 0; v < vexnum; v++) {
        final[v] = FALSE; D[v] = G.arcs[v0][v];
        for (w = 0; w < vexnum; w++) P[v][w] = FALSE;
        if (D[v] < INFINITY) P[v][v0] = p[v][v] = TRUE;
    }
    D[v0] = 0; final[v0] = TRUE;
    for (i = 1; i < vexnum; i++) {
        min = INFINITY;
        for (w = 0; w < vexnum; w++)
            if (!final[w] && D[w] < min) { k = w; min = D[w]; }
        final[k] = TRUE;
        /*考虑新加入的节点k */
        for (w = 0; w < vexnum; w++)
            if (!final[w] && (min + G.arcs[k][w] < D[w])) {
                D[w] = min + G.arcs[k][w];
                P[w] = P[k]; P[w][w] = TRUE;
            }
    }
}
```

P	0	1	2		w		n-1
0							
1							
⋮							
⋮							
w					F	F	F
⋮							
⋮							
n-1							F

P矩阵含义：若 $P[v][w]$ 为 true，则 w 是 v_0 到 v 当前求得最短路径中的顶点。

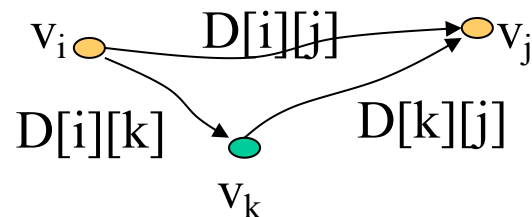


7.6.3 每一对顶点之间的最短路径

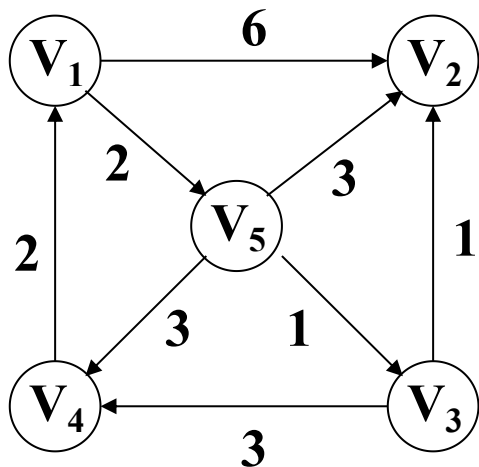
我们可以通过调用 n 次 Dijkstra 算法求的每对顶点间的最短路径，时间复杂度为 $O(n^3)$ 。但还能有其它的求解办法吗？

Floyd 算法 设每个顶点都可以是中间顶点，通过该顶点都有可能使其它顶点间的距离缩短。因此，**将所有顶点都作为中间顶点操作一遍后**，所得到的各顶点对间的距离即是最短距离，所得路径自然也就是最短路径了。该算法是求顶点间最短路径。

Floyd定义了 n 阶方阵序列 $D^{(0)}$ 、 $D^{(1)}$ 、...、 $D^{(k)}$ 、...、 $D^{(n)}$ ，其中 $D^{(0)}$ 为邻接矩阵， $D^{(i)}$ 表示在 $D^{(i-1)}$ 基础上，将 i 点为中间顶点时，各顶点间距离和路径。

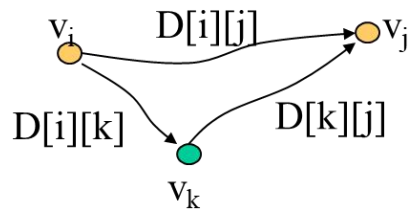


当 $D[i][k]+D[k][j]$ 小于 $D[i][j]$ 时：
用路径 $v_i v_k v_j$ 替代 $v_i v_j$



求左图中各顶点间的最短路径，可先给出其邻接矩阵如右，再逐步测试每个顶点是否可以作为中间顶点降低路径长度：

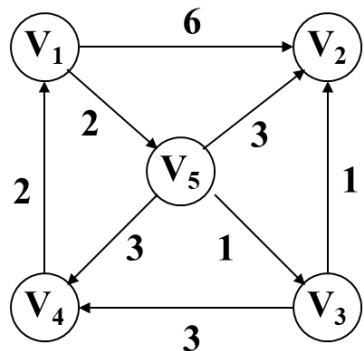
0	6	∞	∞	2
∞	0	∞	∞	∞
∞	1	0	3	∞
2	∞	∞	0	∞
∞	3	1	3	0



$\mathbf{D}^{(0)} =$

$$\begin{bmatrix} 0 & 6 & \infty & \infty & 2 \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 1 & 0 & 3 & \infty \\ 2 & \infty & \infty & 0 & \infty \\ \infty & 3 & 1 & 3 & 0 \end{bmatrix}$$

直达路径



$\mathbf{D}^{(2)} =$

$$\begin{bmatrix} 0 & 6 & \infty & \infty & 2 \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 1 & 0 & 3 & \infty \\ 2 & 8/1 & \infty & 0 & 4/1 \\ \infty & 3 & 1 & 3 & 0 \end{bmatrix}$$

$V_4 \ V_1 \ V_2$
 $V_4 \ V_1 \ V_5$

$\mathbf{D}^{(1)} =$

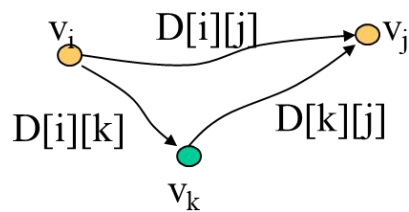
$$\begin{bmatrix} 0 & 6 & \infty & \infty & 2 \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 1 & 0 & 3 & \infty \\ 2 & 8/1 & \infty & 0 & 4/1 \\ \infty & 3 & 1 & 3 & 0 \end{bmatrix}$$

$V_4 \ V_1 \ V_2$
 $V_4 \ V_1 \ V_5$

$\mathbf{D}^{(3)} =$

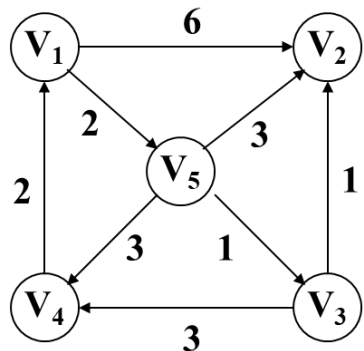
$$\begin{bmatrix} 0 & 6 & \infty & \infty & 2 \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 1 & 0 & 3 & \infty \\ 2 & 8/1 & \infty & 0 & 4/1 \\ \infty & 2/3 & 1 & 3 & 0 \end{bmatrix}$$

$V_4 \ V_1 \ V_2$
 $V_4 \ V_1 \ V_5$
 $V_5 \ V_3 \ V_2$



$$\mathbf{D}^{(4)} = \begin{bmatrix} 0 & 6 & \infty & \infty & 2 \\ \infty & 0 & \infty & \infty & \infty \\ 5/4 & 1 & 0 & 3 & 7/4 \\ 2 & 8/1 & \infty & 0 & 4/1 \\ 5/4 & 2/3 & 1 & 3 & 0 \end{bmatrix}$$

$$\mathbf{D}^{(5)} = \begin{bmatrix} 0 & 4/5 & 3/5 & 5/5 & 2 \\ \infty & 0 & \infty & \infty & \infty \\ 5/4 & 1 & 0 & 3 & 7/4 \\ 2 & 6/5 & 5/5 & 0 & 4/1 \\ 5/4 & 2/3 & 1 & 3 & 0 \end{bmatrix}$$



$V_4 V_1 V_2$
 $V_4 V_1 V_5$
 $V_5 V_3 V_2$
 $V_3 V_4 V_1$
 $V_3 V_4 V_1 V_5$
 $V_5 V_4 V_1$

$V_4 V_1 V_2 V_3 V_2$
 $V_4 V_1 V_5$
 $V_5 V_3 V_2$
 $V_3 V_4 V_1$
 $V_3 V_4 V_1 V_5$
 $V_5 V_4 V_1$
 $V_1 V_5 V_3 V_2$
 $V_1 V_5 V_3$
 $V_1 V_5 V_4$
 $V_4 V_1 V_5 V_3$

各点间最短距离为 $\mathbf{D}^{(5)}$ 所示;

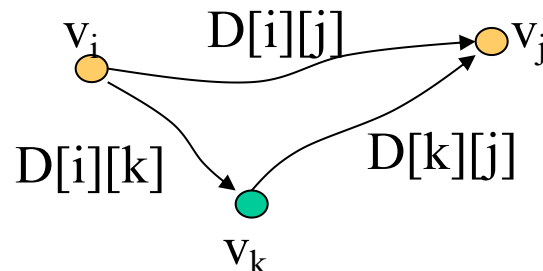
各点间最短路径除标出的外,其余的均是直达路径。

该方法的时间复杂度也为 $O(n^3)$ 。但算法更清晰。

[Floyd算法思想]

- 利用二维数组 $D(0..n-1, 0..n-1)$, $D[i][j]$ 记录当前 v_i 到 v_j 的最短路径长度, 初值 $D^{(-1)}[i][j]=G.arcs[i][j]$
- 集合 S 记录当前允许的中间顶点, 初值 $S=\phi$
 - 依次向 S 中加入 v_0 、 v_1 、...、 v_{n-1} , 这样依次计算 v_0 、 v_1 、...、 v_{n-1} 为中间顶点是的 $D^{(i)}$
- S 中每加入一个顶点, 对 $D[i][j]$ 进行一次修正:
 - 设 $S=\{v_0, v_1, \dots, v_{k-1}\}$, 加入 v_k ,
 - 则 $D^{(k)}[i][j]=\min\{D^{(k-1)}[i][j], D^{(k-1)}[i][k]+D^{(k-1)}[k][j]\}$

$D^{(k)}[i][j]$ 的含义($0 \leq k \leq n-1$):
允许的中间顶点的序号最大为 k 时
从 v_i 到 v_j 的最短路径长度



- 用 $path$ 矩阵记录 ij 两点间的中间顶点序列。

[数据结构] 设图中的顶点个数为n

- 主：邻接矩阵G.arcs[0..n-1][0..n-1]
- 辅：D[0..n-1][0..n-1]：记录最短路径长度
 - path[0..n-1][0..n-1]：记录最短路径，元素为数组类型(线性表)

[算法描述]

```
void ShortestPath_FLOYD1(Mgraph G, PathMatrix &Path,
                          DistanceMatrix &D)
{
    for (v = 0; v < n; v++)
        for (w = 0; w < n; w++) {
            D[v][w] = G.arcs[v][w];
            path[v][w] =  $\phi$ ;
            if (D[v][w] < INFINITY)
                path[v][w] = [v]+[w];
        }
    for (k = 0; k < n; k++)
        for (v = 0; v < n; v++)
            for (w = 0; w < n; w++)
                if (D[v][k] + D[k][w] < D[v][w]) {
                    D[v][w] = D[v][k] + D[k][w];
                    Path[v][w] = Path[v][k]+Path[k][w];
                }
} // ShortestPath_FLOYD1
```

```

void ShortestPath_FLOYD2(Mgraph G, PathMatrix &P,
                        DistanceMatrix &D)
// P[v][w][k]为TRUE, 则从v到w的最短路径中含有k节点
// D[v][w]从v到w的最短路径的长度
{
    for (v = 0; v < n; v++)
        for (w = 0; w < n; w++) {
            D[v][w] = G.arcs[v][w];
            for (k = 0; k < n; k++) p[v][w][k] = FALSE;
            if (D[v][w] < INFINITY)
                P[v][w][v] = P[v][w][w] = TRUE;
        } //for
    for (k = 0; k < n; k++)
        for (v = 0; v < n; v++)
            for (w = 0; w < n; w++)
                if (D[v][k] + D[k][w] < D[v][w]) {
                    D[v][w] = D[v][k] + D[k][w];
                    for (i = 0; i < n; i++)
                        P[v][w][i] = P[v][k][i] || P[k][w][i];
                } //if
} // ShortestPath_FLOYD2

```

本章学习要点

1. 熟悉图的各种存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构和算法有密切联系。

2. 掌握图的两种搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。在学习中应注意图的遍历算法与树的遍历算法之间的类似和差异。

3. 理解最小生成树、拓扑排序及关键路径、最短路径等图应用问题的算法原理和执行过程。